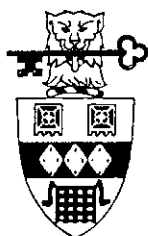


VECTOR

Windows Special ...

- 16-page Educational Supplement 17
- Statistics Library News 33
- Dyalog APL/X Reviewed 66
- Working with Windows 97
- Programming for Events 107
- APL2 and the X Window System 125



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.8 No.1 July 1991

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible. Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, or in workspaces from I-APL, APL*PLUS/PC, APL*PLUS II, IBM APL2/PC or Dyalog APL/386.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1991-92

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£20	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£100	10	5
Overseas Corporate	£155	10	
Sustaining	£430	50	5
Non-voting Student	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £50 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to, Alison Chatterton, whose address is given on the inside back cover.

CONTENTS

EDITORIAL: Windows and APL	Jonathan Barman	Page 3
APL NEWS		
Quick Reference Diary		5
General Correspondence		7
British APL Association News: Summary of Accounts	Nicholas Small	8
News from Sustaining Members	Alison Chatterton	11
The Education Vector	Alan Sykes	17
The Random Vector	David Eastwood	33
REVIEWS		
APL Product Guide	Alison Chatterton	49
SovAPL 5th Anniversary Conference	TM Olsen & A Delmotte	61
Dyalog APL/X	John Searle	66
APL*PLUS II Release 3	Duncan Pearson	73
GENERAL ARTICLES		
Why I Like Windows	Richard & Adrian Smith	78
Integrated System for Demographic Investigations	Victor V. Ponomaryov	85
Experimental Design	Stephen Wynn	88
TECHNICAL SECTION		
Hackers Corner: Proportional Tables in PostScript	Adrian Smith	92
Technical Correspondence		95
Working with Windows	David Eastwood	97
Programming for Events in Dyalog	Peter Donnelly	107
An Interface Between APL2 and the X Window System	JR Jensen and KA Beaty	125
Index to Advertisers		143

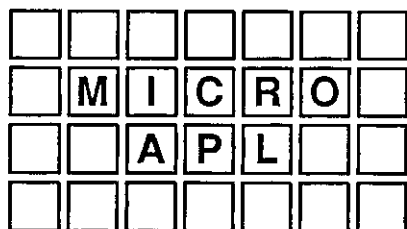
Some things are worth the wait

APL.68000 is not the only APL available for IBM's RISC System/6000. But no other APL running on the RISC System/6000 offers all these features:

- MicroAPL's acclaimed windowing interface
- Seamless integration with X-Windows
- Adherence to OSF/Motif style guidelines
- Very close compatibility with APL2/370
- Binary workspace compatibility across platforms
- High performance through MicroAPL's unique "Portable Assembler"™ technology

APL.68000 for the IBM RISC System/6000

*Fourth to announce.
First to get it right.*



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866
Fax: 071 928 1006

Editorial: Windows and APL

by Jonathan Barman

Windows are wonderful! Most of you must be getting pretty fed up with those sentiments which seem to be getting ever more strident since Windows 3 has sold by the million.

Actually I think that windows are pretty hot stuff. My conversion came about after using OS/2 for about a year. As far as I am concerned the two key ingredients are the consistency of the user interface, and the ability to have several tasks running simultaneously.

The stated advantages of multiple tasks always seemed to me to be rather thin. Constantly switching from one task to another appeared to be a very disorganised way of working. However, having used multiple windows and multiple tasks I have now changed my tune. My basic 'window set' is APL in one window, the favourite text editor in another, a mainframe session in a third, and an appointments reminder running in the background. Hopping about between windows in order to use the best facilities in each seems to speed things up substantially. APL can be used as a super calculator and the results can then be pasted across into the word processor. Of course, if substantial quantities of data need to be transferred then you need functions and files, which comes down to much the same thing as running separate sessions.

The thing that really switches me on is the consistency of the user interface. Window hopping demands consistency. Constantly switching between mainframe and PC applications, where the user interfaces are so very different, is quite difficult to learn and is a bore. Most of the OS/2 applications that I have seen so far are very good, but unfortunately I have come across some which don't stick to the rules.

Applications written in APL need to stick to the rules, as does the APL interpreter itself. Getting the APL interpreter to work with windows is a one time job, and one can only buy a version that has this feature. However, writing an APL application that is properly integrated with windows means that the APL programmer must be able to access all the windowing facilities. There should be no differences in the user interface just because the application has been written in APL. All the little features should be present, the dialogue boxes, standard editing of text, changing fonts, menu bars, buttons, etc. etc.

An APL application simply running in a window is not the same as one which is fully integrated into the environment. If I change the printer driver will the APL application know about it? The APL programmer may have taken great care to create buttons which look and feel like the real thing, but what happens if I change the button colour?

APL interpreters are now able to interface with windows. MicroAPL have had APL integrated into the Macintosh environment for some years. IBM, STSC and Dyalog have all recently announced interfaces with X Windows. We should expect a Windows 3 interface to be announced 'real soon now'. In this issue there are several articles explaining how the interfaces work and how you can use them.

Windows programs seem amazingly complex, with several hundred function calls and as many global variables to set or access. A very low level interface is obviously desirable so that you can poke and tweak exactly what is needed, but APL's interpretive overhead may make it too slow for some of the very low level activities - leave that for C programmers. APL needs a high level interface as well as a low level one.

I have a nasty feeling that every APL supplier will produce radically different high level interfaces. The big problem is that this is a chicken and egg situation. Until a high level interface is available it will be quite difficult to create serious, window-integrated, applications. It will be quite easy to provide APL cover functions to hide the awful window complexities, but I suspect that they may be too slow for live applications. There may be a small band of dedicated enthusiasts, but there probably won't be a large enough group of people with strong views to influence the suppliers before something is actually produced. The APL 'hacker' will not be interested unless there is an interface available which is really easy to use.

This is the first issue of Vector with substantial articles about windows. Hopefully we will be able to have a constant supply of articles to keep you informed about progress on this important topic.

Quick Reference Diary 1991

Date	Venue	Event
4th - 8th August 1991	Stanford	APL91: The Next 25 Years
20 September	IEE London	BAA Meeting
25 October	IEE London	BAA Meeting
22 November	IEE London	BAA Meeting

Starting from February 1991, all British APL Association meetings are held in the IEE, Savoy place. Nearest tube outlets: Temple or Embankment.

Meetings are normally held on the 3rd Friday of the month throughout the autumn and spring.

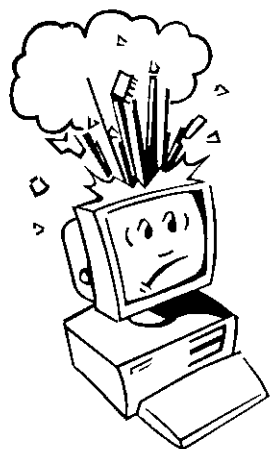
Overseas APL User Groups

The Vector working group is anxious to maintain contact with as many overseas associations as possible. If you are running an APL group and would like to receive Vector in exchange for your own newsletter or journal, please contact Alison Chatterton at the address on the inside back cover. We will endeavour to compile and maintain an accurate list of contacts of overseas groups, and publish this regularly with our APL Product Guide.

Dates for Future Issues of VECTOR

	Vol.8 No.2	Vol.8 No.3	Vol.8 No.4
Copy date	5th Sept 91	1st Dec 91	1st March 92
Ad booking	12th Sept 91	1st Dec 91	1st March 92
Ad Copy	21st Sept 91	12th Dec 91	12th March 92
Distribution	October 91	January 92	April 92

Problems ?



Consultancy

Training

Software

Hardware

Optima

Systems Limited

The Solution !

DSB House, 15A George Street,
Croydon, Surrey, CR0 1LA

Tel: 081-680 5958

Fax: 081-680 5856

Mobile: 0831-228045

General Correspondence

From: Dick Bowman

5 May 1991

The final sentence on page 37 of Vector Vol.7 No.4 contains a couple of inaccuracies which I would like to correct.

Having been part of the ACM SIGAPL Executive Board for almost 4 years now, I can say most emphatically that we consider ourselves to be an international group rather than an American one. We welcome participation from all nations and direct our activities so that the benefits of SIGAPL membership are as widely accessible as they possibly can be.

On the subject of ASL/STATGRAPHICS 'competition' - I can say that when the Board informally considered the possibility of contributing to ASL funding, although this was one of the factors which influenced our decision, it was not the only one.

Dick Bowman
2 Dean Gardens
LONDON E17 3PQ

From: Adrian Smith

5th June 1991

I think someone should take STSC (and in the UK their agents Cocking and Drury) severely to task for making potentially misleading statements in a series of press releases about APL*PLUS/PC release 10. In particular, my attention was brought to this by a banner headline in "PC Shopper" magazine:

APL Under Windows

... reading on I quickly discovered that this referred simply to APL*PLUS/PC, and in my view anyone who bought the product and subsequently found that the documentation quite explicitly stated "This will run under Windows as a non-Windows application" would be justifiably annoyed. You know and I know that APL*PLUS release 9 runs quite happily under Windows, but this surely does not excuse such a blatant attempt to con the public into making yet another unnecessary upgrade.

Adrian Smith,
Brook House, Gilling East, York

British APL Association Summary of Annual Accounts

	1990/91	1989/90	1988/89
	£	£	£
Receipts			
Subscriptions	11787	12341	9835
BCS services	313	599	766
Bank interest	10652	9084	7260
Vector advertising	8331	11546	2107
Conferences and seminars	248	1170	39939
Other	782	87	38
	-----	-----	-----
Total receipts	32113	35226	59945
	-----	-----	-----
Payments			
Meetings	2223	3661	1194
Administration	4510	4331	2839
BCS services	313	599	766
Outstanding Achievement Award	500	500	500
APL Statistical Library	2738	3196	-
Vector production and despatch	12308	10890	10090
Education Vector supplement	4444	3091	1492
Conferences and seminars	3358	-	40095
Other	1366	337	1181
	-----	-----	-----
Total payments	31761	26646	58157
	-----	-----	-----
Assets summary:			
Bank and other balances	81121	80769	72189
Debtors	2928	6984	13145
Creditors	(1046)	(642)	-
	-----	-----	-----
Net assets	83003	87111	85334
	-----	-----	-----

Notes to the Accounts

1. The pence figures have been omitted, so columns may not add exactly.
2. Debtors are mainly advertising and subscriptions.
3. Creditors are mainly advance subscriptions, unpaid bills, and VAT.
4. The value of stocks of Vector and the Software Library have not been assessed.

Treasurer's Remarks on the Annual Accounts.

The Association's overall bank balance has increased over the last two years. As the assets summary shows, this is largely the result of a substantial decrease in the level of outstanding debts, chiefly advertising revenue. Payments for advertising may now be considered to be up to date, so this will not be a factor in future. However, advertising revenue will continue to fall because the number of advertisements placed in Vector has halved over the last two years. This may be due in part to failure to meet publication schedules - advertisers please note that Vector now regularly appears on time!

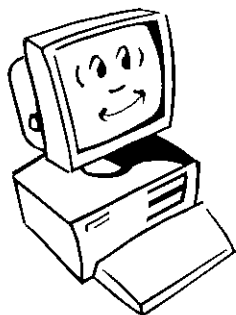
High interest rates have contributed significantly to income recently, but can be expected to be less important in the present financial year.

Administration costs were higher in 1990/91 only because of exceptional costs incurred in sorting out the accounts for 1989/90, and may be expected to fall.

Following the change of venue from the Royal Overseas League to the IEE, there has been a considerable reduction in the cost of meetings and there will be a further reduction in the annual cost of meetings this year.

As has been explained elsewhere, the increase in subscription rates approved at the extraordinary general meeting in March was proposed by the committee so that running costs would be met by subscriptions and advertising revenue, leaving our reserves to be available to spend on projects that will promote the use of APL or otherwise benefit the APL community. The committee would welcome suggestions for suitable projects.

Software ?



STSC - APL*PLUS PC

Full kit - Release 10.....	£399
Upgrade r9 - r10.....	£165
Upgrade r8 - r10.....	£195

STSC - APL*PLUS II

Full kit - Release 3.....	£999
Upgrade r2 - r3.....	£290
Upgrade r1 - r3.....	£320

Dyadic Systems Limited

Dyalog APL PC 386.....	£999
------------------------	------

MicroAPL Ltd

APL 68000 Level 1.....	£87
------------------------	-----

Please call for details on these and other APL products

Optima

Systems Limited

The Solution !

All the above are supplied to our standard terms and conditions.
Prices, specifications and availability may change without notice.
Prices exclude V.A.T.
All trademarks acknowledged.

DSB House, 15A George Street,
Croydon, Surrey, CR0 1LA

Tel: 081-680 5958

Fax: 081-680 5856

Mobile: 0831-228045

News from Sustaining Members

Compiled by Alison Chatterton

Dyadic Systems Limited

Dyadic is pleased to announce an implementation of Dyalog APL for Windows 3.0. The new product is called Dyalog APL/W and is based on Dyalog APL/X for the X Window System. Dyalog APL/W is written specifically for Windows 3.0 and makes full use of the Windows environment. It is a true Windows 3.0 product and cannot be run under DOS alone.

Like Dyalog APL/X, this new implementation will provide the APL developer with the facility to concurrently edit/view functions and variables in separate windows on the screen. The product also features the unique Dyalog APL debugger which lets you step through an application watching the functions pop up and down in windows as they run. Dyalog APL/W will include a means to invoke Windows menus, dialog boxes, scroll bars and so forth in end-user applications. It will also provide DDE support.

In addition to a Windows 3.0 environment, Dyalog APL/W requires a 386 or 486 processor with (ideally) 4Mb+ of memory. On a 386-based system, a 387 coprocessor is strongly recommended. The product runs as a protected mode application in Windows 386 enhanced mode. This means that it can support very large workspaces and even utilise Windows demand paging if required.

Early trials (and this announcement was written in May) indicate that performance will be every bit as good as Dyalog APL for DOS/386.

A detailed specification for Dyalog APL/W will be available at APL91, or can be obtained by post from Dyadic or any Dyalog APL distributor.

As an Authorised IBM RS/6000 VAR, Dyadic is pleased to report significant price reductions, and improvements in performance and specification across the range. The new Model 320H now comes as standard with 16Mb RAM, 400Mb disk and offers 38 MIPS and 11.7 Mflops performance. Based on this model, a single-user workstation with a 16" colour 1280x1024 display, X-Windows, Motif, and Dyalog APL costs around £16,500. A 10-user Dyalog APL system, using ASCII terminals or PCs, can be put together for under £25,000.

Cocking & Drury Ltd

First of all we'd like to apologise to you all for our non-appearance in the last issue. We had anticipated a lot of the news in the issue before, but we should have tried a bit harder to dream something up! We have a lot more to tell you this time around though.

Many of you will have received our publication 'Upgrade' around mid-June. This publication, which used to be a regular feature in Cocking & Drury activities, lapsed some years ago and we were pleased to be able to re-instate it. The 8 page issue contained news of developments at C&D, developments with APL*PLUS, a user-story from London Life, a technical article by Allan Gay on migrating from APL*PLUS PC to the II System for the 386 and more.

Anyone who didn't get a copy but would like one - give us a call.

At the time of writing (early June) we are deep in the throes of preparing for our presence at Software Tools '91. We shall be showing APL*PLUS PC, II System for the 386 and II System for UNIX. (We shall also be showing the full range of Smalltalk/V products - but that's another story for someone else's newsletter!)

The launch of the new APL*PLUS II for UNIX for the IBM, SUN and DEC RISC machines has gone really well. James Wheeler - Vice President of STSC in charge of development - did a presentation in our training room to many of our leading customers the day before his presentation to the BAA.

That - and our marketing work and several mentions in the computer press - has led to a steady stream of enquiries and all our demonstration copies are currently out on loan which bodes well for future sales.

James also gave us an insight into future developments for APL*PLUS. DOS-based machines remain dominant, in their view, with the 32-bit architecture becoming standard but with an increasing interest in extending DOS with Windows - and in UNIX workstations.

STSC strategies will be to concentrate R&D on 32-bit software, exploiting Windows in DOS and X Windows in UNIX. There will be considerable improvements in interfaces to non-APL software - Database managers, graphics, SQL-based client-server databases over networks etc. But more importantly the future of APL*PLUS will include the ability to call APL from other programmes.

STSC have clearly committed themselves to developing APL*PLUS to interface with the various GUI's and to achieving the highest level of compatibility

between them. The result will inevitably be to achieve even more sales for APL*PLUS in the future.

We at Cocking & Drury are delighted to be involved with these developments. We are currently celebrating the most successful year for software sales ever - having set ourselves the target of £500,000 from APL*PLUS products in 1991 we are happy to report that 21 weeks into the year we are ahead of target by over 10% and are now wondering if we can make it £600,000 for the year! We'll let you know.

Our course programme is busy - in the weeks surrounding the writing of this news we shall have given 3 courses in Ireland and one on the Isle-of-Man. Our course tutors are becoming well-travelled.

On the consultancy front we can report that we recently started work for a first-time client, not new to APL but new to us, and that hasn't been a common occurrence in recent months. There are several similar contracts currently under negotiation, as well as new contracts with existing customers, and we have job offers out to two consultants with more to follow. More news on that next time.

MicroAPL

In Vector 7.1, exactly one year ago, MicroAPL announced the launch of our second generation APL - APL.68000 Level II. In this issue of Vector, we hope to do even better. We have two major product announcements to include, one of which signals an entirely new area of operation for MicroAPL.

First of all we have completed work on the first significant upgrade to APL.68000 Level II. Version 2 of APL.68000 Level II removes most of the remaining differences between APL.68000 and IBM's APL2/370. APL.68000 Level II has always been much closer in its specification to APL2/370 than any other APL, even IBM's own APL2/PC, and we have now further narrowed the gap. At the same time we have managed to include some dramatic optimisations in certain key areas of the interpreter.

The main features in Version 2 of APL.68000 Level II include format-by-example; axis on scalar functions; extensions to selective assignment; N-wise reduction; partition with axis. At the level of system commands and system functions we have now implemented the name sorting and string range rules for system commands and we have implemented)SIS.

The optimisations we have implemented focus on simple cases of some of the most widely used functions. For example, scalar integer arithmetic (e.g. +- [l)

has been substantially improved. Catenation operations with simple vectors have been enhanced as have indexing operations with integer indices. The effect of these improvements can lead to marked speed-ups - STSC's benchmark 4 (Mixed functions) is some 2.75 times faster! The final optimisation of note is with respect to pick selective specification where, in some cases, speed-ups of two orders of magnitude have been achieved.

Version 2 of APL.68000 Level II is, however, just the warm-up act to our main product launch of 1991. At APL91 in early August, MicroAPL will be launching the first of a whole new series of implementations of APL.68000. Since early this year we have been working on porting the source code for APL.68000 onto a wide variety of RISC systems (Reduced Instruction Set Computers) and our first release is for the IBM RISC System/6000 family of computers.

The RISC System/6000 is making dramatic inroads into the workstation and supermicro market and here at MicroAPL we felt it to be an appropriate first machine to support. APL.68000 running on the RISC System/6000 will bring with it a number of powerful benefits for APL users. On the one hand we have based our product specification on IBM's APL2 specification (ensuring compliance with large system APLs). On the other hand we have included a number of features in our APL.68000 which are of major importance to the user. There are a whole host of APL language features to complement and enhance the APL2 specification. APL.68000 on the RISC System/6000 will offer a fully thought out and fully developed windowing interface for APL programmers to use. The five years that we have spent developing windowing interfaces on machines like the Mac and the Amiga have allowed us to get the interface right without the burden of all sorts of inappropriate hang-overs from previous implementations.

So what will APL.68000 for the RISC System/6000 include? Firstly it will support either dumb terminal connection or X-terminal connection. The windowing interface will use OSF/Motif and will include all the sorts of features that our Mac users are used to. We are also including on-line documentation in the shape of the entire APL.68000 manual. As usual we will not be subscribing to the theory that binary compatibility is impossible. Any APL.68000 file or workspace from any other version of APL.68000 will be directly usable on the RISC System/6000.

APL.68000 for the RISC System/6000 will be launched at APL91.

On a closing note, our many Atari ST users will be pleased to know that we have completed and are now shipping the upgrade to APL.68000 on the ST. All registered users will have been contacted by post.

Report of the Dutch APL Users Group

In Vector 7.3 the Dutch APL Users Group reported the start of the APL ShareWare Initiative (ASWI). The recent activities of ASWI were as follows:

1. The first activity of ASWI was to decide about the hardware platform and for which APL system the software should be developed. The result of the first discussion was as follows:

	Hardware	Software
primary target	PC's	APL2/PC
secondary target	Atari Mac	APL.68000 II

2. The next activity was to establish a set of standards about coding and documentation. The result of this has been submitted for publication in Vector.
3. The third activity will be to compile a priority list of useful programs and subsequently their specifications.

On Saturday April 6th the Dutch APL Users Group organised a fair at the OASIS premises in Nieuwegein.

A number of microcomputers and workstations were installed with all available APL interpreters such as APL.68000 level I and II, IBM's APL2/PC, Dyalog APL/386, Dyalog APL/X, APL*Plus/PC and APL*Plus II. Various people showed their products and their applications (in alphabetical order):

- Eke van Batenburg showed a simulation for biological research in order to answer the biological question whether left-coiled snails can invade a population of right-coiled snails. Furthermore he demonstrated 2 games which were developed for his son in 1 day using APL.68000.
- In Tom Bonnema's booth one could get various public domain APL systems, such as I-APL, TryAPL2, Sharp APL/PC and J.
- M.Bouwma demonstrated a very useful database package for STSC APL. This package used inverted files and contained a number of functions to create on-screen dialogues in an easy way. A substantial amount of programming was invested to ensure the integrity of the database.
- Louis van Geldrop showed a menu system developed for APL2/PC using the new AP124 processor. He also showed 2 packages already developed for ASWI: a component file system, an object library system and a system of relational data functions for arrays.

- Oasis hosted the whole event. The smooth organisation was mainly due to the efforts of the people of Oasis. They received the visitors in a nice and productive atmosphere. Furthermore, Oasis showed various APL interpreters.
- Jean-Jaque Riethoven showed STAR, a package to analyse the folding of RNA strings, using APL.68000.
- Bernard Smoor organised an extensive display of APL literature.
- Fred Visser was present with I-APL together with English and Dutch documentation.

The fair was attended by about 60 persons among whom nine copies of STSC's Pocket APL were raffled by Oasis Decision Support Systems B.V. The greater part of the visitors were well-known with APL, but also some new faces were discovered.

APL People Limited

Things have been very quiet on the recruitment front in the UK for the first half of this year. We now have a lot of excellent 'APLers' looking for either contract assignments or permanent vacancies. Any companies requiring skilled APL personnel to work either here or abroad should contact me straight away! We currently have a wide range of APL consultants and contract programmers available at very reasonable rates!

Surprisingly business is booming for APL People's sister company, APL Manufacturing, whose production control software has recently been sold to companies as diverse as Philips Electronics, Jaguar Cars, Peak Engineering, and Marconi Defence Systems (to mention but a few!)

THE EDUCATION VECTOR

July 1991

Editor Alan Sykes

This Education Vector has been reprinted from VECTOR Vol.8 No.1. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 2 Blenheim Rd, St Albans, Herts AL1 4NR Tel: 0727 860130.

Contents

Editorial	Alan Sykes	18
Letters		19
Let's Integrate - Using APL	Alan Sykes	22
Labelling	Alan Sykes	30

Dr Alan Sykes,
c/o European Business Management School
Swansea University
Singleton Park, Swansea SA2 8PP
Wales, UK

Editorial

Welcome once again to Education Vector. This editorial will be my last as editor of Education Vector. It seems incredible that I have been doing the job for three years now. Tempus Fugit! It is with some regrets that I relinquish my job - on the one hand I have very much enjoyed the challenge of writing articles designed to encourage the use of APL; on the other hand, it does take time, an all too precious commodity. In any case, fresh ideas and enthusiasm generated by change is generally a good thing.

So I take this opportunity to thank all my readers who have expressed appreciation of Education Vector, including those that have sent their contributions. (See for example the Letters section this issue.) In addition I must thank Adrian Smith for his dedication in type-setting so superbly all that I have submitted, not to mention exercising his prerogative, both as type-setter and extraordinarily-capable-APL programmer, to include apposite 'extras' from time to time! Thanks too to all members of the British APL Association for allowing a very amateur APL programmer to benefit from close contact with the APL professionals.

What of the future for Education Vector? First of all its editorship is in the very capable hands of my friend and colleague Alan Mayer (so only half the name has changed in the new editorial address!) Secondly, I am sure the British APL Association will continue to send out copies of Education Vector to those on the mailing list. If you are a reader abroad, wish to continue receiving it, and have not yet written to me, please do so as soon as possible, so that in the near future the Association can up-date its lists accordingly.

If, however you wish to keep up to date with a somewhat wider spectrum of APL activities, may I recommend that you become a member of the Association and receive those 'glossy' quarterly copies of VECTOR (see Letters). (A lot of the content of VECTOR is technical, and perhaps writers of articles for VECTOR ought to be a little more careful to avoid the danger of being unnecessarily exclusive by using too much unexplained jargon. Nevertheless, it does represent remarkable value for money - there is no other APL journal to beat it. In addition the recent introduction of Random Vector, which reports on the APL Statistics Library Project, should be of interest to many readers of Education Vector.)

But back to Education Vector. To fill sixteen pages of interesting APL once a quarter is not an easy task. But it is worthwhile, as many of the letters I have received attest. Please find that extra bit of time to write up your latest APL activities for inclusion in these columns. APL professionals too have their role to play - not by using its columns just to demonstrate how clever they are, but by providing helpful hints on programming methods and idioms, tools for readers to make use of, and in short, encouragement for those of us who are still in the early stages of learning a language that has so much to offer, both in terms of intellectual enjoyment and practical usefulness.

Letters

Since the plea in a previous column for readers (particularly those overseas) to write in to say they wish to continue receiving Education Vector a number of letters came my way which could be regarded as 'fan' mail. Their expressions of interest in these columns have helped to make my time as Editor very rewarding.

From: Julian Vincent, University of Reading

I have been meaning to write this letter some time - I was a member of the British APL Association for 3 years, receiving its very glossy publication Vector, and finding little in it of interest, since I am neither a totally committed APLer nor am I terrifically computer-literate. I hack around in BASIC sufficiently to be able to write routines for analysing data and controlling one or two bits of lab. machinery. However I feel that as a supervisor of research students, and also because I am curious about most things, I should acquire some knowledge of various aspects of science and its analysis. Hence my tangential interest in APL. However, your column was the only bit of Vector which I found to be consistently interesting, amusing and (even!) useful. When I realised that it would be sent to me even if I was not on the mailing list for Vector, I'm afraid I let the Vector find its own way home.

So this letter is by way of fan mail, perhaps to the extent that your little pamphlet is to me the most important bit of APL education I can pick up. Please keep sending it. If you need a subscription in the future, don't be afraid to ask for it!

From: Manfred Storn, Euskirchen-Kirchheim, Germany.

I am a teacher in a secondary school (Hauptschule) in Germany and have been taking computer lessons since 1982.

I first met APL 3 years ago, by accident, while looking for a programming language - something like COMAL on an Atari ST. I tried C and Modula 2, saw MicroAPL and bought it. Usually I get I-APL or TRY-APL for my pupils to effect the first interest for APL which can be handled more or less by intuition and interactive work. Why write pages of code if there is APL available?

We can get a pretty good German translation of Garry Helzer's manual at a low price, so that everybody can afford it. But I have a lot of difficulties writing

complete programs which can be handled by anyone. In most of the literature there are lots of tricky or highly mathematical functions. But what I need are simple menu-structured useful programs.

At the moment the Education Vector seems the only script which can be used in secondary schools without any further adaptation. The programming code is short enough to put it into the computer without getting square eyes!

As a member of the APL-Club in Germany I get the APL-journal, but it isn't meant to be used for introducing APL to pupils. So I have to translate some parts of the Education Vector.

Just a little information about my school: it has 260 pupils in the age range 10-17. Most still use computer C-64 (even at school); there are very few pc's, but a lot of Amigas (for games!) and some Atari STs. Languages preferred are PL-COMAL and German LOGO for the beginner. At the moment there is a trend to let the programs remain as black boxes.

From: C.E. Williams

I enclose a function *MULCURFIT* which allows Multiple Curve Fitting, the fit of x,y data pairs to the four common equations:-

straight line	$y=a+bx$	(no restrictions)
exponential curve	$y=a\exp(bx)$	(y positive)
log curve	$y=a+b\ln(x)$	(x positive)
power curve	$y=a\exp(b\ln(x))$	(x and y positive)

The data is entered only once and retained, being transformed when required. The results are given with the correlation coefficient. This enables the best fit to be readily seen. There are some restrictions since logs are used (see the comments in brackets above). A test set of data pairs is included, together with the results on I-APL on BBC2. I hope it may be of some use to someone out there!

```

V MULCURFIT;A;B;N;MA;MB;N;SX;SY;R;Q;Z
[1] 'INPUT X AND Y VECTORS'
[2] X←□
[3] Y←□
[4] N←ρX
[5] L:'INPUT REQUIRED FORM LIN(I),EXP(P),LOG(G),POW(W),END(E)'
[6] →{(1+0)='IPGWE')/I,P,G,W,E
[7] 'INVALID CHOICE'
[8] →L
[9] I:A←X ◊ B←Y ◊ Z←1
[10] →PRO
[11] 'y=a+bx'

```

I-APL Ltd order form 1 July 1991

Address for enquiries:
2 Blenheim Rd
St Albans
Herts
AL1 4NR

Address for orders:-
56 The Crescent
Milton, Weston-super-mare
Avon
BS22 8DU

Disks are 360K PC disks except where specified. Other versions to special order.

Please supply:

Prices include UK postage

Part	Item	No. of copies	Price	Total
I-APL and related Products				
PC12	PC clone/Nimbus 360K disk & manual	—	£4.50	—
AT12	PC clone/N 720K 3.5" disk & manual	—	£6.50	—
BBC12	BBC 80T disk Master (& B with 32K sideways RAM) & manual	—	£4.50	—
A12	Archimedes disk and manual	—	£4.50	—
M12	Macintosh disk and photocopied manual	—	£8.50	—
T3	Tutorial by Thomson & Alvord (45pp)	—	£2.50	—
E4	Encyclopedia by Helzer (228pp)	—	£5.50	—
S5	APL in Social Studies by Traberman (36pp)	—	£2.50	—
P6	APL programs for maths classroom, Thomson (185pp)	—	£14.50	—
Disks for I-APL				
FEMC	Disk of Functions for Thomson's book by F Espinasse	—	£2.00	—
FEK12	Maths kits 1 and 2 by Francis Espinasse	—	£2.00	—
FEK34	Maths kits 3 and 4 by Francis Espinasse	—	£2.00	—
MK	Morten Kromberg's GRAF workspaces	—	£2.00	—
ZVJ	Zdenek V Jizba's Lessons workspaces	—	£2.00	—
FINN	FinnAPL workspaces	—	£2.00	—
ISI Products				
SAPL	Sharp APL/PC manual [system] (350pp), disks & Tang. Math	—	£42.00	—
APLIPC	Sharp APL/PC enhanced version 20 for PC & AT clones	—	£55.00	—
APLI386	Sharp APL/PC enhanced version 20 for 386/486 in real mode	—	£60.00	—
DAPLIPC	Disks only version of APL/PC	—	£21.00	—
DAPLI386	Disks only version of APLI386	—	£26.00	—
SAR	Sharp APL Ref. Manual [language] (qto 349pp) by Berry	—	£18.00	—
JPC	'J', (Advanced APL dialect): disk, Dictionary & Tang. Math	—	£17.00	—
JMAC	The Macintosh version of JPC above: Mac 3.5" disk	—	£18.00	—
PIJ	Programming in J (inc. Dictionary) by Kenneth E. Iverson	—	£25.00	—
TryAPL2 Many people can get TryAPL2 from IBM free. If YOU can, don't buy from us.				
TRY720	TryAPL2 720K disk & manual (photocopy 41pp)	—	£7.50	—
TRY360	TryAPL2 two 360K disks & manual (photocopy 41pp)	—	£7.50	—
BOOKS				
AAT	Algebra: an Algorithmic Treatment, Iverson (361pp)	—	£6.50	—
SAAT	Solutions to foregoing by J Iverson (42pp)	—	£2.00	—
EA	Elementary Analysis by K Iverson (218pp)	—	£4.50	—
CNK	Calculus in a New Key by D Orth (286pp)	—	£5.00	—
RCT	Resistive Circuit Theory by R Spence (qto 279pp)	—	£7.50	—
TEY	APL Quote-Quad: The Early Years (qto 465)	—	£11.00	—
SBA	A Source Book in APL (qto 140pp)	—	£5.50	—
AI	APL and Insight by Berry et al. (89pp)	—	£2.50	—
FCP	The Four Cube Problem by McDonnell (qto 27pp)	—	£2.50	—
SM	Star Map by Berry & Thorstensen (41pp)	—	£2.00	—
PA	Probability in APL by L Alvord (120pp)	—	£3.50	—
IA	Introduction to APL by K E Iverson 1984 (110pp)	—	£4.00	—
AL	APL Language Reference for foregoing (128pp)	—	£5.00	—
IAT	Introducing APL to Teachers (25pp) by Iverson	—	£2.00	—
IASE	Intro to APL for Scientists & Engs, Iverson (26pp)	—	£2.00	—
AE	APL in Exposition by K E Iverson (61pp)	—	£2.00	—

Please add £2.00 per parcel for packing

£ 2.00

VAT Regd No 478 1634 13 (VAT @ 17.5% is 30 pence per disk: books are zero-rated)

Total

£

Name _____ (please write legibly)

Address _____

Please pay "I-APL Ltd" Note that
we do not accept credit cards;
please send cheque with order.

Postcode _____

OFFICE USE

--	--	--	--	--

I-APL News for Education Vector 8.1

1 Archimedes 32-bit version - I-APL/32

Paul Chapman has finished it! (He says I can't write a paragraph without exclamation marks and brackets). Although the precision and accuracy of the arithmetic is the same as the previous version, this version gives access to all the available memory so the maximum workspace size is within a quarter megabyte of the memory installed. The disk and notes on differences will be supplied to people who have bought the previous version for £2 plus the packing charge *from the enquiry address only*. It includes a version of DFILE.IWS which will read or write any size of file; previous versions of I-APL only give access to the first 32K. It also includes mechanisms to convert I-APL workspaces into I-APL/32 workspaces and vice versa. All the other facilities of I-APL are available in the 32-bit version. We do not plan any other 32-bit versions of I-APL.

2 ISI Product Changes

ISIAPL01, ISIAPL02 and ISIAPL03 have been superseded by new products. Instead of upgrades to Sharp APL/PC these are each complete products. None of them is shareware. They each include the Sharp APL/PC system documentation, but if you already have this then you may buy disks only without documentation which is much cheaper. The advantage of the new product range is that the purchaser no longer has to carry out any upgrade process. The new product runs straight out of the wrapping. A minor disadvantage is a slight increase in cost as now what you get is three disks in the set instead of a single disk.

~~Sharp APL/PC is the same~~ mainframe version 17 with 370 emulator as before. The interpreter is shareware and is available on two disks from any owner of a copy. Its enormous range of features and facilities make it impractical for anyone but an expert to get the best out of it without the system manual.

APLI/PC is version 20 of mainframe Sharp APL. It supports APL characters on CGA, EGA and VGA. It runs on 8088, 8086 and 80286 machines and the largest workspace possible, with no shared variable processors and the minimum use of DOS memory, is 280K. There is a simplified session manager and editor and there is a simplified asynchronous comms workspace. The FSCR screen workspace is fast. There is an AP124 screen manager. There is a character mapping option which uses only the standard IBM PC character set. This can be used when there are no APL characters available or to avoid CGA graphics which makes FSCR and the editor unuseable.

APLI386/PC is version 20 for 80386 or 80486 machines only. It runs in real mode and requires an XMS driver. It gives all the APLI/PC facilities and runs about 40% faster than Sharp APL/PC. There is a SHELL facility which can run DOS commands and programs in the 640K DOS memory when the interpreter and workspace are in extended memory. Most of the other options of APLI/PC are available.

Both enhancements give colour, and, if you have VGA, function key flip between 25 and 50 row screens. Both drop some Sharp features for the sake of speed (such as the date and time stamp on file components and numeric arguments to `⎕arbin` and `⎕arbout`). Both require extra care with the DOS Shared variable processor.

Anthony Camacho 8 July 1991


```

[12] 'a=',vQ
[13] 'b=',vM
[14] 'r=',vR
[15] →L
[16] P:A+X ◊ B+Y ◊ Z+2
[17] →PRO
[18] 'y=aexp(bx)
[19] 'a=',v*Q
[20] 'b=',vM
[21] 'r=',vR
[22] →L
[23] G:A+X ◊ B+Y ◊ Z+3
[24] →PRO
[25] 'y=a+bln(x)'
[26] 'a=',vQ
[27] 'b=',vM
[28] 'r=',vR
[29] →L
[30] W:A+X ◊ B+Y ◊ Z+4
[31] →PRO
[32] 'y=aexp(bln(x))'
[33] 'a=',v*Q
[34] 'b=',vM
[35] 'r=',vR
[36] →L
[37] PRO: MA+{+/A}+N
[38] MB+{+/B}+N
[39] SX+{(+/A-MA)*2)+(N-1))* .5
[40] SY+{(+/B-MB)*2)+(N-1))* .5
[41] M+{(B+.XA)÷N)-(MA×MB)}÷{(SX*2)×(1÷N)}
[42] Q+MB-M×MA
[43] R+(M×SX)÷SY
[44] →(Z=1)/11
[45] →(Z=2)/18
[46] →(Z=3)/25
[47] →(Z=4)/32
[48] E:→0
v

```

```

X←10 20 40
Y←.1007 .1790 .2960

```

LINEAR:	a=.0064157	b=.042199	r=.99709
EXP:	a=.078308	b=.034398	r=.97393
LOG:	a=-.230136	b=.140879	r=.99351
POWER:	a=.01700	b=.777766	r=.99925

(Editor's comment: I hope I have typed the program correctly together with one or two minor cosmetic changes. It is good to see this sort of program being produced and indeed used.)

Let's Integrate - Using APL

by Alan Sykes

Introduction

The great beauty about APL, and hence its great value to teachers, is the ability to do such a lot with a few small programs. A few simple tools provided for eager students to use can give as much stimulus as some very exotic teaching software. The express aim of this article is to do just that, choosing the topic of integration. No matter how rusty our Mathematics is, most of us can remember what integration is all about - 'areas under curves' is an adequate description for most purposes.

Where to start? Well, since everything is about areas under curves, and since the simplest area to calculate is that of a rectangle, a useful tool is to be able to specify a set of contiguous intervals (each interval of which will support a rectangle of known height).

A GRID Function

As in one of my previous articles, design is quite important. I require here a function, call it *GRID*, which will calculate the end-points of *N* sub-intervals of a specified interval [a,b].

```

      ∇ R←GRID A_B_N;A;B;N;∅IO
[1]   ∅IO←1 ∘ A+A_B_N[1] ∘ B+A_B_N[2] ∘ N+A_B_N[3]
[2]   R←A+(÷N)×(B-A)×0,∅N
      ∇

```

```

      GRID 0 10 10
0 1 2 3 4 5 6 7 8 9 10

```

```

      GRID 0 2 20
0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1 1.1 1.2 1.3 1.4 1.5 1.6
  1.7 1.8 1.9 2

```

Note that the last of the three arguments supplies the number of intervals - not the number of points. Of course this simple function could be made more robust if required - obvious design features might include:

- (a) testing that the right argument is numeric
- (b) testing that the right argument has three elements
- (c) testing that $A \leq B$ and that N is a positive integer.

Although this aspect is not central to the theme of this article, some readers may find it instructive to see just what is involved in providing such error traps. Perhaps the easiest way to do this is to use 'execute' as I suggested in my previous article. This is how *GRID* might be expanded to include the above suggestions.

```

      V R+GRID A_B_N;A;B;N;IO
[1]  A_B_N←,A_B_N a force right argument to be a vector
[2]  ⚡(0≠1+0pA_B_N)/''Right Argument must be numeric''⚡→0'
[3]  ⚡(3≠pA_B_N)/''Right Argument must have 3 elements''⚡→0'
[4]  IO+1 ⚡ A+A_B_N[1] ⚡ B+A_B_N[2] ⚡ N+A_B_N[3]
[5]  ⚡(A>B)/''Interval mis-specified''⚡→0'
[6]  ⚡((N≠N)∨N≤0)/''Number of Intervals must be a
      positive integer''⚡→0'
[7]  R←A+(÷N)×(B-A)×0,1N
      V

```

Having re-written the function with error trapping, it all has to be tested!

```

      GRID 'FRED'
      Right Argument must be numeric
      GRID 2 2p1 2 3 4
      Right Argument must have 3 elements
      GRID 2 1 4
      Interval mis-specified
      GRID 1 2 0
      Number of Intervals must be a positive integer
      GRID 1 2 10.5
      Number of Intervals must be a positive integer
      GRID 0 2 4
      0 0.5 1 1.5 2

```

(Readers who have not gone through this exercise may now realise why it is so easy to crash software. There is no particular difficulty in error-trapping individual functions, but some tests could be easily left out in what is clearly a laborious process.)

Applying *GRID*

Back to our main theme - integration. As an illustration of how effective just this one tool is, let's take a simple function to integrate - say the function *FUN* written

below, given by the expression $x \exp(-x)$, or in APL, $X \times * -X$, and integrate it from 0 to 2

```

      ▽ R←FUN X
[1]   R←X×*-X
      ▽

```

As it happens, my integration is just about up to this calculation (making numerical integration redundant, which however is not the point) and the answer is given by $[-x \exp(-x) - \exp(-x)]$ evaluated at 2 and 0. Hence the correct answer is $1 - 3 \exp(-2)$

```

      1-3×*-2
0.5939941503

```

As a simple approximation to the required integral, the range of integration may be divided into ten intervals and the function evaluated at all these points:

```

      GRID 0 2 10
0 0.2 0.4 0.6 0.8 1 1.2 1.4 1.6 1.8 2

      FUN GRID 0 2 10
0 0.1637461506 0.2681280184 0.3292869817 0.3594631713
  0.3678794412 0.3614330543 0.3452357495 0.3230344288
  0.2975379988 0.2706705665

```

giving the heights of the rectangles. Hence an approximation is the sum of the heights times the interval widths:

```

      .2×+/FUN GRID 0 2 10
0.6172831122

```

Trying finer grids will of course give better approximations:-

```

      .02×+/ FUN GRID 0 2 100
0.5966630121

      .002×+/ FUN GRID 0 2 1000
0.5942644424

```

Given knowledge about the behaviour of the function (increasing from 0 to 1 and then decreasing from 1 onwards) grids can be chosen (always to include the turning point $x=1$ as an end-point) so that upper and lower bounds can be obtained simply by checking which end-point of each interval has the smaller (larger) function value. The easiest way to do this is to turn the result of *GRID* into a two column matrix, each row specifying the end-points of each interval.

```

PP←4
MAT←GRID 0 2 10
MAT←(¯1+MAT),[1.5]1+MAT
MAT      one row for each interval
0      0.2
0.2 0.4
0.4 0.6
0.6 0.8
0.8 1
1      1.2
1.2 1.4
1.4 1.6
1.6 1.8
1.8 2
          FUN MAT
          0      0.1637
          0.1637 0.2681
          0.2681 0.3293
          0.3293 0.3595
          0.3595 0.3679
          0.3679 0.3614
          0.3614 0.3452
          0.3452 0.323
          0.323 0.2975
          0.2975 0.2707

```

*APL and its
array operations!*

```

.2×+/[/FUN MAT      Calculate Maximum of each row and use
0.6367

```

```

.2×+[/\FUN MAT      Calculate Minimum of each row and use
0.5437

```

Hence the integral lies between .5437 and .6367. Now to close in on it by repeating the operation with a finer grid. But first, note that really I have been rather naughty in applying *FUN* after creating *MAT* rather than creating *MAT* after using *FUN* on the result of *GRID*. So, subsequently, I shall reverse the order of operations. (To do so becomes important if the function to be integrated is complex and evaluations take a long time.)

```

MAT←(¯1+MAT),[1.5]1+MAT+FUN GRID 0 2 100
.02×+/[/MAT
0.5986
.02×+[/\MAT
0.5893

```

Hence the integral lies between .5893 and .5986. Once more, take a finer grid:

```

MAT←(¯1+MAT),[1.5]1+MAT+FUN GRID 0 2 1000
.002×+/[/MAT
0.5945
.002×+[/\MAT
0.5935

```

So with one useful function and a few APL array operations, it is possible to explore the very foundations of integration, whereby the existence of the integral is proven by a pincer movement from above and below. (There are some bizarre mathematical functions for which this process fails!)

The Trapezium Rule

After the theory of integration, then consider some more simple techniques of numeric integration. The first method used above, merely approximated the function using a step-function - i.e. lots of horizontal bars. Since most functions integrated in practice are smooth, it seems reasonable to approximate by connecting the function points on the grid with straight lines. This leads to using the trapezium rule to evaluate the area of each trapezium standing on each interval. Each such trapezium has area equal to the average of the heights times the width. If this is applied to all of them, then a formula for the integral is:

$$\text{interval_width} \times [(0.5f(a) + f(a+h) + f(a+2h) + \dots + 0.5f(b))]$$

With a view to the next method, rather than taking the sum of all the f values and subtracting .5 times the sum of the first and last values, we will generate a vector of coefficients .5 1 1 1 15 using the expression .5, ((N-1) ρ 1), .5. This can then be multiplied and added to the f values using +.×:

```
.02×(.5,(99ρ1),.5)+.×FUN GRID 0 2 100
0.593956
```

This is the basis of the trapezium rule, which can now be incorporated into a function. To do so raises the interesting question of how to specify the function to be integrated. The only way possible with first-generation APL interpreters is to pass in the name of the function as a string variable. Consequently, the execute function (see Education Vector April 1991) has to be used.

There is however a problem with passing function names into an APL function, namely that care must be taken to make sure that no local variable names can coincide with the function name passed in. (If such a clash does occur, then what is executed is the LOCAL variable.) A common way of avoiding such a pitfall is to use variable names using Δ or Δ. So with this in mind, our integration function will have as its left argument the name of the function, and as its right argument the parameter of the GRID function to be used.

Here is the function TR_INT. The key part is line 3 to the right of Δ, where a string vector is constructed consisting of the function name, followed by a space (very important) and the GRID command, all within quotes.

```

      ▽ R+ΔFN TR_INT ΔA_B_N;ΔA;ΔB;ΔN;ΔWT
[1]   ΔA+ΔA_B_N[1] ◊ ΔB+ΔA_B_N[2] ◊ ΔN+ΔA_B_N[3]
[2]   ΔWT+0.5,((ΔN-1)ρ1),0.5
[3]   R+(÷ΔN)×(ΔB-ΔA)×ΔWT+.×ΔFN,' GRID ΔA_B_N'
      ▽

      'FUN' TR_INT 0 2 100
0.593956

      'FUN' TR_INT 0 2 1000
0.593994

```

Simpson's Rule

So far so good: one top level function using a suitable tool. What next? Well, as the trapezium rule supersedes the simple step-function approximation, so Simpson's rule which approximates a curve by fitting quadratic curves to each interval normally supersedes the trapezium rule. The coefficients multiplying the end-points of an assumed even number of intervals are now 1 4 2 4 2 4 2....4 1, divided by 3 - otherwise the function is very similar - I have called it *SMP_INT*.

```

      ▽ R+ΔFN SMP_INT ΔA_B_N;ΔA;ΔB;ΔN;ΔWT
[1]   ΔA+ΔA_B_N[1] ◊ ΔB+ΔA_B_N[2] ◊ ΔN+ΔA_B_N[3]
[2]   ΔWT+1,((ΔN-1)ρ 4 2),1
[3]   R+(÷3×ΔN)×(ΔB-ΔA)×ΔWT+.×ΔFN,' GRID ΔA_B_N'
      ▽

      'FUN' SMP_INT 0 2 100
0.593994

```

Note that this is accurate to the number of decimal places shown! Note also that in some cases it is possible to evaluate the integral required by supplying a definition, for example:

```

      'X*-X+' SMP_INT 0 2 100
0.593994

```

Here is another example using the circular functions. First of all, multiples of pi are given by 0 right argument. Then 10 right argument specifies the sine function (operating on angles specified in radians) and 20 right argument gives the cosine function.

01	(pi)
3.14159	
02	(2×pi)
6.28319	
100.5	(sine of pi÷2)
1	

```

      200.5                                (cosine of pi/2)
6.13658E-17
      '10X+' SMP_INT 0,(01),100  (integral of sine from 0 to pi)
2

```

For a final example, consider calculating areas under the Standard Normal probability density function. First to define *NORMDEN*:

```

      V R+NORMDEN X
[1]   R+((02)*-0.5)*-0.5*X*X
      V

```

Hence, to calculate the area under the curve from 0 to 1:

```

      'NORMDEN' SMP_INT 0 1 100
0.341345

```

and so the area under the curve corresponding to mean plus or minus one standard deviation is approximately 68%. Similarly,

```

      'NORMDEN' SMP_INT 0 1.96 100
0.475002

```

thus the area to the right of 1.96 is .5-0.475, or .025

Producing Normal Distribution Tables

With a little program to loop around an array of appropriate values, tables of the Standard Normal distribution can be constructed easily, though it may take some time, depending on the speed of your computer. Here is such an example:

```

      V R+DIM NORMTAB G;C
[1]   R+(pG)p0 0 C+1
[2]   lp:R[C]+0.5-'NORMDEN' SMP_INT 0,G[C],100
[3]   +((pG)≥C+C+1)/lp
[4]   R+DIMpR
      V

```

As an example of its use consider the rather coarse grid of values from 0 to 2.8 in steps of .2

```

      R+□+3 5 NORMTAB ~1+GRID 0 3 15

0.5      0.4207      0.3446      0.2743      0.2119
0.1587    0.1151    0.08076   0.0548      0.03593
0.02275   0.0139    8.198E-3  4.661E-3  2.555E-3

```


With the help of some labelling programs (see the separate article) a table suitable for printing can easily be produced.

```
'Z value/.0/.2/.4/.6/.8/' CLABEL '0/1/2/' RLABEL 'R
```

Z value	.0	.2	.4	.6	.8
0	0.5	0.4207	0.3446	0.2743	0.2119
1	0.1587	0.1151	0.08076	0.0548	0.03593
2	0.02275	0.0139	8.198E-3	4.661E-3	2.555E-3

These calculations yield results which agree with commercially available tables to four decimal places.

Monte Carlo Simulation for Integrals

Finally in this article, consider the possibility of Monte Carlo Simulation to evaluate integrals. This is a technique of sampling the function f to be integrated uniformly at random throughout the range of integration and calculating the resulting sample mean. This is an approximation to the expected value of the random variable $f(X)$, (where X has a Uniform distribution on $[a,b]$) which in turn differs from the integral required by a factor $(b-a)$.

This approach is easily done in APL with the aid of a few more tools. First of all, generate values uniformly at random through the interval a to b :

```
▽ R←N UNIFORM A_B;A;B;T
[1] A←A_B[1] ⋄ B←A_B[2] ⋄ T←10000000000
[2] R←A+(B-A)×(0.5+?NρT)÷T
▽

10 UNIFORM 0 2
1.9156579 1.192201 0.097855169 0.015331741 0.50314947
0.19924829 1.4752251 1.70138 1.8358242 0.75769372
```

Add to the toolkit a program to calculate the mean:

```
▽ R←MEAN X
[1] R←(+/X)÷ρX
▽
```

Now calculate twice the mean of a set of function values (twice because the interval is of length 2):

```
2×MEAN FUN 100 UNIFORM 0 2
0.5628702
```

Again, such a procedure can be called from a program, called *MC_INT*

```

      ▽ ΔR+ΔFN MC_INT ΔA_B_N;ΔA;ΔB;ΔN
[1]   ΔA+ΔA_B_N[1] ◊ ΔB+ΔA_B_N[2] ◊ ΔN+ΔA_B_N[3]
[2]   ΔR+(ΔB-ΔA)×MEAN±ΔFN, ' ', (▽ΔN), ' UNIFORM ΔA_B_N[1 2]'
      ▽

      'FUN' MC_INT 0 2 1000
0.58683034

```

Of course, repeating the same command will give different estimates of integral. Moreover if you play around with it you will soon see that you have to use a lot of values to get a decent accuracy. So the technique is not used for simple integration. It does however have its uses in more complex situations.

Labelling

by Alan Sykes

In a previous Education Vector I looked at one way of tackling the problem of adding column and row titles to a matrix. Recently I have up-dated my method of approach so that it was more automatic, but without the facility to right- or left-justify the column titles.

The programs do have their use in the production of educational material so I have taken the liberty to include them in this issue and also use them in my article.

I guess that many professional APL programmers have their own such utility functions which may or may not be easily available to the general public. I shall be delighted if the professional programmers who read Education Vector throw up their hands in horror at what I produce here, as long as they write to me with their preferred solution so that it too can be made available in a future edition of Education Vector!

The programs are *RLABEL*, *CLABEL*, and *DTB*. To input titles as left arguments, use a separator to create a character string, such as 'label 1/label 2/' . The function *DTB* provides an option of changing a character matrix of labels into such a string, deleting the trailing blanks in the process.

One technical problem with my approach is that I wish to be able to detect blank columns of the array automatically. However, the row titles themselves could contain a blank column. Hence I have used a character from `⎕AV(⎕AV[256]` for APLPLUS), which is used to separate the row labels from the rest of the table. Here are the programs and a short demonstration of their use.

```

V R+ROWLABELS RLABEL TAB;T
[1]  R computes the right argument for CLABEL
[2]  R with rowlabs and formatted num table separated by ⎕AV[256]
[3]  R left argument either a matrix or a vector with separators
[4]  z(1=ppROWLABELS)/'ROWLABELS+BOX ROWLABELS'
[5]  R+ROWLABELS,⎕AV[256],(⌵⌵⌵T=' ')/T+▼TAB
[6]  z(0=⌵/R[(pR)[2]]=' ')/'R+R,' ' ' '
V

```

```

V R+COLLAB CLABEL TABLE;I;J;II;JJ;M;N;MM;NN;TI;WD
[1]  R Puts column labels on the top of table
[2]  R left arg is a set of labels either as string or matrix
[3]  z(1=ppTABLE)/'TABLE+(1,pTABLE)pTABLE'
[4]  z(0=⌵/TABLE[(pTABLE)[2]]=' ')/'TABLE+TABLE,' ' ' '
[5]  z(2=ppCOLLAB)/'COLLAB+DTB COLLAB'
[6]  TABLE+(⌵⌵⌵TABLE=' ')/TABLE
[7]  I+(⌵⌵TABLE=' ') ⌵ TI+(⌵⌵TABLE=⎕AV[256])1 ⌵ +(TI>pI)/cont
[8]  I[1-1+TI]+0 ⌵ I[TI+ 0 1]+ 1 0
[9]  cont:J+I=1φI ⌵ I[J/1pI]+0 ⌵ I[pI]+1
[10] II+I/1pI ⌵ M+II-0,1+II
[11] TABLE[;II[1]]+ ' '
[12] J+COLLAB=1+COLLAB ⌵ JJ+J/1pJ ⌵ N+JJ-0,1+JJ
[13] COLLAB+COLLAB,[0.5]'-' ⌵ COLLAB[;JJ]+ ' '
[14] I[II]+I[II]+3+(M[N]-M ⌵ J[JJ]+J[JJ]+3+(M[N]-N
[15] I+I+1 ⌵ J+J+1 ⌵ WD++/I
[16] R+(2,WD)+J/COLLAB ⌵ R[(R[1;]=' ')/1-1+pR]+ ' '
[17] R+R,[1]((1+pTABLE),WD)+I/TABLE
[18] R+(φ⌵⌵φ-⌵⌵R=' ')/R
V

```

```

V L+BV BOX X;I;J;K;REP;M;F
[1]  R converts segmented string to character matrix
[2]  +(F+0*⎕NC 'BV')/ct ⌵ BV+1+X
[3]  ct:X+X,FpBV ⌵ L+(1,p,X)pX ⌵ +(⌵⌵/I+BV=X)/0
[4]  I+I/1pI ⌵ X[I]+ ' ' ⌵ J+1+I-0,1+I ⌵ K+(M+[/J)-J
[5]  REP+(pX)p1 ⌵ REP[I]+K ⌵ L+((pI),M)pREP/X
V

```

```

V R+DTB MAT;T
[1]  R drops trailing blanks on MAT
[2]  R and returns a vector with separators /
[3]  T+φ⌵⌵φMAT=' '
[4]  R+(~,T.0)/,MAT,'/'
V

```

X←?3 4p10

X

6 10
2 10
4 5
3 2
7 7
9 10

'ROW 1/ROW 2/ROW 3/' RLABEL ▼X

ROW 1 6 10 2 10
ROW 2 4 5 3 2
ROW 3 7 7 9 10

'ROW/COL 1/COL 2/COL 3/COL 4/' CLABEL

'ROW 1/ROW 2/ROW 3/' RLABEL ▼X

ROW	COL 1	COL 2	COL 3	COL 4
---	---	---	---	---
ROW 1	6	10	2	10
ROW 2	4	5	3	2
ROW 3	7	7	9	10

'/COL 1/COL 2/COL 3/COL 4/' CLABEL

'ROW 1/ROW 2/ROW 3/' RLABEL ▼X

	COL 1	COL 2	COL 3	COL 4
	---	---	---	---
ROW 1	6	10	2	10
ROW 2	4	5	3	2
ROW 3	7	7	9	10

MAT←3 5p'ALAN BILL PETER'

MAT

ALAN
BILL
PETER

DTB MAT

ALAN/BILL/PETER/

Readers may be interested to know that these programs are currently used to provide labelled output for several functions from the APL Statistics Library.

THE RANDOM VECTOR

The Newsletter of the APL Statistics Library

Editor David Eastwood

July 1991

Contents

Editorial	David Eastwood	34
Beaton's Recipe for Least-Squares	Jake Ansell and Alan Sykes	36
The Basic Statistics Volume	Jake Ansell	45

Editorial

The official launch of the first three volumes of ASL took place at a British APL Association meeting in London in March 1991. Jake Ansell presented Volume 1 (Basic Statistics), Tony O'Hagan demonstrated the facilities of Volume 2 (Basic Graphics) and Alan Sykes discussed the facilities and recent amendments to Volume 3 (Regression and Generalised Linear Models). This issue of Random Vector contains articles on Volumes 1 and 3.

The article by Jake Ansell and Alan Sykes, in particular, illustrates some of the reasons for starting the ASL project in the first place. In much the same way that collections of APL idioms and toolkit functions are a useful resource for general APL programming, it was felt that the use of APL for Statistical purposes could be encouraged by the provision of a range of tried and tested APL routines. As ASL enters the phase of public availability, I would like to recall some of the guidelines for ASL laid down in 1989 (Vector 6.3 - 'ASL after Gregynog' Tony O'Hagan):

- ASL should consist of APL functions primarily to perform statistical calculations, with basic facilities for displaying the results, but without attempting to develop a sophisticated user interface.
- ASL should be independent of commercial interests, and the ASL functions should be made freely available for only a nominal charge.
- ASL should be documented in book form, giving detailed information on the functions including full code.
- The even wider audience of users of statistics packages will need a user-friendly front-end and more eye-catching graphics. There should be a real commercial interest for companies to provide such a package. Commercial exploitations of ASL for this and other markets should be licensed so as to provide income to continue the development of ASL. The financial management should be handled through the BAA.

Now that the documentation for the first release of ASL is nearing completion, we will be able to set the prices for the various elements of ASL. As stated above, and in our other publications, it is intended that educational and personal users of ASL will only have to pay for the duplication and distribution costs of the software and documentation. Commercial users - either corporate or software houses wishing to include ASL routines in their packages - will have to enter into a licensing agreement with the BAA. As soon as we have finalised duplication details and costs we will publicise the details. In the meantime - contact the ASL committee.

Tony O'Hagan announced, at the launch of the first three volumes of ASL, that he will be standing down as Chairman of the ASL Management Committee at the next ASL Conference due to the pressure of his real work. I would like to take this public opportunity to thank Tony for all the work he has put into the ASL project over the last few years. He has contributed to the project at all levels - submitting code, testing other submissions, chairing Management Committee meetings in a relaxed and effective manner and also making a number of presentations on ASL.

As I mentioned in the last Random Vector, the next ASL conference is set to take place at **Gregynog** between **30 Sept and 2 Oct 1991**. The only costs for delegates will be their travel expenses and we are keen to hear from interested parties who would like an invitation to attend. This will be an important staging post in the ASL project as we hope to start considering the next volumes to add to the library. We shall also be seeking nominations for a new Chairman at the conference and also for an influx of new blood onto the various ASL committees - another good reason to attend(?).

As usual, I include the contact details of the current ASL Management Committee who are:

Deputy Chairman: Alan Sykes,
Department of Management Science and Statistics,
University College of Swansea,
Singleton Park, Swansea SA2 8PP.
Telephone: 0792-295296

Manager: Jake Ansell,
Business Studies, University of Edinburgh,
William Robertson Building, 50 George Square,
Edinburgh EH8 9JY.
Telephone: 031-650-3806

BAA Representative: David Eastwood,
/Newsletter Editor MicroAPL Ltd., South Bank Technopark,
90 London Road, London SE1 6LN.
Telephone: 071-922-8866

BAA Representative: John Searle,
4 Hawks Mews, Greenwich,
LONDON SE10 8RA.
Telephone: 081-858-6811 (home)

Beaton's Recipe for Least-Squares

by J.I. Ansell and A.M. Sykes

Introduction

An exhaustive search through academic papers on the subject of least-squares algorithms would no doubt reveal enough titles to fill the space allotted for this article! So why another one? The aim here is to look at the least-squares algorithm that replaces 'domino' in the generalized linear regression models volume (called *ASLGREG*) in ASL. As such, it updates the article by Sykes and Ansell (published in Vector Volume 1 Number 4, - see [3]) entitled 'When domino is not sufficient'. As this title suggests, it looks at a particular implementation of Beaton's least-squares operator, which in turn involves the notion of sweeping on successive predictor variables. For the sake of completeness, some aspects covered in that paper are also mentioned here.

Beaton's Method

Householder's least-squares operator (alias 'domino' to APLers) operates on the design matrix of a linear statistical model in which each column consists of the values of one predictor variable. In contrast, Beaton's method manipulates a symmetric matrix of sums of squares and products formed from the predictor variables followed by the y variable, by performing successive sweeps (see [1]). The sweep on the 'k'th variable is given by:

```

      V S+K SWEEP S;T;TV
[1]  S+S-(TV*. *TV+S[K;])÷T+S[K;K]
[2]  S[;K]+S[K;]+TV÷|T
[3]  S[K;K]←-÷T
      V

```

Where *S* denotes the sum of squares and products matrix. The values of *S* after the sweep on the kth variable all have appropriate statistical interpretations. In particular, having 'swept' variables 1,2,...,k, the top-left k by k matrix is minus the covariance matrix of the regression coefficients; the first k elements in the last column (corresponding to the y variable) are the regression coefficients, and the bottom right-hand corner element is the residual sum of squares of the y variable after regression on variables 1 to k. Also note that repeating the same sweep

twice is equivalent to not having swept on that variable at all, a feature useful for stepwise procedures.

As an example, consider the regression data used in our previous paper.

```

      [+D+1,X1,X2,X3,[1.5]Y
1      2.5      1.05      0.625      2.6
1      2.375      1.96      3      1.3
1      3.125      1.7      1.75      2.5
1      1      0.87      1.375      1.5
1      1.875      1.3      1.75      3
1      1.5      1.73      3.125      2.2
1      0.875      1.68      3.5      2.2
1      0.125      1.04      2.5      2.4
1      3.5      1.47      0.875      1.7
1      1.625      1.57      2.625      2.2

```

Now form the sums of squares and products matrix:

```

      [+S+(%D)+.x%D
10      18.5      14.37      21.125      21.6
18.5      44.094      27.791      34.203      39.575
14.37      27.791      21.804      32.276      30.684
21.125      34.203      32.276      53.328      45.05
21.6      39.575      30.684      45.05      49.12

```

Now sweep on the first three columns, i.e. regress y on the constant vector, x1 and x2

```

      3 SWEEP 2 SWEEP 1 SWEEP S
-1.9034      0.040355      1.203      -4.885E-15      2.6029
0.040355      -0.11619      0.1215      -0.8      -1.5763E-3
1.203      0.1215      -0.99359      2.5      -0.30615
-4.885E-15      -0.8      2.5      1.5099E-14      5.9952E-15
2.6029      -1.5763E-3      -0.30615      5.9952E-15      2.3546

```

Hence the regression coefficients are given by 2.6029, $-1.5763E^{-3}$ and -0.30615 , as verified by using 'domino':

```

      Y%D[:,1 2 3]
2.6029 -1.5763E-3 -0.30615

```

Also, the residual sum of squares of the y data after regression is 2.3546, which may also be verified by the raw APL code:

```
+/T×T+Y-D[;1 2 3]+.×B+Y⊖D[;1 2 3]
2.3546
```

the k th diagonal element (the pivot) plays a crucial role in the sweep operation since it is used to divide other quantities. A pivot close to zero may lead to considerable numerical inaccuracies. However, the sweep makes no distinction between the y -variable and other variables not so far swept on. Hence the pivot represents the residual sum of squares for that variable when regressed on the variables swept so far. For example, the 4 4 element of the result of the sweeps above, equal to $1.5099E^{-14}$ shows that the variable x_3 can be represented as a linear combination of the previous variables: in fact the figures in column 4 show that $X_3 = (-.8 \times X_1) \div 2.5 \times X_2$.

Collinearity

When predictor variables are (near-) linearly dependent, as in the example above (a situation labelled collinearity), the use of domino either produces a *DOMAIN ERROR* or large regression coefficients which have huge variances and which are highly correlated. In simple terms, duplication of information confuses the algorithm, and results can sometimes owe more to the small details of the algorithm than the original problem.

A simple way out of this dilemma is to miss out the variable(s) which duplicate(s) information. This is where the pivot plays its role. Before pivoting on the k th variable, $s[k;k]$ can be related to the correlation coefficient of the k th variable on those that have been pivoted. Hence it is possible to decide to not include the k th variable if 1 minus its r -squared value is, say, less than $.001$ - the criticality parameter.

Frane in his paper on checking the tolerance in regression calculations (see [2]) showed that prior to using the k th pivot, it is also easy to see whether, if the k th variable had been included previously, variables currently included would have failed the inclusion test. In either case, the k th variable is omitted.

Frane's illustrative example uses the correlation matrix C of three predictor variables given below.

$$C = \begin{bmatrix} 1 & 0.99498743710662 & 0.099498743710662 \\ 0.99498743710662 & 1 & 0 \\ 0.099498743710662 & 0 & 1 \end{bmatrix}$$

If the sweep operator is applied to variables 1 and 2, then the pivot for the third sweep is .01, which is above the criticality parameter .001.

$$\begin{array}{ccc} & 2 \text{ SWEEP } 1 & \text{SWEEP } C \\ \begin{array}{c} -100 \\ 99.499 \\ 9.9499 \end{array} & \begin{array}{c} 99.499 \\ -100 \\ -9.9 \end{array} & \begin{array}{c} 9.9499 \\ -9.9 \\ 0.01 \end{array} \end{array}$$

However, if variables 2 and 3 were swept first, then the pivot for sweeping on the variable 1 is then less than .001.

$$\begin{array}{ccc} & 3 \text{ SWEEP } 2 & \text{SWEEP } C \\ \begin{array}{c} 1E^{-4} \\ 0.99499 \\ 0.099499 \end{array} & \begin{array}{c} 0.99499 \\ -1 \\ 0 \end{array} & \begin{array}{c} 0.099499 \\ 0 \\ -1 \end{array} \end{array}$$

The Basic Design

It is clear from the above discussion, that Beaton's method can thus be applied to produce the covariance matrix of the estimates, the estimates, the residual sum of squares, and information on what variables are nearly linearly dependent on previously included variables. In addition, as the consecutive sweeps are performed, the residual sum of squares of the intermediate models can be 'taken down and used in evidence'.

Hence the result of the weighted least squares routine ΔWLS takes the form of an n by $[n+3]$ matrix m such that:

- (a) $M[1N; 1N]$ is the covariance matrix of the estimates
- (b) $M[; N+1]$ is a column of the regression coefficients
- (c) $M[; N+2]$ is a column of the consecutive residual sum of squares
- (d) $M[; N+3]$ is a boolean vector with a zero value indicating that the corresponding variable has been omitted, for one of two possible reasons.

The function *BEATON* (also supplied in *ASLGREG*) illustrates this form of output. It operates, using a right argument equal to the sum of squares and products matrix, and a two-element vector left argument specifying the criticality parameter and the number of variables to be swept.

```
.001 3 BEATON S
-1.9034      0.040355  1.203      -4.885E-15  2.6029      1  2.464
0.040355    -0.11619  0.1215     -0.8        -1.5763E-3  1  2.449
1.203       0.1215    -0.99359  2.5          -0.30615    1  2.3546
-4.885E-15  -0.8      2.5        1.5099E-14  5.9952E-15  0  0
```

```
.001 4 BEATON S
-1.9034      0.040355  1.203      -4.885E-15  2.6029      1  2.464
0.040355    -0.11619  0.1215     -0.8        -1.5763E-3  1  2.449
1.203       0.1215    -0.99359  2.5          -0.30615    1  2.3546
-4.885E-15  -0.8      2.5        1.5099E-14  5.9952E-15  0  2.3546
```

Because of the collinearity already noted, these two statements provide nearly identical results. Note the succession of residual sum of squares, namely 2.464 (after fitting the constant term) 2.449 after additionally fitting x_1 , and so on. Applying *BEATON* to Frane's correlation matrix of predictor variables gives the following result.

```
.001 3 BEATON C
-100      99.499  9.9499  1  0.9901
99.499    -100    -9.9    1  0.01
9.9499    -9.9    0.01    0  0.01
```

Thus the function *BEATON* incorporates a check for both possible occurrences of collinearity. As such it follows the methodology suggested by Frane, and also, according to the paper by Clarke (see [1]), that employed in the Royal Statistical Society's generalized linear modelling package GLIM.

The Constant Vector

Most (but not necessarily all) regression models include an intercept term (the constant vector). Sweeping on it essentially converts the sums of squares and products matrix S to the equivalent one where each variable is replaced by the same variable centred about its mean. However, this conversion is subject to potentially large rounding errors, and should be avoided. (This is the essential reason why, in our previous paper, Beaton's method was out-performed by, for

example Bjorck's method.) Hence to apply Beaton's method successfully this factor must be taken into account.

The Final Recipe - ΔWLS

The result of applying ΔWLS has already been described above. But what are its parameters? First the left argument is a two-element numeric vector consisting of the criticality parameter and a boolean value, with 1 indicating that the constant vector is included, 0 that it is excluded. The effect of this flag, if set, is that the sums of squares and products matrix calculated is that centred around the mean. All regression results are then transformed back to standard form before delivery.

Secondly, the right argument is a segmented string indicating in order the names of the design matrix, the y-variable, (optionally) a prior-weight vector, and (optionally) a boolean vector indicating predictor variables not to be included. As an example illustrating its use consider the following quadratic regression problem.

```
DES+(110)*. *0 1 2
YV+(DES+. *1 -1 2)+-50+?10p100
YV
21 16 60 64 49 27 108 113 175 233
YV@DES
49.483 -17.46 3.4583
```

```
.001 1 ΔWLS 'DES/YV/'
1.3833 -0.525 0.041667 49.483 1 45874
-0.525 0.24129 -0.020833 -17.46 1 10926
0.041667 -0.020833 1.8939E-3 3.4583 1 4611.6
```

Notice how the last column gives sufficient information to test that the quadratic term is necessary - this is exploited in *ASLGREG* by the provision of a *TESTHYPOTHESIS* function.

Now repeat the fit missing the last point out.

```
PW+(110)<10
.001 1 ΔWLS 'DES/YV/PW/'
1.619 -0.67857 0.059524 39.833 1 22060
-0.67857 0.34134 -0.032468 -11.173 1 6507.4
0.059524 -0.032468 3.2468E-3 2.7273 1 4216.5
```

As a final example, consider the data used already, contained in the matrix D (with the y-variable in the last column).

```
.001 1 ΔWLS 'D[;1 2 3 4]/D[;5]/'
1.9034 -0.040355 -1.203 0 2.6029 1 2.464
-0.040355 0.11619 -0.1215 0 -1.5763E-3 1 2.449
-1.203 -0.1215 0.99359 0 -0.30615 1 2.3546
0 0 0 0 0 0 2.3546
```

The penultimate column shows that variable x_3 has not been included, because it is linearly dependent on x_1 and x_2 .

Accuracy Considerations

How well does the proposed algorithm ΔWLS perform according to the benchmark introduced in our previous paper? The answer is: very well indeed. In that paper ([3]), Greenfield's data (depending on two parameters n , and m) was analysed by different methods and the number of significant figures correct quoted. For n equal to 0 and m bigger than 4, domino gave domain error, and other methods gave varying numbers of significant figures.

The function *GREENFIELD* constructs x,y data, applies ΔWLS , and compares the answer with the true answer, outputting a 2 by 3 matrix such that each row consists of the estimate, true estimate and percentage relative error.

```
V R←GREENFIELD N_M
[1]  # Creates Greenfield's data given N_M as two non -ve intgrs
[2]  # Compares least-squares output from ΔWLS with true values
[3]  # Outputs both and the relative % error to the nearest intgr
[4]  # See Sykes and Ansell Vector Vol.1 No.4
[5]  Y←7+ 1 2 3 4 5 ×10*-N_M[1]
[6]  X←1,[1.5]8+ 1 3 2 4 5 ×10*-N_M[2]
[7]  R← 1E-3 1 ΔWLS 'X/Y/'
[8]  R←R[;3],(7+(0.3×10*-N_M[1])-7.2×10*--/N_M),0.9×10*--/N_M
[9]  R←R 3 2 pR,100×|(R[1 2]-R[3 4])/R[1 2]
V
```

```
GREENFIELD 0 5
-719992.699978941 -719992.7 2.92493227896128E-9
89999.9999973676 90000 2.92490262339789E-9
```

```
GREENFIELD 0 13
-72148849499738.5 -7199999999992.7 0.206308902744647
9018606187468.23 9000000000000 0.206308902744626
```

As m increases, the size of error increases. By m equal to 13, the previous methods used gave no correct significant figures. As examples such as the ones above show, ΔWLS has a relative error of at most .2% for the range of m values from 0 to 13.

Speed

How quick is ΔWLS , compared with domino? To make such a comparison is slightly unfair on ΔWLS as the latter gives so much more information than domino. It is annoying that Householder's method has the potential to give all the information that ΔWLS does. It was such a consideration that prompted one of us to write a statistician's form of domino - see [4]; nevertheless it is the obvious comparison to take.

The following table gives timings of domino and ΔWLS in milliseconds for a variety of design matrix sizes: the timings were made using Dyalog APL on an IBM Model 70.

		Number of Columns		
		5	10	20
		$\Delta/\Delta WLS$	$\Delta/\Delta WLS$	$\Delta/\Delta WLS$
Number of	50	60/270	160/660	550/1760
	100	110/440	330/880	1150/2470
Rows	1000	1210/2750	3410/6480	11200/17800

As expected, the larger the problem, the more ΔWLS gains on domino, as the former only operates on columns to produce the sum of squares and products matrix; thereafter, the sweep operator is much faster than the equivalent Householder operation on a column. For small problems the ratio seems to be about 4:1, but for only medium sized problems (e.g. 100 by 20) this has dropped almost to 2:1.

Conclusion

Beaton's recipe can be programmed efficiently in APL to provide a reliable, accurate, and flexible algorithm that is reasonably fast. It is superior to domino in coping with problems of collinearity. It is particularly gratifying to be able to analyse a particular model and to be able to extract residual sums of squares of intermediate models without further calculation.

References

- [1] Clarke M.R.B. *The Gauss-Jordan Sweep Operator with Detection of Collinearity*. ASI78, Applied Statistics Volume 31 No. 2, 1982, pp166-168.
- [2] Frane J.W. *A Note on Checking Tolerance in Matrix Inversion and Regression*. Technometrics Volume 19, No. 4, November 1977, pp 513-514.
- [3] Sykes A.M. and Ansell J.I. *When Domino Is Not Sufficient*. Vector Volume 1, no. 4, pp 155-160.
- [4] Sykes A.M. *A Second Generation Domino for Statisticians*. Proceedings of APL87, Dallas, pp 369-380.

Bottom Shelf: Basic Statistics

by Jake Ansell (University of Edinburgh)

Introduction

To give structure to the ASL library it is divided into shelves. Whilst most shelves have specific application the Bottom Shelf is a general shelf which will contain functions to be used throughout the library. Each shelf may be divided into a series of volumes.

The Bottom Shelf will eventually consist of the four volumes: basic statistics, numerical analysis, data management and utilities. ASL volumes are not packages but a set of well tested and established APL routines. Packages may, of course, be constructed from the routines. Those wishing to provide commercial packages based on ASL will be allowed to do so through a licensing agreement with the British APL Association.

In this article we will review the Basic Statistics volume including a brief outline of the material contained within the shelf. Since the design process for this volume differs from those previously described (the Graphics and Regression Shelves) I have added some comments on design and standards. (The Basic Statistics volume may possibly form the blueprint for the development of subsequent shelves.)

A list of the functions in the Basic Statistics volume was given in the first edition of Random Vector. For those who desire a more formal account Norman Thomson has produced a Basic Statistics specification document based on APL2 which discusses in more detail aspects of the volume.

Contents of the Volume

At the first ASL Workshop, held at Gregynog, Wales, it was decided that the Basic Statistics should cover the material required to teach first year undergraduate or A Level students Statistics. This would then be enhanced as the library grew with the inclusion of functions common to more than one shelf.

The statistics currently within the volume can be divided into the following areas:

- Descriptive and Summary Statistics
- Distribution functions
- Significance tests
- ANOVA
- Time Series
- Regression and Random Number Generators

It was realised though there would be need for some mathematical and utility functions. The current version consists of about 120 functions, though some are variants with particular attributes.

The majority of functions unsurprisingly belong to the first set, Descriptive and Summary Statistics. This includes functions to calculate means, variances, standard deviations and similar quantities as well as simple line printer barcharts. Not included are functions which need better graphic output than simple line graphics such as box-whisker, pie charts etc. These are contained within the Graphics Shelf.

As far as distributions are concerned we have been conservative providing only functions relating to Binomial, Poisson, Hypergeometric, Chi Squared, Normal, F and t. Where appropriate the functions provided yield the density, quantile and tail probability value. More specialised functions will appear either in subsequent basic statistics volumes or in a separate Distribution shelf.

To be controversial we have included a number of test statistics based on traditional or frequentist approach to Statistics. Again it is hoped eventually to provide appropriate Bayesian approaches to inferential problems. The tests provided are both parametric and non-parametric, for example *TSTAT* yields the value of t-test statistic and *MANNWHIT* provides a Mann-Whitney test statistic value.

In more specialised areas we have restricted ourselves to very basic functions, for example the bivariate measures are limited to correlation. In Time Series, functions such as moving averages, seasonal adjustment and exponential weighted moving averages are given. Similarly restrictions are imposed on ANOVA with one-way and two-way functions provided. Again it is expected that other shelves will provide more adequate cover of these areas.

Whilst the Regression shelf provides a powerful suite of programs for regression, it was still felt appropriate to have basic regression functions in the Bottom Shelf. Hence the Bottom Shelf provides functions to obtain parameter estimates,

predictions, residuals and ANOVA tables for linear (multiple), simple linear and polynomial regression.

Again random number generators have been provided for a basic set of distributions and should cover those generally used. It is expected that a wider range of alternative routines will be provided in a Random Number shelf.

Development Process

Early in the evolution of ASL it was felt that the best way to proceed would be to establish working parties which would review material which had been produced or collated by a single author. When the working party was satisfied the material would be released to a larger group for testing and comment.

In the context of Bottom Shelf this approach has worked relatively well. Obviously there is a need to ensure that the working party does not continually till the same ground and that it attempts to maintain consistency over its decisions. A feature of software developed by voluntary labour is that timescales tend to be rather elastic. However even within the Basic Statistics volume the implementation time has not been unduly long and where delays have occurred it has been to ensure better software has been produced.

The ASL programmer, Tuks Gillespie, acted as collator and designer for the material in the Basic Statistics volume. The original source material on which she based her work was Norman Thomson's Bottom Shelf: Basic Statistics [1] and University College of Swansea's first year statistics course material based on APL. The working party which reviewed the software was Alan Hawkes, Norman Thomson, Alan Sykes, Alan Meyer, Mike Day, and Jake Ansell. The group meet regularly over about eighteen months. The group which tested the software is too large to name fully, however Maurice Jordan and Tony O'Hagan deserve special mention given the considerable effort they put in to testing and commenting on the material. Maurice Jordan adapted his APL testbed functions for basic statistics and subjected the volume to rigorous inspection.

Standards

Within such a major undertaking it is clear that there is a need for standards. Considerable attention has been paid to developing standards which will last throughout the likely extended lifetime of the project. There have been two major inputs: comments taken from APL programmers (particularly Adrian Smith), and those standards which have emerged through development of the software.

A more detailed article might eventually be written about the standards ASL has evolved. At the present time I will concentrate on those that directly affect presentation of the function. A major step in writing an APL function is naming it. Two aspects initially have to be taken into account. Firstly the name must identify as clearly as possible the function's use and secondly it must not clash with other names.

Even in the Basic Statistics volume the working party has devoted time ensuring appropriate names are used. Names will usually conform to the following pattern <root><inflection><serial no.>, where the root is the type of function, and the inflection modifies the root to identify a function performing a specific task. Where alternative algorithms are available to implement this task, the serial number identifies the particular algorithm used. (Users would select the algorithm they preferred.) *CHIQUANT1* is an example of this convention.

To avoid the problem of name clashes a *Names Guardian* has been established which will record name usage and decide on the new name(s) if there are any clashes.

Within a function (where plausible) variable names are used which imply type or structure of variable. For example for binomial functions *N_P* is the variable name used for the parameters.

Every ASL function will be fully commented and where appropriate will be annotated. The comments statements start with a two letter code and a list of those used are given below:

FN - Purpose/objective
RE - Result
SS - Shelf Source
RA - Right Argument
LA - Left argument
AL - Algorithm detail

This, of course, allows for automatic documentation, particularly helpful in ensuring consistency of production.

Reference

[1] Norman Thomson, [1990], '*Bottom Shelf: Basic Statistics*'.

APL Product Guide

Compiled by Alison Chatterton

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to Alison Chatterton, at the address on the inside back cover.

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	AWL486	4,450	486 based 25MHz PC, 140MB Disk, 4MB RAM, VGA Colour. (inc. 1 year on site maint.)
	AWL 386	3,095+	386 based 25 & 33MHz PC, 140MB Disk, 4MB RAM, VGA Colour. (inc. 1 year on site maint.)
APL People	IBM PCs & compatibles	poa	Includes PC, mono/colour monitor, APL Interpreter, operating system software, plus optional printers, graphics boards, additional memory etc.
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	Spectrum	7,000+	Expandable multi-user APL computer using Motorola 68000. Std. configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES(£)	DETAILS
REUTER:FILE	SHARP APL	poa	International Network application systems and public databases.
Unlware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	DYALOG APL DOS 386		Dyadics PC 386 APL Interpreter
APL Software	APL*Plus/PC Release 9	535	STSC's APL for IBM PC, PC/AT and PS2. Upgrades from earlier Releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1600	Incorporates mainframe features & performance in a version of APL for the PC
	Run-time	poa	

	Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	376	IBM's APL 2 for the PC.
Cocking/Drury	APL*PLUS PC Rel 10	477	STSC's full featured APL for IBM's and compatibles - Version 10 includes the quad-NA facility to interface to non-APL software, support for MS Windows and mouse devices. The User-command processor has been built in to the Interpreter. Upgrades to version 10 are available from Version 9 and earlier releases.
	APL*PLUS PC Run-Time	poa	Closed version of the Interpreter for developers, prevents user exposure to APL.
	APL*PLUS II System	1600	High powered APL Interpreter for the 80386 chip. Price includes one years maintenance and free upgrades - volume discounts VERSION 3 NOW AVAILABLE.
	APL*PLUS II Developer System	poa	All the features of the APL*PLUS II System plus unlimited free run-times to enable developers to distribute their applications.
	APL*PLUS UNIX	poa	STSC's 2nd generation APL for all major Unix computers and workstations. Version 4 for existing platforms and for the IBM RS/6000 available from January.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers running under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dyalgo	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and bSM Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle Interface available for IBM, Sun and Xenix versions.
I-APL Ltd	I-APL/PC or RML Nimbus	4.50	ISO conforming interpreter. Supplied only with manual. (see 'Other Products' for accompanying books)
	I-APL/BBC	4.50	As above
	I-APL/Archimedes	4.50	As above
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. From all IBM dealers.
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000 Level I		
	Mac, ST, Amiga, QL	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II		
	ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	

	APL*PLUS II V 3.0	1395	
REUTER:FILE	SHARP APL	poa	for IBM mainframes
Unlware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL&PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
	APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	IBM & compatibles	poa	IBM and 'budget' APL VDU's - monochrome/colour/graphics.
Dyadic	IBM 3151	599	Monochrome APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
	IBM 6154	1,228	Colour APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
General Software	Mellordata	poa	
Shandell	HDS3200/10 APL	965	15" screen, 8 page memory, windows. 80/132 columns, full overstrike. Multi-host multi-session support. ANSI X3.64, DEC VT100, VT220, Tektronix 4010/4014 1024 x 390 resolution.
	HDS3200/25 APL	1065	As above plus switchable 25 or 50 line screen. 75 Hz refresh. Resolution 1024 x 780 in graphics mode.
	HDS3200/35 APL	1265	As HDS3200/25 plus local pan & zoom.
	HDS3200/5C APL	1395	14" colour monitor. 75 Hz refresh. 8 or 16 colours from palette of 256. Display memory 96 lines of 80 or 132 columns. APL processing & keyboard with full overstrike. Windows, multi-host, multi-session support. ANSI X3.64, DEC VT220, VT100, VT52 emulation.

APL PRINTERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Epson series	200	Inexpensive dot-matrix and NLQ printers
	Quen-data & Qume etc	500	Daisy-wheel printers
Dyadic	Various	poa	Range of APL printers available.
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Phillips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Syndicate Manager	poa	Lloyd's managing agent's syndicate / company accounting system. Stamp & Personal accounts (inc. Run offs)
APL-385	APL-385	50(PC), 125(mf)	including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities

APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILE	poa	The First APL compiler!
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet incorporating data exchange between APL, FOCUS, IFPS, SAS, APL/DI, ADRIII, Lotus123, Visicalc, Multiplan & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(for PC's)	APL*PLUS PC Tools	275	Utilities including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	90	327x IRMA support.
	FIN & STAT. LIBRARY	250	Financial & Statistical routines
	SPREADSHEET MGR	150	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	PC Utilities including: APLMAC (windows); Unlock (unlocks functions in .AWS); DTEX (text and spreadsheet imp/exp). Mostly available in English or French.
INFOSTROY	Russificator	poa	Drivers and documentation for use with APL*PLUS/PC system and other STSC software with Cyrillic alphabet (PC).
Interprocess	IEDIT	1900-3200	Full screen APL2 editor with immediate APL execution, and a full-screen debugger
(mainframe)	AFM	8200-9800	High performance component and keyed file system (VS APL and APL2)
	Format	1650	A QuadFMT data formatter for VS APL and APL2
	FSM124	1650	AP124 programming for APL applications without GDDM (APL2)
	PowerCode	1300	External functions for APL2
	CALL/AP	3000	for calling non-APL programs (VS APL and APL2)

	UCF	1800	Inter-user data transfer for VM users via IUVC
(PC)	AFM	115	Single user component and keyed files for APL2/PC
Mercla	STATGRAPHICS 4	684	Integrated statistics/graphics system for the PC. Now with macros. Bulk and educational discounts available.
	Upgrade 3 to 4	215	
	Upgrade pre 3 to 4	375	
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC and 386. Sales Forecasting, Inventory Control, Master Scheduling, Distribution Requirements, Planning etc
	TWIGS		A modular library of tools to teach and explore state-of-the-art materials management concepts.
	T	299	Time series forecasting
	W	99	Warehouse replenishment
	I	199	Inventory Management
	G	99	Grouping requirements into EOQ's
	S	99	Scheduling production/purchasing
		599	All 5 modules above
		4999	All 5 modules site licence
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
	STATGRAPHICS Rel 4	590	
REUTER:FILE	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	75	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	NEWSFLASH	poa	Real time message exchange
	VIEWPOINT	poa	4GL - Info centre product
Uniware (mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	poa	component files for IBM VSAPL under VM/CMS and MVS/TSO aimed for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	POA	

	FILEPRINT	poa	
	FILESORT	poa	
	FILECONVERT	poa	
	FILEMANAGER(EMMA)	poa	STSC's database package
	EXECUCALC	poa	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL(VM or TSO)
(microcomputer)			
	STATGRAPHICS	poa	Statistics and graphics for PCs
	STATGRAPHICS UNISTAT	poa	An add-on module to STATGRAPHICS: Data analysis software.
	APL*PLUS TOOLS		
	-VOL1	poa	Incl. 327 AE IRMA support, RAM disk, full screen data entry, menu input, report generation, games
	-VOL2	poa	Incl. File documentor, screen editor, exception handling.
	SPREADSHEET MNGR	poa	APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY interfaces.
	APL*PLUS/PC FIN & STAT.LIBRARY	poa	Collection of financial and statistical utilities.
	POCKET APL	poa	Smaller version of APL*PLUS/PC.
	UNIASM	poa	Collection of assembler routines for APL*PLUS/PC users.
	UNITAB	poa	APL*PLUS/PC spreadsheet-like data entry and validation system.
	Tha APL DEBUGGER	poa	First released APL*PLUS/PC debugger.
	APL2C	poa	Interface between APL*PLUS/PC and DATALIGHT C language
Warwick University			
	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Consult'	poa	PC Based APL system design, programming and implementation.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
APL People	Consultancy	poa	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL6800, C/Unix, TSO/MVS, VM/CMS, graphics, Operational Research etc. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry - designing, programming and implementing applications.
Carracho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-275	Junior consultant
		275-350	Consultant
		300-450	Senior consultant

		400-600	Principal Consultant
		450-750	Managing consultant
Peter Cyriax	Consultancy	100-150	Junior Consultant
		120-200	Consultant
		160-300	Senior Consultant
Delphi	Consultancy	poa	APL system development on mainframes and micros.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping; all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Jan A. Clark	Consultancy	poa	Computer-based Information Systems Implementation where acceptance is critical. APL on PC and Macintosh. Human Factors of HCI; novice ease of use; online assistance; training courses; distance-learning materials.
INFOSTROY	Consultancy	poa	Localization of APL software for the Soviet Union software market
Intelligent Programs Ltd	Consultancy	175-350	Systems development, enhancements, support.
	Documentation	150-250	Preparation of new manuals, rewriting of existing materials.
	Training	150-250	Training for APL experts through to non-technical system users.
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.C.	Consultancy	240-500	Business analysis and APL consultancy
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	250	Specialising in Banking, Financial & Planning Systems.
REUTER:FILE	Consultancy	poa	Consultancy & support service world-wide.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Rex Swain	Consultancy	poa	Independent consultant, 15 years experience. Custom software development & training, PC and/or mainframe.
Wicklife Computer Ltd	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adlee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	An APL Tutorial	2.50	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2nd Edn)	5.50	228pp by Helzer
	APL in Social Studies	2.50	38pp by Traberman
	I-APL Instruction Manual (2nd Edn)	2.50	55pp by Camacho & Ziemann

APL Programs for the Mathematics Classroom (Springer-Verlag)

14.50 185pp by Thomson

** Please add one pound packing charge per order **

REUTER:FILE

Productivity Tools poa Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.

Databases poa Financial, aviation, energy and socioeconomic.

ISI

Tangible Math \$19 An APL Approach to Math - Shareware, includes base Sharp APL

J \$24 Dictionary APL simplified and enhanced - Shareware (Mac,PC)

SharpAPL/PC \$74 Registered Shareware and Reference Manual

ISI APL \$99 Improved APL/PC - enhancements, performance, large workspaces

Renaissance
Data Systems

Booksellers

The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.	Visa/Mcd
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15	N/N
Dutch APL U.G.	Holland	-	Mini-congress, APL ShareWare Initiative		
APL Club Austria	Austria	-	Quarterly Meetings	200AS/person, 1000AS per company	N/N
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings		

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245) 252414 ext.240
Adlee	Bernard Smoor	Dorpsstraat 50, 4128 BZ LEXMOND, Netherlands. Tel: 31.3474.2337, fax: 31.3474.2342
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APLBUG	Jorge Mezel	117 East Creek Dr., Menlo Park, CA 94025, USA
APL Club Austria	Erich Gall	IBM Osterreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Allis	The Old Malthouse, Clarence ST, BATH, BA1 5NS. Tel: 0225-462602
	Jill Moss	
Buckland Management Systems	John Buckland	Westwood, 19 Grange Road, Camberley, Surrey, GU15 2DH Tel: 0276 684327
Anthony Camacho		2 Bienenheim Road, St. Albans, Herts AL1 4NR. Tel: 0727-860130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN. Tel: 081-767 4264
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071436 9481 Fax: 071-436 0524
Datatrade Ltd.	Tony Checksfield	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384

Dutch APL U.G.	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: 03474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256 811125 Fax: 0256 811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
General Software Ltd	M.E.Martin	22 Russell Road, Northholt, Middx, UB5 4QS. Tel:081-864-9537
HMW Trading Systems Ltd	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA. Tel: 071-353 4212 Telex: 926604 HAMHSEG Fax: 071-353 3325
HRH Systems	Dick Holt	Box 4496, Silver Spring, Maryland 20904
Ian A. Clark		9 Hill End, Frosterley, Bp. Auckland, Co. Durham DL13 2SX Tel: 038852-7190
I-APL Ltd	Anthony Camacho J C Business Services	2 Blenheim Road, St. Albans, AL1 4NR. Phone 0727-860130 for queries 58 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU N.B. PAID ORDERS ONLY
IBM UK Ltd	Nat.Enq. Centre	414 Chiswick High Rd, London W4 5TF Tel: 081-747 0747
INFOSTROY	Alexei Miroshnikov	3 S. TulenIn Lane, Leningrad 191188 USSR. Tel:812-238-6392 Fax:812-319-9709
Interprocess Systems	Stella Chamberlain	9040 Roswell Road, Suite 690, Atlanta, Georgia 30350-1131 Tel: (404) 992-8400
Intelligent Programs Ltd	Mike Bucknall	Unit 7, Hermitage Ct, 6-10 Sampson St, London E1 9NA Tel:071-481-4813
ISI	Eric Iverson Orders	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel:(416) 925-6098 3512 Cameron Mills Road, Alexandria, Virginia, USA 22305-1103 Tel:(703) 548-1799
Mercla Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5096
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866
M.T.I.C.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: 09327-80848
Peter Cyriax Systems	Peter Cyriax	12 Gloucester Place, London W1H 3AW Tel: 071-935 2983
Parallax Systems Inc.	Kevin Weaver	Avery Road, Box 319, Garrison, NY 10524, U.S.A. Tel: 914-424-4265
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Road, Camberley, Surrey, GU15 1LB. Tel: 0276-20789
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A.. Tel: (212)864-3078
REUTER:FILE	Paul Jackson	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA Tel: 0203 256562
The Rochester Group	Robert Pullman	50 S.Union St., Rochester NY 14607, U.S.A. Tel:716-454-4360 or 716-454-4641
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax:02407-3118
Sugar Mill Software	Lawrence H. Nitz	1180 Kila Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel:203-868-0131
Sydney APLUG	Rob Hodgkinson	GPO Box 1425,Sydney, NSW 2001, Australia
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France. Tel:(1) 45-27-20-61. Fax:(1)45-27-20-61. Telex: 648348F UNIWARE
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel:0946-692588
Warwick Univ.	Prof.Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL. Tel:0203-523369
Zark Incorporated	Gary A. Bergquist	53 Shenipsit St, Vernon CT 06066

Reviews and Product Announcements

Zark Update

The current version of Zark contains 28 units (chapters) instead of the 26 reviewed by Emily Timson. The last two units cover encode, decode, quad-divide, and more. The current version is available for APL*PLUS PC (STSC), APL*PLUS II (STSC), and mainframe APL2 (IBM). Other APL platforms are under consideration.

The product is formally called the "Zark APL Tutor". The company is "Zark Incorporated". With the purchase of the Tutor, you get a courtesy one-year subscription to the quarterly publication "Zark APL Tutor News", a copy of the "Zark Library of Utility Functions", and a free upgrade of the Tutor a year after you buy it.

The price is \$299 US dollars for the PC versions and \$5000 for the APL2 Mainframe version.

Gary A. Bergquist
President Zark Incorporated
53 Shenipsit Street
Vernon CT 06066

IBM Announces APL2 for the RISC System/6000

APL2 is IBM's strategic interactive programming language; APL2/6000 brings this powerful and productive environment to the IBM RISC System/6000 family of processors. It is a full nested array implementation under AIX, compatible with mainframe APL2 and APL2 on the PC.

The APL2/6000 session manager has the same "look and feel" as the PC and host APL products, with the added benefit of a new AP to give you a programmable interface to X-Windows, including mouse support.

The Art of Amazement

Review of article in the Guardian, 13 December 1991

APL featured in the Computer section of the Guardian last December with an article about mazes. Adrian Fisher, of Minotaur Designs, has been experimenting with maze designs and has come up with a new variant: colour mazes. Colour mazes require the walker to change colour at every junction, making it more difficult to plot a route. The traditional maze increases difficulty by restricting the view of the overall plan with high hedges. A colour maze may allow an overall view and still be sufficiently difficult to make it fun.

An example of a three-coloured maze would be one with a traffic light sequence of colours: red, yellow and green. Say you start on red. At the first junction only the yellow can be followed according to the rules. This leads to the next junction where a choice of two green routes is offered, plus a yellow path which cannot be taken. If there was no red line to this junction then this would be a dead end if you entered it on one of the green lines.

It is simple to ensure that every node has all the possible colours entering it, but it is very difficult to determine if there are any 'black holes' from which there is no escape, particularly if the 'black hole' involved a lengthy sequence of moves.

Anthony Camacho developed a maze testing program, using Sharp APL on a PC, which checks out proposed mazes prior to it being laid out for real. Anthony presented a paper about his work at the APL 89 conference. Reading the Conference Proceedings after having read the Guardian article is quite interesting as one can recognise the origin of some of the wording! Obviously the Proceedings go into much more detail than would be possible in the Guardian, but it was surprising how much one could glean from the article.

Adrian Fisher has written a book on mazes, *The Art of the Maze*, published by Weidenfeld & Nicolson at £18. The Guardian article did not exactly review the book, but it certainly gave the impression that it would be a well worth while purchase for anyone interested in mazes.

SovAPL 5th Anniversary Conference "APL Methodology and Application" 23-25 April 1991, Leningrad, USSR

notes by Thomas M Olsen

About 70 people attended the conference, 52 from the USSR. Most of the Soviets were for Obninsk nuclear reactor centre (near Moscow), where almost all computer work is done in APL! One woman, Aya Aibarsha, came from Almà Atà, in Kazakhstan. More were expected but, perhaps because of the recent 300% increase in USSR prices, did not arrive. Those who did were greatly appreciated. I enjoyed the young engineers, who were shy at first, but eager to speak English, with questions about Three-Mile Island. One was on his way to Chernobyl.

The conference site, with accommodation, was the Central Institute for Increasing Qualifications of Leningrad Branches (of technology etc.). Each guest room was quite comfortable, with shower-tub, window to courtyard, Soviet made colour TV and a small thermoelectric refrigerator.

In the Soviet papers I saw user-friendly menus of panels and the Cyrillic alphabet used freely. STSC APL*PLUS PC is common among Soviet APL users, but APL2/PC is gaining popularity. The Soviet APL/85 interpreter (APL90 Conf. Proc.) works only on Soviet machines.

The paper I found most interesting was A. Kononov's, on The View of a Potential (APL) User. He saw four user categories, relating these to operating environments and specific software products. The author was asked to send a copy to ACM which may undertake to offer it in English.

There were minor technical problems at the presentations, such as no lapel microphones, or the translator getting ahead of the speaker (we spoke a paragraph at a time, waiting for this to be translated), or the moderator losing control during the Question and Answer session.

SovAPL '91 Conference Notes by Alain Delmotte

I was been invited to attend the annual conference of the Union of Soviet APL Users which took place on 23rd to 25th April 1991. This was my first visit to an Eastern European country. Apart from being the sixth seminar of SovAPL and the celebration of the fifth anniversary of the Association, this conference was also a kind of rehearsal for the APL92 conference. The first conference took place 62 months ago, but this was not the real start of APL in the USSR. In '74 already there was a microcomputer with a one

line display and two cassette recorders which was working in APL; but that is another story.

With the exception of some minor problems in timing and temperature in one of the conference rooms, the organisation was very good, and I must even say that the foreign visitors were almost spoiled by the kindness of our Soviet hosts. The majority of the participants were from the USSR, from as far as Almà Atà; there were some fifteen foreign guests from the USA, Finland, Sweden, Denmark, Italy and Belgium. Our Soviet friends regretted the absence of K. Iverson who unfortunately could not come.

The conference was opened by A. Kondrashev, the President of the SovAPL, with the theme: "APL Methodology and Application". There were presentations by some of the foreign visitors and by Soviet APL users.

Perspectives of International APL Cooperation. E. Juvonen (Finland). Erkki saw this conference as the realisation of old dreams (he said that he wouldn't even have dreamt of that as too fantastic) and was looking forward to increased cooperation. He was happy to announce APL92 in Leningrad, 6-10 July, with the theme: "APL: The Language in Common" This major event owes a lot to Erkki. Lynne Shaw (USA) gave to A. Kondrashev the Charter making SovAPL the first chapter of SigAPL/ACM in the USSR.

Academician V.P. Ivannicov supported the creation of SovAPL. He is the editor of the magazine of the Academy of Sciences "Programming". He finds it important to see the creation of other specialised societies which could eventually be united in a Federation of Programmers' Societies (he is himself interested in LISP).

LMFBR Diagnostics System: on the way from APL to APL2. D. Lukyanov & A. Skomorokhov (USSR). The authors work at Obninsk at the Nuclear Research Institute. APL is extensively used in the domain of monitoring radioactivity for the security of the nuclear reactors. The authors have developed several APL functions libraries accessible from menus. APL is used because the problems require frequent adjustments. The system had to be divided into sub-systems as the computer did not have enough power.

Is APL2 better than APL? J. Brown (USA). This subject is well known and doesn't need more development.

The Soviet Union is ready for APL2. A. Skomorokhov (USSR). What is interesting here is that it is TRYAPL2 which is professionally used (waiting for the full package!). Alexander also explained his present activity in spreading TRYAPL2 in the nuclear industry, other industries, institutes, schools,.... He organises courses and seminars for schools. A lot of enthusiasm.

Practical experience with APL in the USA. T. Olsen (USA). Thomas presented his personal experience with APL (APL2) and extended the view to the other systems available (including J and Mathematica). He also gave some information on APL Journals and Publications as well as a brief view on the evolution and comparison of notation. Thomas spoke some Russian as did Erkki.

The Use of APL in the Teaching at the Technical University of Denmark. P. Gjerlof (Denmark). Per related his experience of teaching APL and problem solving for the intensive courses at the Technical University. The subjects offered to the students are life problems: wave action on sea structures, satellites, whale song recording and analysis,... The students who use APL go further and faster than those using FORTRAN (even if they have to learn APL, FORTRAN is a prerequisite for the courses). Per enjoys his teaching and finds that "There is no better means to learn and understand something than teaching it."

R. Kornemark (Sweden) presented APL2/6000 on IBM AIX.

C. Holter (Sweden) presented APE. He had to explain the acronym WYSIWYG, especially to the interpreter. I forgot to say that all presentations were translated into either English or Russian; the interpreter standing next to the speaker.

Can APL*PLUS/PC speak Russian? A. Kondrashev & N. Puntikov (USSR). It is of utmost importance to have the APL systems speaking in both Latin and Cyrillic alphabets: keyboard access, screen displays, printers,... and it should be possible to mix the two. The authors went through different solutions but none fulfilled all the needs. They eventually designed a new character table to have the 66 Cyrillic characters along the Latin and APL ones. It still remains a problem when importing external Russian files, but this is solved by external modules.

APL-KRON Portable Russian speaking system for the IBM PC. V. Dubonikov (USSR). APL-KRON is an extended (APL2) PC-portable APL. It has been developed by the Soviet-British Joint Company "Ural/TEKNESIS". Apart from the APL2 type of extensions, APL-KRON includes new functions for Relational DataBase management and for windows handling (including graphics). There is a language function which allows you to switch from English to Russian and back. Future plans include compatibility with FoxBase files, crossed tables, mouse support, generalised arrays. APL-KRON is a commercial product (\$349) and has a runtime version.

APL as a Profession. N. Gritsenko, E. Korovina, A. Miroshnikov (USSR). The authors presented their views on the requirements of the present day APL programmer. If before, the productivity of the programmer increased by regular small steps from beginner to expert, now there is a plateau and a large step

before reaching expertise. This is due to the necessity to include non-APL techniques like X-windows, networking,.... Furthermore the work can no longer be done by one person but requires a teamwork and information exchange. The authors are part of a new company, Infostroy, created in December 1990 in Leningrad by a State Building Company. Their work is primarily but not only oriented to the building industry. They also support the users of legal APL*PLUS/PC.

Non Linear Models Identification with non-Traditional Statistical Methods. V. Kupriyanof, O. Luksha, O. Obratsov (USSR). A fully integrated system was presented to identify non-linear models. The system is interactive and provides graphical representations of the match of the identification to the observed values. The process is iterative and allows one to choose between different techniques (random search, simplex, Marquardt, bootstrap). It was first written in FORTRAN but then migrated to APL*PLUS, as the first version was bulky and required a preliminary knowledge of the model. The authors plan to move to APL2 as this would allow them to call FORTRAN routines (they have only APL*PLUS/PC v.7).

There were other presentations in parallel sessions.

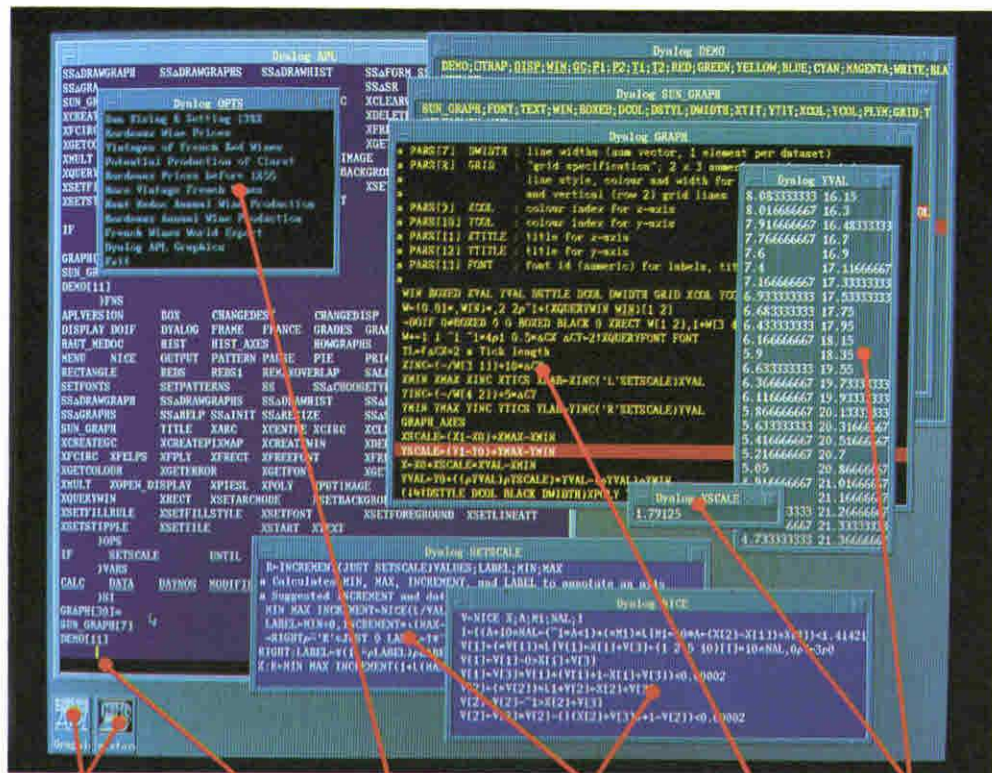
The cultural activities allowed us to visit some beautiful museums and castles, including the Ermitage; but we could only have a short apercu as these would have required some months or years to appreciate fully. We had a welcome dinner, a get-together, a banquet, a wonderful ballet evening (Nutcracker), a circus evening (myself in Moscow, but the others in Leningrad).

SovAPL has created an Award for outstanding achievement in the development of APL in the USSR. The first award was granted to E. Juvonen for his action in opening the Soviet APL community to the world. A new publication was distributed during the conference. It is in Russian and is called APL Language World. Our Soviet guests, and I would prefer to say our Soviet friends, have fully understood the value of extra-session contacts and work. The participants in the "night seminars" will not forget the hard work undertaken on those occasions.

In general I was only disappointed by the excessive marketing aspect of some presentations (IBM). Also there were no representatives from other companies especially STSC. At present the APL interpreters in the USSR are either very good locally developed interpreters (on mini and now micro-computers) or APL*PLUS/PC. Why nobody from STSC? It is very difficult to understand. If the figures of Personal Computer World (Jan 90, p.104) are correct, I cannot understand that a market of 28 million micro-computers by the year 2000 is not worth moving for.

Altogether I very much enjoyed the conference and the other activities. I would then suggest: TRYAPL92. It will be a full package of APL work, cultural activities and above all hearty friendship.

GETS YOU THERE FASTER!



You can "iconify" and restore your windows as and when required. Here you see icons for an APL graphics window and a second Unix

You can execute expressions in the Session whilst Edit and Trace windows remain open.

You can edit character matrices and vectors of character vectors.

You can edit several functions and/or operators at the same time. Lines can easily be copied/moved between functions or to and from the Session.

The TRACE facility lets you step through your application line by line. As you do so, the APL stack is displayed and the current execution line is highlighted.

Watch your variables change as you run your code.

A 1-user licence for Dyalog APL/X costs just £1,500 and is available for a range of UNIX workstations including IBM RISC System/6000, Sun SPARCstations, and Hewlett-Packard HP9000-300/400 Series.

In addition to the unique Development Environment pictured above, Dyalog APL/X provides a powerful set of tools to help you produce good-looking, responsive and easy-to-use interfaces for end-user applications. Other important features include graphics support, external variables, a really fast component file system and a native file interface. You can easily access external functions written in C or Fortran, and an interface to Oracle is available on most systems.

All Trademarks, Servicemarks and Trade Names are the acknowledged property of the Companies concerned.

For further details contact:

Dyadic Systems Limited
Riverside View, Basing Road, Old Basing,
Basingstoke, Hampshire RG24 0AL.
Tel: (0256) 811125 Fax: (0256) 811130

DYADIC

Dyalog APL/X

reviewed by John Searle

Introduction

Dyalog APL/X is the version of Dyadic Systems Limited's APL interpreter which runs under the Unix X-Windows display manager. It has been available since late 1990 and runs on the IBM RS/6000 under AIX, the Sun Sparc range under SunOS, and the HP 9000 range under HP-UX. It is compatible with Version 6.0 which has been available for much longer on other platforms. I should emphasise that although this review is based on my usage of the product at work, it represents my own opinions and not necessarily those of my employer. Reviewing at work was necessary as the concept of the take-home, lap-top Unix workstation is not yet with us (although several manufacturers are reportedly working on it!)

The major success of APL/X has been the achievement of binary compatibility across multiple machines. The same workspace file may be loaded across a networked file system on a Sun, HP or IBM workstation. Obviously, the APL interpreter is compiled by Dyadic to a different executable on each machine, but the user's application is completely transportable. For portability between remote machines, 3.5 inch disk transfer is possible between Sun and IBM. The DWSOUT /DWSIN workspaces are provided to move from DOS to Unix or vice versa.

The Product

APL/X is a set of disks (or a tape in the case of HP), two manuals (in the same style as the DOS manuals) and an "X-Windows Interface booklet". Installation is easy, but a Peter or Pauline is probably at hand to assist. The set of distributed workspaces and files is quite extensive, including all of the usual workspaces plus some new ones specific to X-Windows.

In order to use Dyalog APL/X, you need one of the above-mentioned machines, and the entry cost is not small. The minimum reasonable requirement would be to use a stand-alone Sun IPC workstation which would cost around £6,000. The minimum entry price in the HP or IBM range is around £12,000, but the price /performance relatives are changing regularly. Dyadic are re-sellers of the RS/6000 range so no doubt they can give you a more exact quotation. Having paid that much for a workstation, Dyadic therefore assume you can afford the price of £1,500 for a full release of APL/X.

Look and Feel

A typical Dyalog APL/X developer's session involves a terminal window, an immediate execution window, a $\square SM$ application window, several cascading edit windows and possibly one or more graphics windows.

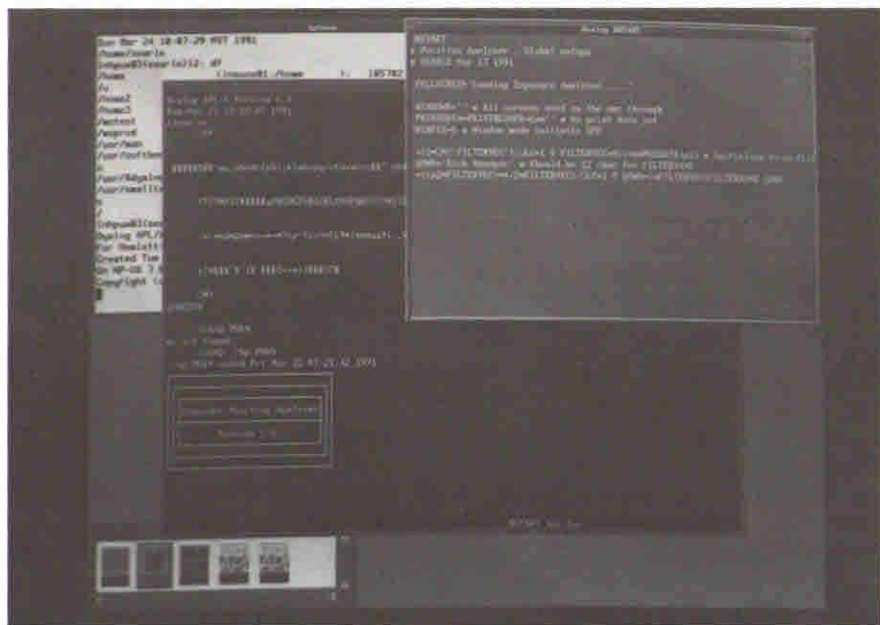


Figure 1 shows how the various windows can be arranged. The geometry of the windows is configurable, and the application window can be given an appropriate name. As a user, it is possible to set up APL/X so that only the relevant application windows are ever visible. In the same way, multiple edit windows cascade from the top-right corner. Each window is initially the same size, but the mouse can be used to move and resize them. Code can be copied, moved, cut and pasted as needed.

The Sun, HP and IBM keyboards are all different, and this presents a problem when jumping between machines. The key mappings are all different, and a memory-partitioning system is required for the human brain in order to cope. However, at least Dyadic have adopted the policy of matching an operation to the word that appears on the keytop wherever possible. Thus the CUT operation is now found on the "Delete Line" or "Cut" keys, and not on the "Page Down" key as so many of its DOS users have come to know and love. You need to

remember that Escape from the Editor is a positive action which saves your changes. Like most things Dyadic, however, if you don't like it the way it is distributed, you can change the mapping yourself.

Keeping track of the workstation mouse is a somewhat difficult process which requires the user to become an expert Rat Catcher. The mouse must be in the window that you are currently "working" within, but invariably isn't. APL/X is quite intuitive at deliberately placing the on-screen pointer within the correct window. What is really needed is a helmet that points a beam at the window you are actually looking at. Needless to say, you will spend a lot of time typing into the wrong window, which can be time-wasting in most cases and catastrophic in others.

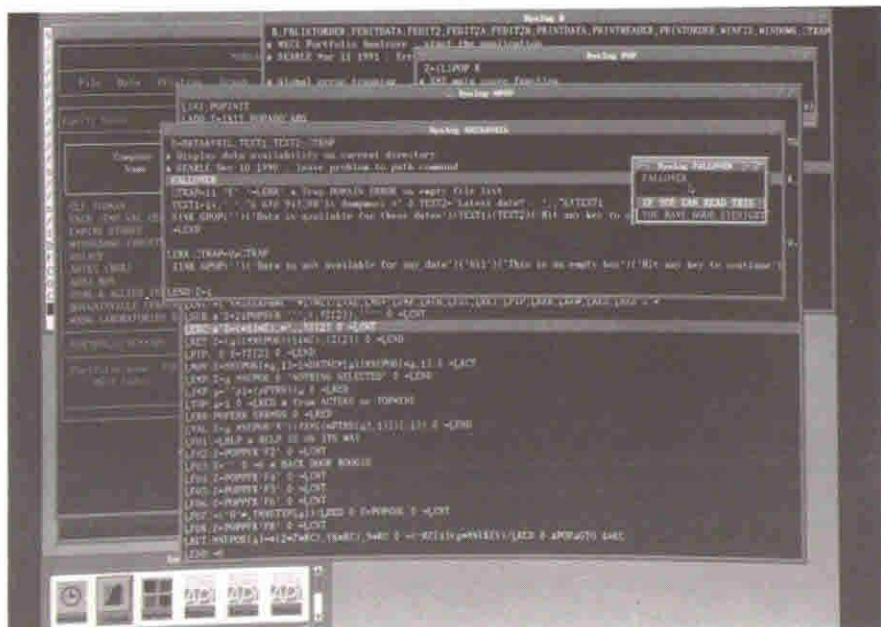


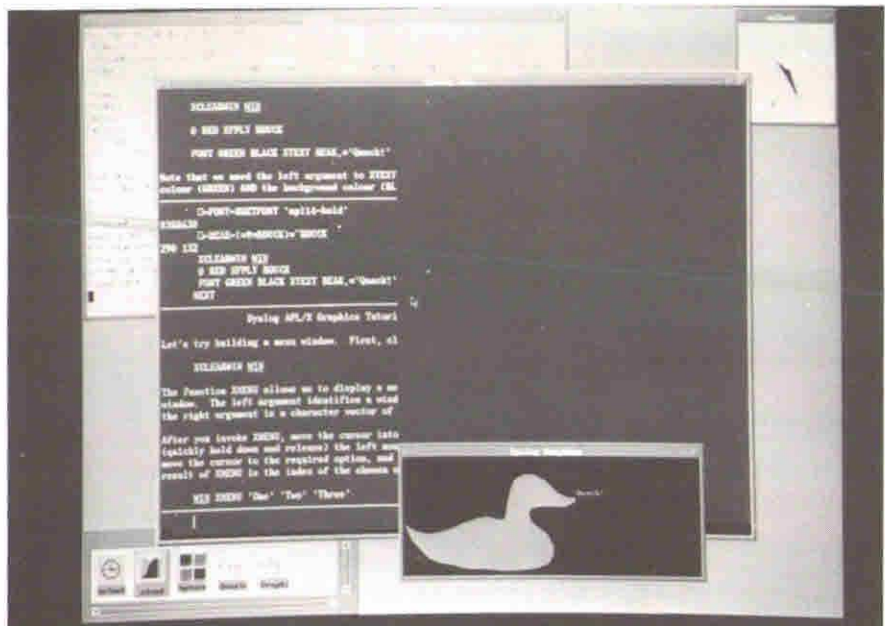
Figure 2 show how the "Trace" feature of APL/X causes a cascade of windows to appear. Each window is sized according to the function code, which can create a rather confusing display (although it needs to be distinctly different to the "Edit" window display in colour and behaviour). I find that I use this feature very infrequently, since an Escape from a function which localises $\square SM$ will cause the window to disappear.

Capabilities

APL/X has no inbuilt workspace size limitation, and is only constrained by the amount of memory (and swap space allocation) on the workstation. It is quite a powerful feeling to get a WS FULL in a 30 megabyte workspace.

An application under X-Windows typically has many sub-windows, and so I suggested that `⌈SM` should be able to address multiple screens. Dyadic's answer to this has been to associate an APL process with each window, and to make it possible to fire up multiple APL's as one "application". This is an elegant solution which is more in keeping with the Unix style.

Each of the machines uses a different X-window management system. Sun is the most radically different system called OpenWindows. Both IBM and HP have adopted the Motif style, which is not as artistic as OpenWindows but much more business-like, and generally faster. Each system has a different set of fonts, but Dyadic have coped with this and APL fonts are available for each architecture. Across a network, it is possible to run APL/X on a Sun Server but display it on a HP or IBM. I have even been able to display APL/X on a Sun workstation which was running a third-party Motif-based window manager.



APL/X comes with XGRAPHS and XTUTOR workspaces which take advantage of the X-Windows Graphics facilities. Figure 3 demonstrates that the infamous Dyadic Duck has even found its way into this modern world. Graphics appear in separate windows, and multiple windows can be driven from the one APL function. Reproducing graphical output is reasonably simple using Unix primitives "xwd" and "xpr".

APL/X uses an auxiliary processor (AP) interface to programs written in C. It is possible to associate a piece of C code with an APL function, pass parameters to it and get results back from it. The "aplarray" structure is an integral part of the interpreter which can be used within an AP, allowing C programs to return nested arrays. I have seen AP's for calling APL/X from an external C program, for inter-process and inter-machine communication, and for talking to the Oracle database. I am using an AP which shares variables (including nested arrays) between APL/X and Sharp APL on an IBM 3090. This is a powerful feature which allows the product, with some imagination, to do almost anything that can be done on the workstation.

Weaknesses

I feel that this product has few weaknesses. Most of the things that I don't like about it are configurable, so I can change them by modifying a file. There are a few irritating behaviours that are "hard-coded", such as the way the cursor only partly tabs into a character field in a $\square SM$ window. I have been involved as a beta-tester, so I've seen a lot of warts but equally a lot of rapid improvements.

My main bugbear has been an "Insufficient Swap Space" message whenever I try to fire up more than about 3 APL workspaces with associated C processes, or when APL/X is competing with other memory-hungry Unix programs. However, this is a parameter that can be configured by the Unix System Administrator.

Dyadic have chosen not to emulate the $\square WIN$ screen handling system of STSC, and so it is a significant task to try and convert an APL*PLUS application if it has a lot of screen code.

The Dyadic approach to editing is a "point and shoot" approach, but sometimes I shoot myself in the foot. The natural thing to do is type a function name, press the edit button, edit the function, exit the editor, and hit return (or otherwise move the cursor). However, if this is a niladic function, it will be executed, whereas functions which take arguments will not. You can end up executing many functions unintentionally.

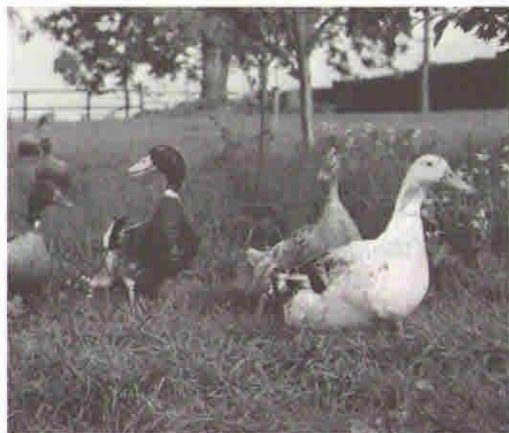
Potential

It will be essential to combine the features of APL with the resources supplied by the window manager software, if APL is to survive as a choice in the brave new world of computing. In other words, APL should be able to address "Widgets and Gadgets" as workspace objects to be manipulated. Dyadic have demonstrated some experimental software which does this.

I have found myself putting most of the "data" in an application into functions rather than tabular variables, so that I can edit the data with the ")ED" function editor. I am looking forward to Version 6.1 in which the editor has been extended to simple data arrays, so that data can stay as data.

Benchmarks

Benchmarking APL/X is quite difficult because Unix does not reliably report subsecond cpu usage back to the interpreter, and hence $\square AI$ cannot report usage finer than 10 milliseconds. Since most benchmarking is now finely tuned, comparisons are difficult. However, my general impression of APL/X is that it is fast, and the RS/6000 version takes advantage of the optimizing compiler for AIX, so that it is very fast. The user's general impression of speed is, however, governed by the performance of the underlying X-window manager.



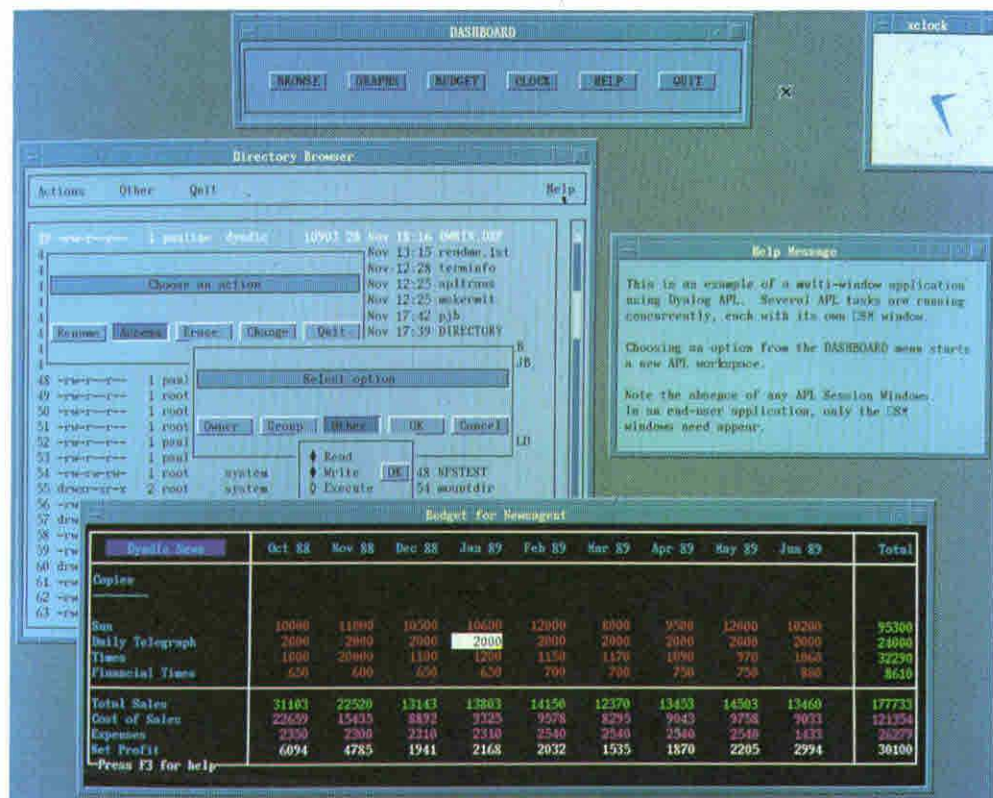
The Dyalog APL Design Group at Work

Conclusion

APL/X feels like such a natural X-Windows product, and the immediate reaction might be "What is All the Fuss About?" Since Dyadic have always used Unix internally, it is not surprising that their Unix APL fits so smoothly. This is a connoisseur's APL which makes the language truly competitive with other more "modern" application development choices.



AS YOUR USERS SEE IT!



This photograph illustrates the sort of modern windows applications you can develop using Dyalog APL/X. At the heart of the user-interface is a system variable `□SM` which represents the layout, content and behaviour of a set of fields within an X window on the display. Users interact with `□SM` using the keyboard and/or mouse through a system function `□SR`. Together these tools provide an extremely powerful yet natural and easy-to-program user-interface for your APL applications.

Dyalog APL/X is just one of a fully compatible family of APL products for UNIX workstations, multi-user UNIX systems and DOS/386 PCs.

All Trademarks, Servicemarks and Trade Names are the acknowledged property of the Companies concerned.

For further details contact:

Dyadic Systems Limited
Riverside View, Basing Road, Old Basing,
Basingstoke, Hampshire RG24 0AL
Tel: (0256) 811125 Fax: (0256) 811130

DYADIC

APL*PLUS II Release 3

reviewed by Duncan Pearson

This is an account of the experiences of me and my colleagues in moving from APL*Plus Version 2 to Version 3. They are unlikely to be everyone's but they will give a flavour to those contemplating the move.

The main differences between versions 2 and 3 are as follows.

- Location of the session manager in memory

- /NA enhancements

- Support workspaces

- Mouse support

- User command processor

These are all of benefit to a greater or lesser extent. However the garden is not all roses. I will cover the advantages offered by these five things and then go on to share some of the troubles that we have had with the move.

Session Manager

In version 2 there are two components to the Plus II system. There is a standard DOS program which handles the APL session and editing ring and an interpreter which uses the Pharlap DOS Extender to run in 80386 32-bit protected mode.

So we have:

Location	Program	Size
0 - 640k	Session manager	Configurable
	Pharlap DOS extender program	230k
640k - ?	Interpreter	
	Workspace	

The majority of PC programs are for normal DOS and have to fight with each other for their share of the 640k available. No matter what you do to the machine, if you use DOS then 640k is all you get for these programs so memory here is valuable to anyone who wants to have more than one program resident in memory at the same time. For corporate users one of the main uses for this type of program is communication over a network or terminal emulation for the mainframe. These can take over 100k away from you. With the standard configuration of 80k of session manager APL takes over 300k of extra memory.

Above the 640k line, memory is not a considerable problem. At about £50/meg the cost of circumventing WS FULL is not astronomical, so the more that can be placed up here the better.

In version 3 the session manager is bundled with the interpreter as one .exe file and is loaded into high memory. This still leaves 230k of the DOS Extender program in the first 640k but the additional 80k or so (depending on your configuration) that you have is worthwhile.

Quad-NA

The improvements to `□NA` are twofold. For programs written in Metaware High C to run in protected mode, there is the facility to pass as arguments to an associated function nested arrays. The interpretation of the content of the nested array is the problem of the C program, or, more to the point, the C programmer and to aid this a text file is provided explaining the structure in the workspace of a nested array. Fine! This is in stark contrast to Dyalog APL which comes with a library of C functions to aid the conversion of data between APL arrays and other more normal formats. The APL*Plus II for UNIX interpreter is provided with such a library. Let's hope that STSC get round to providing it for the humble PC users.

Unfortunately if you want your C program to talk to anything else on your machine you are stuck with Microsoft or Borland and have to work in real mode. This is where the extra 80k from the session manager becomes very handy. What STSC have done here is to allow the use of programs compiled for the Large memory model. A C program has two parts, the data and the code. Each part can occupy one or more segment (64k chunk) of memory. Then depending on how many are used by either part we have four possible combinations:

Model	Data	Code
Small	1 segment	1 segment
Compact	> 1 segment	1 segment
Medium	1 segment	> 1 segment
Large	> 1 segment	> 1 segment

The documentation for version 3 says that all four memory models are supported. The truth of this depends entirely on one's interpretation of support. I found that identical source compiled for the small or compact models worked fine but for the medium or large models it did not. It turned out that in order to use programs with more than one code segment I needed to change a word in an assembler source file (`m5c51stk.asm`) and re-assemble it. Luckily I was using Microsoft Quick C and had an assembler bundled with my compiler, so I am

now able to use my large model programs without problems. I am grateful to Cocking and Drury and STSC for pointing out what I needed to do, but I believe that it should not have been necessary.

Support Workspaces

Here is a really useful enhancement to the APL*Plus II product. They have taken the workspaces from Library 11 on the mainframe and provided them free with the interpreter (with APL*Plus/PC you have to buy them as APL*Plus/PC Tools). For me the significant facilities provided are in the area of workspace maintenance. The workspace WSDOC and those associated provide an extremely comprehensive documentation of a user's own workspace with complete cross reference. This includes for each identifier used a list of all references (indicating for variables any assignments). This, together with trees and tables for each function of all its global objects, is all that you could desire to help you when you pick the short straw and have to wade into someone else's code to try to find where the global *TMPDATA* is being mysteriously changed. There is also a workspace WSCOMP which does a very intelligent comparison of two workspaces. There are other workspaces which I have not had time to investigate but for me these two alone make the change worthwhile.

Mouse Support

After about three years STSC have worked out what those funny things attached by string to the back of the computer are for. So they have provided "mouse support". In the session, the mouse block (cursor?) can be used to move the real cursor, to move between sessions, to operate the top menu, to scroll the session, to tag blocks and to open new sessions for identifiers clicked on. This makes the session considerably easier to use and is all very sensibly managed.

My concern is in the area of the programming interface. As explained in Ray Cannon's article on mouse control there are a number of ways in which you can control, via DOS interrupts (`□INT`), the behaviour of the mouse. The `□MOUSE` function gives but a small subset of these allowing the setting and reading of the mouse position, the setting of the behaviour of the mouse with respect to the cursor and a few related things. There is not the facility to place bounds on the movement of the mouse or to track user drag events which as a programmer I need if I am going to provide all but the most rudimentary support. My advice is leave it out or use the `□INT` through your own functions. I hope they do better when it comes to windows.

User Command Processor

This is a facility available on the mainframe for some time and in release 10 of APL*Plus/PC that lets you write your own system commands, available wherever you are with no messy functions to clear up afterwards. It basically puts the functions on a file and provides access to that file via commands of the form]*COMMAND*. It gets the functions required, executes them then erases them afterwards. It's a pity they didn't think to get it to untie the file as well, but never mind.

I think that this is a lot of fun. My manager thinks that what I get paid for is doing REAL work. Ay, there's the rub. If you sit down and submerge yourself in it for a few days, or a week then you can start to be able to provide some really useful programming tools, but don't expect anything before that. Even if you don't write your own user commands there are some useful ones on the demonstration file, like a *FNS* with wildcards, a *WSLOC* and a function to find the free space on a disc. Some of the utilities used to do these things can be pulled off the file and are genuinely useful in their own right. I have taken all my small workspace management utilities and converted them to user commands and I now have immediate access to them without needing to remember which library I put them on this week.

I suppose when it comes down to it you either love it or think it's a waste of time. I love it but I can see why some might not.

Summary

So after all that we have to weigh up the pros and cons and decide whether it's worth the extra money. For me it undoubtedly is. The facilities provided by the new workspaces et al together with the extra room below the 640k line are a great boon. It seems a shame though that they didn't finish the job. The limited nature of the mouse support and the trouble with *QNA*, together with problems that we have had getting the DOS PRINT program to work make me feel that their minds were somewhere else when this one slipped through the door.

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers

Why I Like Windows

by Richard Smith (age 7)

The Mouse

The mouse is easier to handle than the keyboard, that is, except when dog-hairs get on the ball and the mouse sticks. The only thing about it is, I would like a bigger space to move it in! The mouse's buttons are also nice - they're a good size.

The Icons

Icons are not only easy to design (if you've got an hDC Icon Designer), but they are also easy to 'do', because all you have to do is click the hDC icon (a red box with hDC written in it) and select Icon Designer from the menu. When Icon Designer has loaded, a rectangular window will appear. This window will have a: a squared square for 'doing' your icons and b: a space which shows all your finished icons. Talking about putting windows down to 'icons' is explained in the **Windows** section. Icons are the little 'pictures' in the windows at the start-up.

Comparing with the Keyboard

For most things it is either easier or the only possible way to use the mouse. But for some things it is impossible to use the mouse, things like APL or RCS are 'keyboard only' functions, and these functions are accessible from Microsoft Windows, or, at least, can be made to be. To pull a menu down, press Alt with the underlined letter in the menu word, then to select an option, press the underlined letter, and you've got your option!

The Windows

The windows can be worked by the keyboard, for example: in games Alt G gives you the games menu, but this isn't as easy as doing it with the mouse. When you put a window down to an icon, an icon, usually its own, goes down under **Program Manager**, with its name label underneath it. By getting hold of window 'title bars' (the blue/brown stripe at the top of the windows - blue if you are currently using that window, brown otherwise) with the mouse, you can move windows. After moving a window, it will have a blue title bar, until you either click or move a different window.

Window Menus

The menus at the top of windows like Write can be used easily, even with the keyboard (as explained in **Comparing with the Keyboard**). All you have to do with the mouse, though, is click the menu you want and then click the option you want.

Iconising Uses

If you are working with anything running under Microsoft Windows and somebody comes and pesters because he/she wants to do something else, you can put whatever you're using down to an icon (as explained in **The Windows**). After doing this he/she can get on with whatever they want to, but you haven't lost your work. After he/she has finished on the computer, you can get on with your work. Putting the start-up windows down to icons saves space.

Window Buttons

The buttons at the top of every window can only be 'pressed' with the mouse, and I think that there ought to be a way of making use of those buttons with the keyboard. Other than that, the windows are well designed, because they are easy to use with a mouse, as all you have to do is click the left mouse button on the window button you want.

Why I Like Microsoft Windows?

by Adrian Smith (Age 38)

A User's View

A couple of old advertising slogans come immediately to mind: "every home should have one" and "so simple even the cat could use it". I have no idea what the former was trying to sell me; I believe the latter had something to do with John Cleese and Sony video recorders ... anyway as Richard rightly points out, Windows software is easier to use than the old text-based stuff. The interesting question is why?

Obviously, for anything involving pointing or drawing, the mouse is an enormous improvement over the keyboard. However in itself this doesn't explain why Windows software like Powerpoint is an order of magnitude easier

to work with than (say) Lotus Freelance. The same difficulty occurs in explaining why Windows Write can be picked up by a 7-year old with absolutely zero instruction, and yet an old warhorse like WordPerfect takes a 3-day course, and you still don't know how to get a PostScript print to file!

For what it's worth, I think there are two key reasons why Windows software is a real step forward in both functionality and ease of use:

- the underlying philosophy. You hear a lot of loose talk these days about **empowering** the user. I say loose talk advisedly, because of course *empowered users* are exactly what your average IT department hates. Windows is aimed squarely at individuals; it is another step in freeing users from the strictures imposed by corporate IT, and as such it is both a powerful and a dangerous idea.
- absolute consistency of use. We have IBM and their CUA guidelines to thank for this one! A lot of developers seem to think that they can claim CUA compatibility simply by strapping on a pull-down menu bar, and using the 'right' colour scheme; this is **rubbish**. Real CUA applications rigorously apply the 'object-action' model to dialogues (the converse is neatly illustrated by Lotus-123, where you always pick an action before you select the patch of the spreadsheet to do it to); real CUA applications have a set of menus going 'File ... Edit ... View ... Options Help', with the Print command under the File menu; real CUA applications already know what printers you have available, and where the output is to go; real CUA applications use your personal choice of colours ... and so on.

The main reason why Powerpoint is easier than Freelance is because you can learn it incrementally from Paintbrush; it has a 'line tool' and a 'text tool' which operate just like the ones you played with before. It also has a 'text edit' tool which behaves just like Windows Write, except that it works in a stretchable frame within your artwork. Strangely enough, the spell-checker in Powerpoint is just like the spell-checker in Word, and you can even get them to share dictionaries! If you want to move things around, then you just learned all the necessary skills playing Solitaire and Backgammon, ... and so on.

In summary, as a user I like Windows because I am all for giving individuals the freedom to exploit their environment in any way they choose. In the long term, I believe that companies which encourage employees to do this will prosper better than companies which try to control their employees' every move. By providing an environment where all software looks and operates in the same way, Windows and CUA make it far more practical for users to choose the best package for the job, safe in the knowledge that they won't be faced with a whole new set of commands and menus.

Oh by the way ... you can do things like multitasking and cut-and-paste and background printing. So what! Fast task-switching is all I really use, and I could have had that with Software Carousel for £50 under DOS on my old XT! Windows does the job very prettily (as long as you have an extra £25 to spend on an Icon designer), but I cannot see something selling 3 million copies simply by offering a nice 'point-and-grunt' DOS menu and an efficient task switcher, can you?

A Programmer's View

Cast your mind back to the heyday of motoring, maybe around 1908 when you could have a really fabulous day out in your new machine; OK it frightened the horses, and you soon got used to the joys of retrieving the contents of a ball-race from half a mile of loose gravel ... but this was *real motorist stuff* ... no quiche eaters here! But the skills needed to operate the old horse and trap were simple and well-known, and it could beat the fancy new machinery to town and back any day of the week; why on earth should anyone want to change?

I reckon I can operate the old horse and trap technology pretty well ... it took me about an hour to write a little C routine to send a bunch of files to a PostScript printer, suitably wrapped up in font selection and pagination. I know how to read and write files, and if I want to display some output, I simply use a file called 'stdout' and everything duly appears on the screen. Similarly in APL; we have always had the simple convention that if you leave the result of a statement hanging in mid-air, APL outputs it to the standard output device (usually a screen) after applying some default formatting rules.

Can we continue for ever with the simple life? The motor car opened up vastly wider horizons, and made possible journeys that could never be contemplated in horse-drawn days; once tempted by the promise of a better tomorrow, you will find it hard to stay with the old ways!

Life begins to get a harder once you let your application have full control of the screen. You might need to direct output to specific fields (in GDDM mainframe terms):

```
'A'  FSMΔPUT 6 2▼DATA  
'B'  FSMΔPUT  DESCR
```

... and you need to be a bit fussier about getting the shape and type of the data right. On the PC you probably just bung things in windows, and assume that the windows utility will handle details like nice borders, scrolling, and so on:

2 3 WINΔPUT ΔNL 3 Δ Display function names in scrolling wdw

What you tend to forget is that some muggins has put in an inordinate amount of time and effort in making things this easy! *WINΔPUT* is about 40 lines of very solid APL (in APL*PLUS/PC with *ΔWPUT*, about 25 lines in Dyalog with *ΔSM*) which took a good deal of designing and debugging. On the mainframe, I wouldn't know a 126 *ΔSVO* 'CTLS' if it jumped out and bit me, and yet I once wrote all that low-level stuff to manage the screen through GDDM calls.

My first reaction on opening the manual for the *Windows Software Development Kit* (the SDK for short) was that it was just like opening the *GDDM Application Programmers Guide* for the first time. It is written in English, so you feel you ought to be able to understand it, and yet there are so many unfamiliar concepts, and a lot of the words have new, precisely defined, meanings. In short, there is a big learning curve to go up before you can even think about things which were previously almost trivial! Take the obvious example ... here is the "hello, world" program in C and in APL:

```
main()
{ printf("hello, world\n"); } /* K&R page 6 */

'hello, world'        Δ APL wins even here!
```

Compare this with Peter Donnelly's sample to get 'Press Here' on the simplest X-widget: a button. Just what happens to you when you 'realise your widget tree' (see page 116)? Why might you want your application to activateCallback? Do you really need all that portmanteau of code just to display a simple message? How on earth can ordinary mortals hope to generate useful systems ever again??

Of course the answer is the same as the resolution to the horseless-carriage dilemma (it was predicted that by 1925 half the world's population would be chauffeurs); we don't all need to become expert mechanics to drive a modern car. Most of us don't even know how a modern car works (could you fix the electronic ignition on your 16v fuel-injected Japanese toy?); we just turn on the engine and drive away. It took around 20 years for the car to evolve from a complex engineering marvel to a commodity (Henry Ford had a lot to do with it); the interesting question is how long it will take for windows to get to the same state.

The Next Two Years

Over the next two years, we really are going to see the shape of things to come. The APL developers are already taking the first tentative steps towards integration of MS Windows and X/Motif into the APL development environment (of course MicroAPL and I-APL have already done it for the Mac). I appreciate Peter's argument that 18 lines of APL is better than 72 lines of C, but I want to write 1 line of APL:

```
ACTION←XYPOS XΔBUTTON 'Press Here'
```

... what's more I don't want a couple of hundred things with names like *disarmCallback* listed whenever I do *)FNS*. For cheap and cheerful output, I want to use the APL default display, and have the user think I just wrote something really fancy in X. If the user grabs my window and pulls it all out of shape, I don't want my code scattered with *□TRAPS* just so I can recover. All that stuff should be buried in utilities where the application need never see it.

I think we need two things to happen:

1. The APL interpreter writers must provide the basic hooks into the underlying window manager in a tolerably clean way. I have a personal preference for auxiliary processors, but if STSC chose to implement another 100 *□functions* I shouldn't really mind.
2. A bunch of APL systems programmers (people who don't mind coding at the AP level, or who can handle *□WIN* without flinching) should get together and define a common application-level interface for APL under windows. This should be common across all the different window managers (the Mac, MS Windows, X/Motif etc.) and it should also be common across all the available APLs (APL.68000, APL2/6000, Dyalog APL, APL*PLUS, I-APL etc.).

The first of these is already well underway, and is clearly going to generate the usual variety of different approaches to the same task. The hard part is to motivate someone to begin work on the second; clearly it is in no vendor's direct interest to write vendor-independent code, and yet in the long term all the vendors will sink or swim together. Just as the car makers reached a rapid consensus on the 'proper' positions for the accelerator and brake, so should the APL vendors on the 'proper' way to program a push-button. STSC and Sharp did it for component files, so it should be possible for Dyadic, MicroAPL and the rest to do it for windows! The *CUA Advanced Interface Guide* would be a good place to start.



Cocking & Drury Ltd

**Your sole UK supplier
of the entire APL*PLUS software range**

including:

APL*PLUS PC

APL*PLUS II for the 386

APL*PLUS II for UNIX

APL*PLUS VAX/VMS

APL*PLUS Mainframe

**as well as Run-Time and Run-Time Developer Kits
and all delivered ex-stock!**

**The only dealer authorised to provide maintenance and support for APL*PLUS
All software prices include 12 months free maintenance, with free upgrades (except
APL*PLUS PC).**

**We also offer a wide range of courses in APL and the most experienced APL
consultancy service in the country.**

Whatever your APL needs - call the experts first.

Cocking & Drury Ltd 180 Tottenham Court Road, LONDON W1P 9LE

Phone 071-436 9481 Fax 071-436 0524

Integrated System for Demographic Investigations

by Victor V. Ponomaryov

*Central Research Economic Institute at the State Planning Committee of the
RSFSR, 6-ya Krasnoarmeyskaya ul., 24, Leningrad, 198052, USSR*

The system is intended for specialists, who deal with investigations of demographic processes and use personal computers in their work.

Demographic investigations, and in particular forecasting the growth and movement of population, nowadays play a very important role in a practical solution of economic problems. For example, the knowledge of demographic situation in a region and of the direction of its possible changes is necessary to work out the regional programs of economic and social development. The territories for which these investigations are usually made may differ in size ranging from part of a town to the whole country.

The most important things in demographic investigations are to find out the factors which have an influence on the values of birth and death rates and tendencies of their changes, and to find out the connection between the structure of population and its growth, on the one hand, and migration processes, on the other.

Demographic forecast, especially the forecast of family structure, is necessary for planning housing construction and education development. For example, regional government need to know when, where and how many schools they have to build.

The scientists of the North-West Branch of the Central Research Economic Institute at the State Planning Committee of RSFSR have been carrying out demographic investigations during many years. According to our experience, obtained from practical experience, is that a good demographic forecast is the most accurate among other economic and social forecasts.

The reason for this accuracy is that the methods for demographic calculations have been developing during quite a long time. They are well-known now and permit the scientists to obtain maximum information from poor initial data. The demographic methods are easy to understand and describe the real processes

quite well. Also, the demographers are, as a rule, quite experienced in the field of their investigations.

The central planning organs began to use computers for demographic calculations many years ago, but at that time the computers used were big and inconvenient and it was very difficult to use them in scientific research. Now the opportunity to use personal computers gives the new quality to this kind of work.

The first version of the system for demographic investigations which already has been carried out is based upon the well-known method of ages movement. In fact this method is the simplest one and permits to obtain the most accurate results in the forecasting of sex-age structure of population.

The system has three important features:

1. The structure of the input and output data files is made the same as in STATGRAPHICS. Such file organisation allows us to make high level statistical investigations preparing the information for demographic calculations, in particular, making forecasts of birth and death rates changes, people migration movement and so on. It also permits us to present the results of investigations in good graphic form.
2. The system has an exit to APL and return to the system without reloading the workspace. It appeared that when solving a real problem it is more convenient to prepare input matrixes by using APL itself than with the help of a numeric editor, for example, because it's impossible to formalise in detail the process of preparing information.
3. The system can be easily expanded by including in it new functions which arise during the process of investigation. It was the very reason why APL was chosen as the main programming language for the system.

The system which we are using now in our work consists of the following blocks: the block of preparing input information, the block of calculations, the results scanning block, the printing results block and the graphics block.

The block of preparing input information is used to operate with data files, to create, to copy and to delete files and to edit the information in data files. It also enables us to calculate the coefficients of birth and death rate using the retrospective information.

The calculations block makes it possible to calculate the sex-age structure matrix of population for a given number of years using the ages movement method. It includes the calculation of arbitrary ages groups of population, the average life time and the summary coefficients of birth rate.

The result scanning block enables us to output on the monitor screen the table of the summary population characteristics such as the number of born, the number of dead, the natural and migrational increase, general increase and population size at the beginning of the year, as well as the corresponding average characteristics for the year. It also allows us to scan the sex-age structure matrix on the whole or in fragments and to output on the screen the birth and death rate coefficient matrixes.

The printing block makes it possible to output the same kind of results to the line printer using simultaneously the most convenient printing parameters: font, interval and the number of copies.

Finally, the graphic block permits us to output to the screen and to the printer the demographic pyramid according to the 5-year or random age intervals. All other forms of graphic result representation are supposed to be executed by means of STATGRAPHICS.

This version does not realise everything that we consider necessary even at the moment. Particularly, we have not yet realised the calculations of the population family structure and some service functions. Nevertheless, this version can be considered completed in principal due to two reason. First, it makes it possible to computerise the minimum calculations necessary for demographic investigations. Second, it has already become evident that it is necessary to rewrite some parts of the system if not the most part of it. In this connection a few words might be said about the problems we had to face during our work.

The first problem is that APL is so powerful a language that the programmer is tempted to begin programming the system right after the idea has occurred to him without studying beforehand the system structure and its component parts. That this way is a false one becomes evident when the most part of the system is already completed. Beginning with a certain moment it becomes impossible to introduce any corrections because the system begins to fall apart. Our experience convinces us that even having at one's disposal such powerful means as APL it's necessary to accurately study the details at the project stage.

The second problem, which is probably typical for the Soviet Union only, is that the new APL versions are greatly delayed there. As a result you usually find out that the problems that took much of your effort when programming the system could be easily and gracefully solved with the help of the new version. You begin to feel your backwardness acutely and you have an irresistible wish to rewrite everything. If the programmer cannot resist this wish he drives himself into the mode of continuous rewriting the same system version.

Still we hope that, since the problems are clear enough, they are quite solvable.

Experimental Design

by Stephen Wynn

APL is famed for conciseness. We all have favourite one-liners which require many lines in other languages. In this article we discuss a line which, from a practical point of view, solves not just an isolated problem but an entire subject, that of creating orthogonal experimental designs. This subject has given rise to a very large literature of which we recommend the excellent book *Quality Through Design* [1].

The Formula

I help produce longer lasting batteries. There are a surprisingly large number of parameters that can be varied in the design of a battery. Suppose we want to test X parameters to find how these affect performance, and decide to test PI values of the I th parameter $I = 1 \dots X$. To do this we make N batteries and test their performance. The question is then how to vary the parameters for each battery to get the most information. Let $F1, F2 \dots FK$ be the prime factors of N . $N = F1 \times F2 \times \dots \times FK$. Let $D1, D2 \dots DX$ be vectors of dimension K which define the design of our experiment. The one-liner mentioned above defines the values YI of the I th parameter so that for the J th battery the I th parameter has the value $YI[J]$.

$$YI \leftarrow PI | N | (D1[1] \times F1) \circ + (DI[2] \times F2) \circ + \dots + DI[K] \times FK$$

If $F \leftarrow F1, F2, \dots FK$, then in APL2 we have:

$$YI \leftarrow PI | N | \circ (\circ +) / DI \times \iota F$$

Let $P \leftarrow P1, \dots PK$, D and Y be the matrices defined by $D[I;] \leftarrow DI$ and $Y[I;] \leftarrow YI$ for $I=1 \dots X$ then clearly:

$$Y \leftarrow (\& (N, X) \rho P) | N | (X, N) \rho \circ (\circ +) / D \times (\rho D) \rho \iota F$$

For example if $N \leftarrow 8$, $F \leftarrow 3 \rho 2$, $X \leftarrow 4$, $P \leftarrow 4 \rho 2$, $\square IO \leftarrow 0$ the following matrix D produces the matrix Y :

Parameter	Design matrix D	Values Y of parameters being tested for battery number							
		1	2	3	4	5	6	7	8
1	1 0 0	0	0	0	0	1	1	1	1
2	0 1 0	0	0	1	1	0	0	1	1
3	0 0 1	0	1	0	1	0	1	0	1
4	1 1 1	0	1	1	0	1	0	0	1

Thus for example 0 0 0 0 1 1 1 1 is given by $,2|8|(1 \times 12) \cdot + (0 \times 12) \cdot + (0 \times 12)$. For the first battery all the parameters have the value 0, for the second the first two parameters have the value 0 and the second two 1 and so on, where 0 and 1 represent the two values being tested.

A Notation

I use a shorthand notation for these designs. Rank the prime factors of N in order of ascending magnitude e.g. if $N=36$ then the factors are 2 2 3 3. In this case $K=4$ as there are 4 prime factors. Let A be the vector $K+1$, B be $K+0$ 1, C be the vector $K+0$ 0 1 and so on. Then put the A B C variables together so that for example ABC is the vector $K+(1, 1, 1)$ and $AAAB$ is the vector $K+(3, 1, 0)$. The above design which is the standard 2×4 design with 8 trials is defined by A,B,C,ABC .

Consider for example a 5×3 design with 25 batteries. The design given by $A,B,AAAABB$ can be represented by a Latin Square where the first parameter being tested goes from 0 to 4 along the Y axis ($\square IO+0$) the second parameter from 0 to 4 along the X axis and the values of the third parameter are given by the table:

$$5|(4 \times 15) \cdot + 2 \times 15$$

0	2	4	1	3
4	1	3	0	2
3	0	2	4	1
2	4	1	3	0
1	3	0	2	4

The A B C notation gives a way of classifying designs. A table of good designs can be constructed. This then gives a compact way of representing designs without pages of figures that are Y values.

number of batteries	number of values of parameters						factors for best design			
8	2	2	2	2			A	B	C	ABC
8	4	4					AAC	BBBCC		
9	3	3	3	3			A	B	AB	ABB
18	3	3	3				ABB	ACC	ABC	
25	5	5	5				A	B	AAAAB	
36	2	2	2	2	3	3	A	B	ACD	BCD C D

The Goodness of Design.

To construct a design for a given number of parameters and batteries it may be necessary to experiment with the factors $A B C \dots$ until we find a good design. My system asks for the factors and then produces a number which is a measure of how good the design is. After a small amount of practice you find that the number of combinations which are sensible is very limited and almost any one produces a good design. There is often only one best design.

The information we require about the parameters are in the first instance the linear effects, that is how the output of the battery varies linearly with the value of the parameters. Secondly we are interested in the quadratic effects and so on. To give independent estimates of these effects we need firstly the correlations between the YI variables to be zero. Secondly to find independent estimates of the quadratic effects we need the correlation of the linear effects with the quadratic effects and then the quadratic effects with each other to be zero.

After taking away their means from $Y1, Y2, Y3 \dots$ we need $+/YI \times YJ$ to be zero for all $1 \leq I < J \leq X$ then all of $+/YI \times YJ \times JK$ to be zero and so on. The goodness of design is 1 if all the $+/YI \times YJ$ are zero, plus 1 if in addition all the $+/YI \times YJ \times YK$ are zero, plus 1 if all the $+/YI \times YJ \times YK \times YL$ are zero and so on. That is we try to make interactions have as high an order as possible. The exact formula which I use is not given here as it needs more than one line of code. It gives a value 2 1 1 2 2.4 1.6 respectively for the six designs in the above table.

The formula with which we started gives integer values for Y . Whereas any give values are to be tested which may not even be evenly spaced. The integers are simply changed for the values to be tested. The goodness of design criterion is equally valid in this case. The design matrix Y is printed out for the chosen design and the quantities $(+/YI \times YJ) \div +/|YI \times YJ, (+/YI \times YJ \times YK) \div +/|YI \times YJ \times YK$ and so on, where these are not zero. This gives information about interactions.

- [1] N Logothetis and H P Wynn, *Quality Through Design: Experimental Design, Off-Line Quality Control and Taguchi's Contributions*, Oxford University Press, 1989.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: PostScript and APL	Adrian Smith	92
Technical Correspondence	Norman Thomson	95
Working with Windows	David Eastwood	97
Programming for Events in Dyalog	Peter Donnelly	107
An Interface Between APL2 and the X Window System	JR Jensen and KA Beaty	125

Hackers' Corner: Working with PostScript and APL

by Adrian Smith

My immediate problem was to output a decent looking table of figures from an APL application to a PostScript printer. Rather than chickening out and just using Courier (and hence falling into the category of someone who thinks WordPerfect is a wordprocessor), I thought it would be more fun to get APL to write a little PostScript program to do the hard work.

Here is the kind of output I was after:

Hello from TEST.TXT ... the following lines are tabbed

Name	Age	Notes
Adrian	38	Well known troublemaker
Gill	38	Wife of well-known troublemaker
Richard	7	Generally a good child
Tim	5	Generally a bad child

The end.

The two problems to be solved were the simple one of lining up the columns in a proportional font, and the secondary issue of getting right justification on the numeric data.

The APL code to do this is mostly just a few globals to set up font name, size and pitch, as well as the page layout and tab positions. All the serious processing is done with a few PostScript routines, which go as a prologue on the front of the file:

```
%!PS-Adobe-2.0
%% Prologue for PostScript print
/inch {72 mul} def
/LM 1.00 inch def /BM 1.00 inch def /TM 1.00 inch def
/PW 11.25 inch def /PH 8.6 inch def
```

The '%' is simply a comment, and the first line is a useful convention ... several printers such as the Qume PS range will switch automatically between Epson, LaserJet and PostScript emulation ... they recognise a PostScript file by the '%!' at

the beginning. The subsequent lines define a trivial function to convert from inches to points, and a bunch of globals to define the page size and margins.

Next we set up some tabs, measured in inches from the left margin. My simple convention is to make right tabs negative ... if you want centred or decimal tabs you will have to devise something better!

```
/TABS [0 .5 -3 3.2] def /TB 0 def
/PITCH 20 def
```

Define Top Of Form as the Page Height less the Top Margin, and a null function to Orient the page (not required if we are in portrait mode, but essential for landscape ... see below). Then define FormFeed as a function to print the current page, orient it and go to the top ...

```
/TOF { /VPOS PH TM sub def } def
/ORIENT {} def
/FF { showpage ORIENT TOF } def
```

... and NewLine to set the typeball to the next tab, output any pending text, reset the tab bar to the first tab, and drop down Pitch positions. It checks if we have overstepped the bottom margin ... if so we need a FormFeed:

```
/NL { TAB /VPOS VPOS PITCH sub def /TB 0 def
      VPOS BM le {FF} if } def
```

Now for the hard bit! The PostScript 'get' primitive is essentially APL's array indexing, so read the first line of TAB as $tb+TABS[TB]$ (zero origin, of course). The next line reads *moveto (LM+inch |tb)*, VPOS (remember that negative tabs are right aligned, so the sign must be ignored at this stage); then we check that we have not fallen off the end of the tab rack, and if there is another tab to go, we increment the tab counter TB. (length is the PostScript equivalent of p, but because we are in $\square IO+0$, we need to subtract 1.) Finally, we check the sign of the tab and either display the preceeding text directly (using show) or right align it (using our own rshow function)

```
/TAB { TABS TB get /tb exch def
      tb abs inch LM add VPOS moveto
      TABS length 1 sub TB gt { /TB TB 1 add def } if
      tb 0 lt {rshow} {show} ifelse } def
/rshow { dup stringwidth pop neg 0 rmoveto show } def
```

If the user has requested landscape printing, we can throw in a little function to rotate the paper and shift origin appropriately:

```
% Landscape mode requested
/ORIENT { PH 0 translate 90 rotate } def
ORIENT TOF
```

And finally, we select the requested font and display the data. Note that APL needs to make no allowance for pagination ... all the hard work is done by the PostScript NL function which will allow for the specified margins and throw pages accordingly.

```
/Helvetica findfont 18 scalefont setfont
(Hello from TEST.TXT ... the following lines are tabbed) NL
() TAB (Name) TAB (Age) TAB (Notes) NL
() TAB (Adrian) TAB (38) TAB (Well known troublemaker) NL
() TAB (Gill) TAB (38) TAB (Wife of well-known troublemaker) NL
() TAB (Richard) TAB (7) TAB (Generally a good child) NL
() TAB (Tim) TAB (5) TAB (Generally a bad child) NL
() NL
(The end.) NL
FF
```

Note the final FormFeed to throw out the last page!

Simple lines are just wrapped up in an opening parenthesis and a ') NL'; anything between parentheses is treated as a text string, and hence is the argument to the NL function (remember that NL calls TAB to do the output!). Where we would logically insert tab characters in the text, we instead insert the string ') TAB ('. This conveniently takes account of the first tab at position zero (i.e. the left margin) by inserting a null string at the first tab ... subsequent tabbed sections are output by the TAB function at the appropriate position on the page.

I'm afraid I wrote all this in a boring old first generation APL, and simply hard-coded the PostScript into my reporting function. The correct solution is obviously to write a general-purpose formatter that will take a nested argument (in the example a 3-element array consisting of two text vectors separated by a numeric vector) and the required tab positions, and will automatically generate the PostScript file ready to send to the printer.

It could even use a table of character widths to work out the optimum tab positions for itself (automatically left and right aligning text and numbers); this one is definitely left as an exercise for the reader ... I will stick to the trial-and-error approach until I have a real need for something better.

TECHNICAL CORRESPONDENCE

From: Norman Thomson

20 May 1991

David Appleton in his article on generating Stirling Numbers of the second kind (Vol.7 No.4) concludes that it may be appropriate to use the recurrence relation:

$$\sigma^n_i = n \sigma^{n-1}_{i-1} + \sigma^{n-1}_{i-2}$$

I agree, but surely the appropriate development is a recursive function, e.g. *ST0* below which generates the *n*th row of the table:

```
[0]  Z←ST0 N;T
[1]  →0 IF N=Z+1
[2]  Z←(−1φT)+(1N)×T←(ST0 N−1),0
```

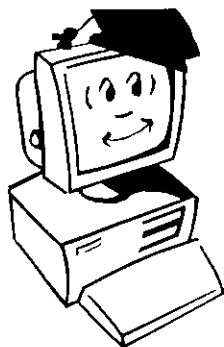
If the entire table is required, use

```
[0]  Z←ST1 N;T;U
[1]  →0 IF N≠Z+1 1φ1
[2]  Z←U,[1](−1φT)+(1N)×T←,(−,N)←U←(ST1 N−1),0
```

which contains redundant zeros in its upper triangle but could trivially be transformed to David's compressed table where Stirling rows are read as diagonals.

Norman Thomson
IBM UK Laboratories Ltd
Hursley Park
WINCHESTER
Hants SO21 2JN

Training ?



APL Beginner

3 day full course.....	£450
1 day "refresher" course.....	£180
1 day introduction.....	£180

APL Intermediate

1 day "refresher" course.....	£200
-------------------------------	------

In-House courses

Course per day	£ call
----------------------	--------

Prices includes all course material and refreshments. Low cost overnight accommodation and evening meals can be arranged to suit your requirements. Course content can be varied to suit individual needs.

Please call for details on these and other courses

Optima

Systems Limited

The Solution !

All the above are supplied to our standard terms and conditions.
Prices, specifications and availability may change without notice.
Prices exclude V.A.T.

All trademarks acknowledged.

DSB House, 15A George Street,
Croydon, Surrey, CR0 1LA

Tel: 081-680 5958

Fax: 081-680 5856

Mobile: 0831-228045

Working with Windows

by David Eastwood

Introduction

It would appear that the APL community as a whole is now being given the chance to explore the world of windows. The 1990 Vendors' Forum held recently in London by the British APL Association was almost entirely devoted to the impact of windowing environments on mainstream APL development. Here at MicroAPL we were delighted to be told by Adrian Smith (*Vector 7.3 Windows3: The Implications for APL*) that we were 'laughing like drains' about all this, and in fact we have been perfecting our drain imitations ever since.

On a more serious note, we have, via work on machines like the Apple Mac, been developing systems in a windowing environment for some time now and it would appear that a lot of the rules about working in these environments will apply irrespective of the flavour of window being used. The more that one studies the requirements of programming in a windowing environment, or, more correctly of programming a graphical user interface, the more it becomes apparent that certain general 'rules' exist. There are slight variations between the jargon used, for example, to describe using Windows3 and the Macintosh but the underlying principles are the same.

It is generally maintained that programming graphical interfaces is difficult. Articles discussing graphical interfaces quote figures like 'adding twice as much again to the workload' (Program Now, May 1991) when working with conventional programming languages. Graphical information is more extensive than text and requires more effort to specify it. Furthermore direct manipulation of the interface by the user of an application implies a different underlying program structure. The application must constantly monitor user actions which may affect one of many visible components. This jump in difficulty, together with the demands of producing applications which conform to the style guidelines in effect explains the proliferation of high level design aids for the creation of graphical applications.

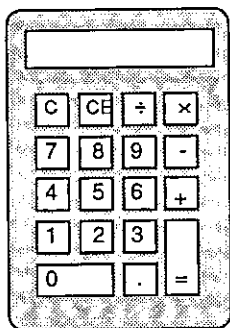
Working in APL, we do not have these sorts of design tools directly available, although a lot of work is going on into producing interfaces to these tools. I shall be, in this article, illustrating the way in which a simple window-based application might be developed almost entirely in APL and also discussing the

way in which one works in a full windowing environment. The work is carried out on the Apple Macintosh.

The facilities offered by an APL which purports to support windows can either be designed to give upwards compatibility with some previous non-windowing version or can be designed from scratch to take advantage of the environment in a way which is relevant to the environment. If one considers the latter route, it would further appear that an APL programmer is placed in a position analogous to the programmer considering using, for example, AP124. The programmer can either choose to build the application 'by hand' or can write or find some screen development utilities. In building the sample application discussed here, I have chosen to use non-APL development aids to illustrate that there are perfectly well invented wheels available.

The Application

Most windowing environments include a 'calculator' facility which allows the user to call up on the screen a functional picture of a very simple pocket calculator. Quite why an APL user might want to do this I won't say, but it does offer a useful, small, self-contained application to develop. The calculator should look rather like:



with a few basic arithmetic functions and the usual clear buttons. The user can either click on the appropriate button on the picture or hit the equivalent key on the keyboard.

Initial Design

The easiest way to implement the calculator is to draw a picture of the calculator on the screen and to identify certain regions of the picture as graphical input regions (i.e. the calculator buttons) and other regions as output areas (the calculator display). The entire picture will then be placed in its own window, allowing it to be moved around the screen and 'closed' via a standard close box. The application requires a fixed sized window and so the calculator window will be declared without scroll bars. This decision will drastically simplify the programming requirements since the application will not need to retain all sorts of global information about current window size, relative position of the scroll bars and so on. The APL application can leave a lot of the window handling to the APL interpreter and underlying operating system but there are four specific 'events' (more on this later) which need to be handled by the application:

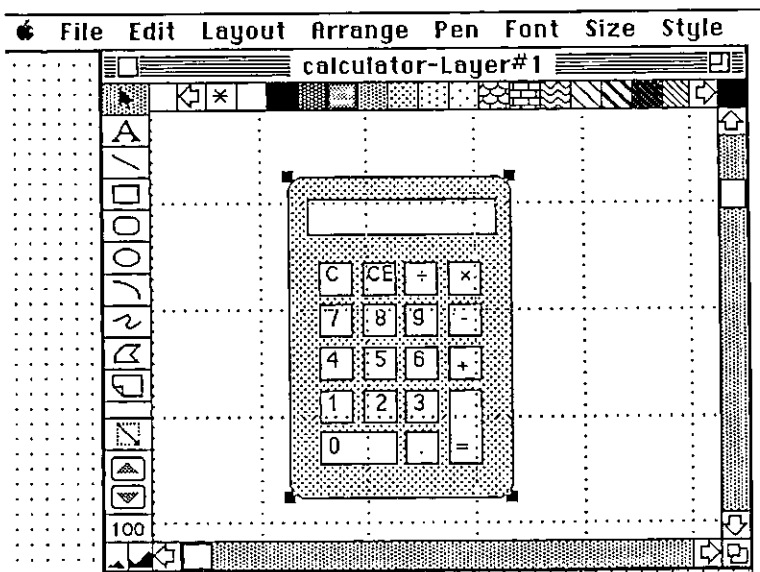
- Key down
- Mouse click
- Window redraw request
- Close window

The third will require the most handling. Because we have chosen a picture as the basis of the application, the APL application will have to redraw the picture each time the system issues a redraw request. This request will be issued when the calculator window has been overwritten by one of the other windows on the system (for example by moving another application over the calculator and then uncovering it again).

Graphical Design

The picture of the calculator can be produced in one or a number of ways. It could be drawn within APL by first drawing it on graph paper and then writing some simple graphics code using line and curve drawing graphics calls. A more elegant approach might be to produce some screen design tool within APL.

A third, and I think easier route, is to select a good drawing package such as MacDraw and use that. With MacDraw the calculator layout below took about 10 minutes to set up:



Once the drawing is done it can be copied into the system clipboard and then read into APL:

```
CALC←GETCLIP 1  ⍝Utility to get a picture from the clipboard
pCALC          ⍝Picture is held as a small integer variable
383
6←CALC
100270079 165363 7803137 1610579296 9175296 184549375
PICRECT CALC   ⍝Another utility shows the size -
0 0 173 119    ⍝ top left to bottom right
```

Now that the picture is 'in APL' we need to collect some information about the dimensions of the picture. Again, rather than doing this manually, it is very simple to collect the coordinates of active regions of the picture via some APL code. First we put the picture in a window (window number 10 is used here), and then a small function is used to collect the coordinates.


```

VR+FOO
[1] 'CALC' NEWWINDOW 10 100 100 270 220 1 104 1,7p0
[2] DRAWPICTURE CALC R      Set up window and draw picture
[3] ARROW R                  Arrow pointer
[4] R+GETCOORDS 10
[5] DISPOSEWINDOW 10 R      Close the window again
V 1991-05-24 16.50.33

V R+GETCOORDS WI
[1] R Utility to get mouse coordinates in window <WIN>
[2] WIN SETEVENTMAP 1 1 3,11p1 R Just pick up mouse events
[3] R+0 2p0
[4] L:'Click on a point, hit same point twice to quit'
[5] + (0<pX+GETEVENTS)/CL R Loop till an event happens
[6] L1:-(0<pX)/L R No more events to process, get more
[7] +(-(WIN,3)=X[1;1 2])/EL1 R Must be right wdw, event type
[8] 'Position marked :',V,X[;4 5]
[9] '((-1 2+R)=X[;4 5])/EXIT'
[10] R+R,[1]X[1;4 5]
[11] EL1:X+(1,1+pX)+X O +L1
[12] EXIT:'Exit request'
V 1991-05-24 14.15.15

```

The APL Coding

We now have a picture of the calculator, and we know its size and the coordinates of points that interest us. Now for the actual APL application. The key point to bear in mind when starting a windowing application in APL (or indeed in any language) is that the application will be 'watching' the various events that take place in the environment, discarding some and processing others. Once in a while some additional processing will take place - in this case doing some arithmetic for the calculator. As an aside, my definition of an **event** is 'something happening' rather than an APL error - I have always suspected that equating events to errors is a function of what you subconsciously expect your code to produce.

In a windowing environment, it is the responsibility of the application to maintain the contents of its window. The underlying system software (in this case the Mac operating system) will inform the application of events that affect its window (or windows). Some events can be managed by the underlying environment but some will need to be handled by the application. In this example application, quite a lot of the window update has to be carried out by the application since the window contents are graphical and the Mac does not 'remember' graphical commands. Were we using a text-only window, then the

redraw functions could be left to the operating system and we would merely note the revised positions.

Rather than list the entire application code, which will only be of direct use to the Mac user, I have reproduced the 'skeleton' function that I use to build my event handlers. As you will see from the comments, the sequence required is to define the window to be used for the application, then to indicate which events are to be monitored. This is setting up a mask of those events which are both likely to happen and also which will affect the application. You will note that the right argument to *NEWWINDOW* contains a window size which derives from the picture itself (elements 2 3 4 5 of the right argument are top left, bottom right of the window). As far as the event handling is concerned (the function *SETEVENTMAP*), the options are:

- | | |
|----------------------|--|
| 0 Ignore the event | 1 APL handles the event |
| 2 Pass event to user | 3 APL handles the event and informs the user |

and the right argument of *SETEVENTMAP* is set accordingly.

```

VSKELTON;WIN;EVENTS;OLDEST
[1]  A Skeleton event handler
[2]  A Define the window to be used e.g.:
[3]  WIN+10 A Window to be used by application
[4]  WIN NEWWINDOW 10 100 100 270 220 1 104 1,700
[5]  A
[6]  A Set Event Map
[7]  A Set the right argument for events to be monitored e.g.:
[8]  WIN SETEVENTMAP 3 1 3 1 1 1 1 1 3 3 1 1 1 0
[9]  A ***** EVENT HANDLER *****
[10] L0:ARROW A Set pointer to arrow
[11] L1:→(0=1+ρEVENTS+GETEVENTS)/L1
[12] L2:→(0=1+ρEVENTS)/L0 ♦ OLDEST+EVENTS[1;]
[13] EVENTS+1 0+EVENTS
[14] A Handle Events
[15] A Insert labels and code for those events being handled
[16] →(EV1,L2,EV3,L2,L2,L2,L2,L2,EV9,EV10,4ρL2)[OLDEST[2]]
[17] A
[18] A Each event processor will process its event
[19] A then (usually) return to L2
[20] EV1:A EVENT - KEYDOWN
[21] EV3:A EVENT - MOUSE DOWN
[22] EV9:A EVENT - REDRAW
[23] EV10:A EVENT - CLOSE
[24] EXIT:DISPOSEWINDOW WIN
V 1991-05-24 14.32.45

```

The various types of events (14 in all) that might need to be processed (line 8 above) are:

01 Key Down	02 Key up	03 Mouse Down	04 Mouse up
05 Menu Select	06 Disk insert	07 Activate window	08 Deactivate window
09 Redraw window	10 Close window	11 Move window	12 Resize window
13 Full window	14 Scroll window		

In this rather simple application, there are only four events to be handled - and indeed the 'Close' handling could just as easily be a branch to 0 to quit the application. In a larger application, you might wish to ask for a confirmation from the user before shutting down.

If the user hits a key, it can be validated against a list of the keys on the calculator. Equally a mouse click can be checked against the 'active' regions matrix. The extract shown below illustrates that part of the main function which checks mouse clicks:

```
[31] EV3:␣                               Event - Mouse down
[32] →(0=BUTTON+OLDEST[6 7]WITHIN COORDS)/L0
[33] CHAR+CHARS[1;BUTTON] ␣ Find which 'button' was pressed
[34] STR:␣                               Process string
```

The event being processed (*OLDEST*) contains window-relative mouse coordinates which are checked against a global variable *COORDS* by the function *WITHIN*. Once it has been established which button on the calculator has been pressed control passes to the calculator string handler to either adjust the display on the calculator or to carry out some arithmetic prior to doing so.

Interestingly enough, it was this arithmetical part of the code that proved the most difficult. I was surprised at the amount of processing logic built into even the simplest pocket calculator. Overall, however, other than standard Mac utilities, only three functions were written:

<i>GO</i>	The main event loop	47 lines long
<i>WITHIN</i>	Routine to validate active regions	9 lines long
<i>PROCESS</i>	Calculator string	49 lines long

About half of each function was comment. The APL68000 utilities used are generally direct calls to the operating system via a cover function. The cover function generally does little more than validate the arguments and allows the parameter passing to be carried out in a manner suited to an APL function.

Design Considerations

One can begin to get a flavour of the sort of code required in a graphical application, even from a simple example like the one used here. Richard Nabavi, in a paper at APL89 (*APL Windowing Systems - where next?*) suggested developments in APL whereby the need for the sort of central event loop shown here is removed and instead a number of independent functions are used, each one being 'woken up' by a specific event or events.

A further feature of this type of application is the number of global variables needed to manage the application. Traditional full-screen applications have always demanded a lot of global screen information but graphical applications need even more. The need to log information about the state of a window is further increased by the fact that it is difficult to predict the sequences of events that will lead to a particular function being used. For example, within a windowing environment, an application option can usually be selected by a double-click of the mouse or a single click and a menu selection. To process this within APL will require knowledge of where the mouse is when it is clicked, when it is clicked (to check for a double-click) and which menu options are valid for that selection.

The calculator functions used globals for

- Coordinates of calculator buttons
- Characters linked to calculator buttons
- Indicators to show whether the calculator button was a number or a fn
- Window and picture size information

but as I have said earlier, we have avoided a number of other requirements for globals by using a fixed sized window. An application requiring a window to contain text (which may be bigger than the window, especially if the window is resized) would need to register information about

- Current window size
- Position of scroll bars
- Text selected (if any)

and dialog boxes (again avoided here) need information on their layout, the state of any radio buttons and so on.

It should be emphasised that the APL application is only concerned about what is going on inside the window. Where the window is does not concern us. The system will inform the APL application if any other applications or windows

have corrupted the contents of our application window merely by signalling a Redraw event.

When writing an application which uses multiple windows, the programmer has a choice of continuing with the single event handler shown above, or starting to use an event loop for each window. This latter approach can be taken to the extent that each window within the application is controlled by a single APL task and the tasks communicate via shared variables.

Conclusions

Once a few basic tricks are learnt, working in a windowing environment can be quite easy. It is important to think out the requirement in advance, and certainly the initial choices of visual effects to be used can have a profound effect on the complexity of the code.

It is also interesting to discuss the way in which the work was done. Producing the calculator application was fairly fast, and certainly being able to use a package like MacDraw made the front end design much easier. At a recent seminar held by IBM UK, we were told that a survey of Fortune 500 PC users showed an average of 1.6 applications per PC. When a fully integrated and consistent graphical environment (note those two qualifications) was installed, the number of applications per PC rose to 7. This has been the case on the Mac in my experience.

All the applications that run on the MAC have a consistent user interface (no need to read manuals) and can pass data to one another without any great struggle. Using the non-APL drawing tool to draw my front end saved time and a single clipboard operation took the pertinent data into APL. The real benefit came at the end however. Any proper system needs documenting, and this article could be said to approximate to the documentation for the application.

In common with many other APLers I have spent literally years battling with the problem of printing APL documentation. One needs to find an APL font, then to persuade a wordprocessing package to use it and finally one needs to get a printer to condescend to print the stuff. That's assuming you can move the code to the word processor in the first place. On the Mac, the fact that one application has a special font means that all applications can use it, and if the text can be shown on-screen, then it can be printed. Finally, how easy is it to include pictures?

In order to produce this article I used four applications, and usually there were at least two running simultaneously. The packages were:

MacWrite
MacDraw
MacPaint
APL68000

I used MacDraw to produce the calculator image in the first place. Then MacPaint allowed me to edit a screen snapshot, cutting it down a bit and resizing it. I used the wordprocessor MacWrite for the article itself. Pictures are imported into MacWrite via the clipboard and treated as rather large characters. The APL examples came over as text via the clipboard and on occasions went back again to check that alterations I made in the wordprocessor could still actually be executed. That last point is actually very important considering function documentation - how often has some small typesetting change introduced an error into a listing?

Considering the pros and cons of actually using APL at all for this sort of application, I think it can be fairly said that APL at least holds its own against some of the applications generators. If the graphic design element can be done easily, then the internal processing seems to work quite well in APL. The APL code still seems quite a lot less verbose than alternative systems and the array handling facilities can be helpful when having to process responses to events in lots of different graphical regions or within a large dialog box. It is also the case that APL's calculative facilities are light-years ahead of those offered in many of the other development environments. In order to get my calculator logic sorted out, I only had to shuffle the input to allow a line like:

```
[32] RES+1NUM1, FN1, NUM2
```

to be executed. The calculator could then have been extended to allow all sorts of arithmetical functions to be run.

In speed terms (usually APL's downfall with this sort of application) the APL calculator was somewhat slower than the built-in Mac calculator, but could keep up with my fastest data entry, which is what counts.

Dyalog APL/X for the X Window System

by Peter Donnelly

This article describes how Dyalog APL takes advantage of the facilities of the X Window System, and discusses plans for exploiting Motif and Windows 3.0.

First Some History

Dyalog APL Version 1.0 was launched at APL83 in Washington and has subsequently been ported to over 50 different Unix-based computer systems. In those days most Unix systems were accessed using black-and-white "glass teletype" screens connected by 9600 baud (or slower) RS232 lines. These devices imposed severe limitations on the type of user-interface that could be provided, although even in those days Dyalog APL sported a full-screen Session Manager and an "AP124-like" Screen Manager.

Towards the end of 1987, Dyadic decided to break away from the restrictions imposed by RS232 lines. We decided to assume that the user had a bit-mapped colour display with near-instantaneous screen update and, ultimately, some kind of a window manager. Effectively we decided to design for the "workstation".

A New Screen Manager

The first result of this new strategy was `⌈SM` which was first demonstrated at APL-ication in September 1988. This was (and arguably still is) a radical approach to the age-old problem of providing a way of driving the user-interface that is both effective AND fits in well with the APL language.

So what makes `⌈SM` (and its companion system function `⌈SR`) so different? Firstly, `⌈SM` uses a nested array to represent the screen. In one fell swoop, the difficulty of specifying how you put a string of text and/or a number at a specific position on the screen was resolved, i.e.

```
⌈SM+1 3p'Hello World' 10 20  ⍝ "Hello World" at row 10, col 20.
⌈SM+⌈SM,[1]42 10 40          ⍝ & the number 42 at row 10, col 40.
```

DISPLAY ⌈SM

```

+-----+
|  +-----+  |
|  |Hello World| 10 20  |
|  +-----+  |
|  42                10 40  |
|  +-----+  |
+-----+
```

Secondly, the effect of localising $\square SM$ is to stick a piece of perspex onto the current screen definition (you can see it, but you can't touch it) and then paint the new screen definition on top. When you exit from a function in which $\square SM$ is localised, the perspex and whatever was stuck on it are discarded, and the screen automatically reverts to its previous appearance. This makes the provision of pop-up help, pop-up menus and pop-up dialog boxes a totally trivial task.

A third important feature is that $\square SM$ handles most if not all of the first level of user-interaction automatically. For example, both horizontal and vertical scrolling within a simple field AND scrolling of related fields is done internally. Data conversion and formatting of numbers and dates is also performed by $\square SM$ and $\square SR$, so that the APL programmer can concentrate on the application and not on the housekeeping.

Finally, (and this is not immediately obvious) $\square SM$ and $\square SR$ encourage an "event-driven" approach to writing applications. Essentially, what you do is to assign a whole collection of user-interface components (text entry fields, buttons, menus, etc.) to $\square SM$, then "give" it all to the user to do what he wants and in whatever order he considers appropriate. You control things by defining certain field behaviours in advance which subsequently trigger "events" causing $\square SR$ to terminate. After identifying and then processing the particular event, you go back into $\square SR$ (which is effectively re-entrant) and carry on.

```

...
Z←0
Loop:
Z←('EP' 'F1')((6⌊ρZ)+Z)  $\square SR$  FIELDS
+ (Z[4]←'F1' 'EP')/Help Exit
+ ((Z[1]←WFDsopt)∧Z[5]EVENT MODIFIED)/Ship
+ ((Z[1]←WFDctype)∧Z[5]EVENT ENTER)/Ctype
+ ((Z[1]←WFDprod)∧Z[5]EVENT ENTER)/Getprod
+ ((Z[1]←WFDdate)∧Z[5]EVENT BADFIELD)/BadDate
+ ((Z[1]←WFDdate)∧Z[5]EVENT EXIT)/GoodDate
A
A----- Bad Date
BadDate:
 $\square SM$ [WFDmsg;1]←c78+'Invalid date' ⋄ Z←(2+Z),1 ⋄ →Loop
A----- Help
Help:
4 5 16 0 HELP1 WFDHELP ⋄ →Loop
A----- Let user select customer type
Ctype:
A CTYPE displays a menu of customer types and returns user's choice.
A Note that the behaviour of this field is reset so that menu only
A appears on the first visit.
 $\square SM$ [WFDctype;1 7]←(CTYPE),0 ⋄ →Loop
...

```

Figure 1 : The $\square SR$ "event loop"

Figure 1 illustrates a "snippet" of code taken from the *DEMOWFD* function in the $\square SM$ demonstration workspace *SMDEMO*, which is included with all versions of Dyalog APL. Note that if the user enters an invalid date, $\square SR$ terminates with a BADFIELD event. This event is processed by displaying a suitable error message, then $\square SR$ is

called again. The value *Z* in the left argument, which was returned when `□SR` terminated, contains the current screen context (field, cursor position, keystroke) and ensures that the interaction with the user is restarted where it left off. Similarly, when the cursor enters the `WFDctype` field, `□SR` exits with an `ENTER` event. This is processed by displaying a pop-up menu (done by the function `CTYPE`) for the user to choose an option, then continuing once again.

This process is remarkably similar to the "event loop" used by X Window Toolkits like Motif and OPEN LOOK, and the "message loop" used by Microsoft Windows 3.0, but more of this later ...

Using Windows for the Developer

Let us leave the discussion of `□SM` and move on to describe how Dyalog APL uses "windows" to improve the APL programmer's lot.

In 1988, Dyadic began the development of Dyalog APL for DOS/386 using the Phar Lap 386 DOS|extender that had just become commercially available. At about that time, I happened to be invited along to a Microsoft seminar on OS/2. I don't remember much of what was said about OS/2, but I was forcibly struck by the powerful development tools provided by Quick-C and (horror of horrors) BASIC. They were, I realised, INFINITELY superior to those provided by any current implementation of APL. Although several colleagues in Dyadic had been pointing this fact out for some time, I for one only really got the message when I saw how much easier it was to debug a program in BASIC than in APL! That did it - the Dyalog APL Development Environment (codenamed "JADE") was born.

The first implementation of "JADE" (*Just Another Development Environment*) appeared in Dyalog APL for DOS/386, but it has since been ported to other versions and is even available on ASCII terminals. In simple terms, you have three types of windows. First there is the Session Window in which you enter APL expressions and in which results are displayed. Secondly, you may have one or more Edit Windows in which you can view and/or edit functions and variables. The key thing here is that you can freely switch from one Edit Window to another, and to and from the Session. You can edit all of your functions at the same time, use "Cut-and-Paste" to move lines of code between them, try out expressions in the Session then copy them into functions, and generally have a field day. The great thing is that everything is visible (or at worst only partially obscured) at the same time.

The third type of window is used by the Tracer. This is invoked by typing an expression and pressing the TRACE key (usually mapped to Ctrl-Enter). When you trace a defined function or operator, it appears in a pop-up window on the screen and the next line to be executed is highlighted. If you press Enter, the line is executed and you stop at the next one. If you press TRACE, the execution of any defined functions in the line is also traced, with the functions popping up in their own windows as they run. At any point, the APL execution stack is reflected by a stack of overlapping windows on the screen.

Debugging is further enhanced by the ability to display variables in Edit Windows and watch them change as you step through the code.

JADE was implemented for DOS in character mode, using line-drawing characters to draw borders around the Edit and Trace windows. Facilities to move and resize the windows were built into the APL interpreter. However, it was designed with a Graphical window manager (which would handle all the moving, resizing, etc. itself) in mind.

First Pick Your Window System ...

Early in 1990 Dyadic began the development of Dyalog APL/X for the X Window System. We would have preferred to have started sooner, but previously X had appeared to be only one of the many windowing systems available for Unix. The market leader, Sun Microsystems, had its own product called SunWindows which it was trying to promote as an industry standard. Apollo and Hewlett Packard also had their own proprietary window systems. All of these products had completely different programming interfaces, and the task of supporting them all was clearly out of the question. We had to pick the one that would become the most widely adopted. When the Open Software Foundation (IBM, DEC, HP, etc.) adopted X and when Sun announced that they would support it on the new SPARCstation, Dyadic took the plunge and started work on X.

I don't want to spend too much time describing the intricacies of the X Window System, but there is one very important point that I need to emphasise. X is not a window manager that imposes a particular user interface style. On the contrary, X provides mechanisms to support many quite different interface styles. Unlike Macintosh or Microsoft Windows, X does not have a specific "look and feel". X does not provide user interface components such as button boxes, scroll-bars, menus or dialog boxes. However, window managers built using X (such as OSF Motif and Sun's OPEN LOOK) do impose a specific user interface style and do provide the "widgets" with which (thanks to Microsoft's pricing policy) we are now all becoming so familiar.

Although we felt reasonably secure in choosing X, it was not at all clear at the beginning of 1990 which window manager would emerge as the Unix standard. Battle lines between the Open Software Foundation (Motif) and the System V camp lead by Sun and AT&T (OPEN LOOK) had only just been drawn. For this reason we decided to concentrate first on porting JADE to X, and to postpone major extensions to $\square SM$ until one or other window manager emerged as the standard. Several people have commented on the fact that Dyalog APL/X does more for the programmer than it does for the user. This is a temporary thing and is part of our evolutionary plan for the product. We couldn't extend $\square SM$ without plumping for one or other window manager, otherwise the end result would lack the consistent "look and feel" that is considered essential in end-user applications. This is much less of a problem for the developer, for whom power and speed are far more important. In 1991 it is now reasonably clear that Motif is emerging as the Unix standard. Had we chosen OPEN LOOK (which was the leading contender early in 1990) we would have got it badly wrong!

JADE Under X

In Dyalog APL/X, the Session and each of the Edit and Trace Windows are all separately managed "top-level" windows under the control of whichever window manager is in operation. In Microsoft Windows 3.0 terms, all the windows are OVERLAPPING in style. The appearance of the window borders and of the controls used to manipulate them differs from one window manager to another. However, the appearance of the window contents and the behaviour of the mouse and keyboard within the windows are consistent across all the machines we support. (OK, I lied, but if all systems had the same keyboard it would be true!)

At this point I must digress to discuss "Keyboard Focus"...

Most if not all of the Unix window managers support two different so-called "keyboard focus policies". This refers to the mechanism by which the user tells the system which window he/she wants to talk to. Microsoft Windows only supports (imposes) the "explicit-focus" model which requires the user to click on a window first. Motif and OPEN LOOK allow the user to choose between this method and the "pointer-focus" model, in which the window which contains the sprite (the cursor associated with the mouse) has the focus. Dyalog APL/X necessarily supports both, but I will assume the use of the "pointer-focus" model (because I prefer it) from now on. So back to telling you how it all works ...

As in most other applications with a Graphical User Interface, you have two cursors. One is the cursor associated with the mouse, to which I will refer as the sprite. The other is the text input cursor which indicates the position at which the next character you type will be displayed. This I will call the cursor. Actually, you have several cursors, one for each window, but only the cursor in the window which has the focus is visible.

So suppose that you have just started Dyalog APL/X. On your screen you will see a single window entitled "Dyalog Session" in which appears the customary banner, "clear ws" message and 6-space prompt. There are no scroll bars and no menus; not because such things are undesirable, but because Dyalog APL/X is written for X and not (yet) for one of the window managers that sit on top of X. At this stage, you can if you wish put the mouse away in your pocket (thus ensuring that the sprite remains stationary in the Session Window) and proceed as before. Users of Dyalog APL for DOS/386 who move to the IBM RS/6000 will even find the keyboard layout familiar. Sun, HP and DEC keyboards are (sadly) somewhat different. However we have been given a mouse to play with, so let's use it!

Using the Mouse

The first thing that you will find is that you can position the cursor (the text cursor) by moving the mouse and clicking the left button. This behaviour is consistent in all Dyalog APL/X windows, and is also supported by `□SR`. Double-clicking the left button over any APL name opens an Edit Window for that object (the first click positions the cursor, the second says "edit it"). When an Edit Window is opened, the sprite is automatically moved to the centre of the Edit Window and the cursor positioned in the

above or below as required. Releasing the button ends the "tag" leaving the block highlighted. Double-clicking the right button clears (untags) the currently selected block.

Additional commands can be mapped to the mouse buttons used in conjunction with the Shift, Ctrl and Alt keys. For example, Shift+Left Button can mean HOME. These mappings can be changed by the user.

Extending the Screen Manager for X

As I pointed out earlier, the task of extending the Dyalog APL user interface tools ($\square SM/\square SR$) to conform with a full-blooded window manager with scroll bars, menus, dialog boxes, and so forth, requires first that we select which window manager we will support. Until one or other of them emerged as a standard, Dyadic was unwilling to gamble on making this choice, and to have attempted several would have taken too long. Nevertheless, the current release of Dyalog APL/X (Version 6.1) contains several important extensions to $\square SM/\square SR$ to make it possible to simulate the "look-and-feel" of your choice.

The first point is that $\square SM$ has its own separate window on the display. This accelerates the process of developing and debugging the user-interface because you can type in one window and see the results in another. The initial size of the $\square SM$ window and the character string displayed in its title bar are configurable. The Session Window is displayed only if required, so that in a "closed" application the user would only see your $\square SM$ screen configured as you require it.

If the user resizes the $\square SM$ window, a *RESIZE* error (event code 1007) is generated after the execution of the current line and before the next. If the resize occurs during a call to $\square SR$, it terminates gracefully and returns a result; so the application can resume where it was interrupted after the resize has been handled. Generally, an APL program should set $\square TRAP$ to catch resize events and re-format the display according to the newly selected geometry.

$\square SR$ has been extended to accept button presses and releases as exit keys. This means that you can specify that $\square SR$ terminates and returns a result in response to a button event. This makes it very easy to program pop-up and pull-down menus. The photograph shown in Figure 3 shows such a menu function being traced, together with the resulting $\square SM$ display. Note that *MENU*[10] specifies 'D1' (the code for pressing [Down button 1]) as an exit key along with 'ER' (enter) and 'EP' (escape).

Another extension to $\square SM/\square SR$ is that a new BUTTON behaviour type has been added. A field with BUTTON behaviour may only be selected by pointing at it with the sprite and clicking a button. A BUTTON field cannot otherwise accept user input, and the cursor cannot be positioned within it. When $\square SR$ exits it returns information about the field addressed by the cursor and about the BUTTON field that was selected. An example of where this is useful is the provision of context-sensitive help.

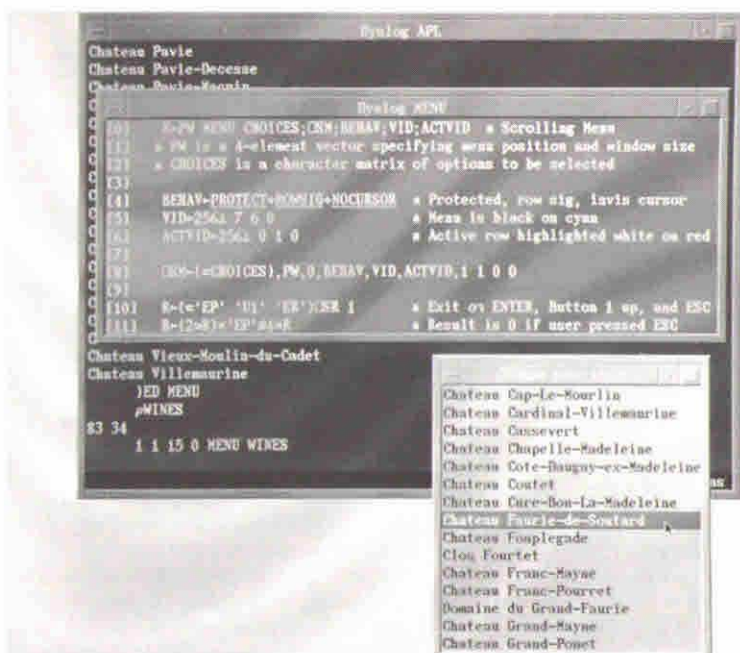


Figure 3: Photograph of the MENU function

Multi-window Applications

Many Dyalog APL/X users have asked us to support multiple instances of the $\square SM$ window, and this will be provided for in a future release as described later on in this article. In the meantime it is perfectly possible to provide a multi-window front-end by running several Dyalog APL tasks concurrently, with separate $\square SM$ windows attached to each workspace. There are in fact certain advantages to this approach because each workspace deals only with a particular aspect of an application and can perhaps be re-used in another - object-oriented programming, perhaps?

The key feature of a multi-window application is that the user can interact with several windows at the same time. In a true multi-tasking environment like Unix, this is very simple because the Operating System does it for you. All that is required is to provide a way of communicating between the separate tasks.

The photograph illustrated in Figure 4 shows such an application. It consists of five separate APL tasks, each with its own $\square SM$ window. At the top of the screen is positioned the DASHBOARD, which controls the application as a whole. Clicking on any of its buttons simply starts another APL task which)LOADs the appropriate workspace.

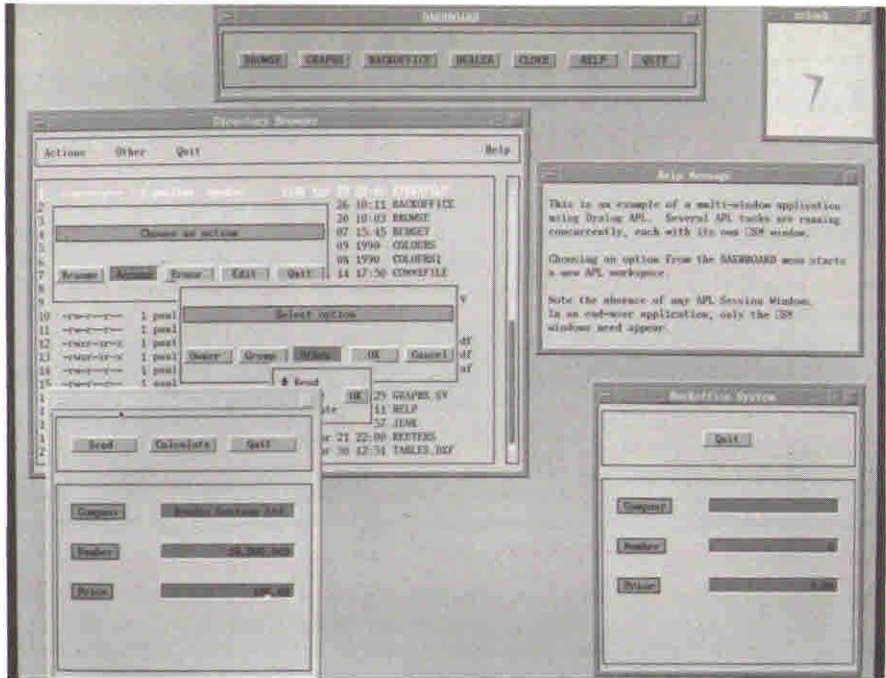


Figure 4: Photograph of DASHBOARD multi-window application

The DEALER and BACKOFFICE workspaces communicate with one another using an External Variable. This is a unique feature of Dyalog APL whereby a disk file can be associated with a variable in a workspace. When the variable changes, the associated file is automatically updated with the new value(s). When the variable is referenced, the file is read. As it is a file, the data can be shared between different users, or in this case, by different APL tasks run by the same user. Note that to avoid the cost of continually polling for a new value, the DEALER workspace notifies the BACKOFFICE workspace when something has changed by sending it a Unix signal.

BACKOFFICE sits in `DL` or `SR` using few, if any, cpu cycles. When a signal is sent by DEALER, it interrupts the `DL` or `SR` and causes BACKOFFICE to spring into action. The new data is obtained by referencing the External Variable. After processing, BACKOFFICE goes back into its wait state pending the next interrupt.

This method of communication proves to be very fast, because small, frequently used files tend to be permanently cached in main memory by Unix. What the photograph may fail to convey is that the various boxes and buttons displayed by `SM` have the same 3-D appearance as Motif and Windows 3.0. This is achieved by drawing the top and left sides of the box in a light colour, and the bottom and right sides in a dark colour. The lines used to draw the boxes also employ specially designed characters which are part of a new font supplied with Dyalog APL/X Version 6.1.

A Motif Interface

Dyadic now believes that OSF Motif is emerging as the accepted standard window manager for Unix systems. OPEN LOOK may offer certain technical benefits, and is thought by many to have superior user-interface features. Nevertheless, Motif's strong resemblance to Presentation Manager and to Windows 3.0 makes it the likely winner.

To investigate the possibilities of an interface to Motif we have developed an Auxiliary Processor (AP). This is a separate program written in C that communicates with the APL interpreter through Unix pipes. You start an AP with dyadic `□SH`. The left argument is the name of the C-program, the right argument is an optional vector of vectors containing start-up parameters.

```
'motif' □SH '' ⍝ Start the motif AP
```

The effect, which is analogous to `⌋COPY`, is to define one or more External Functions in your workspace. These have the same syntax and behaviour as regular defined functions, but are effectively entry points into compiled sub-routines in the AP. At the time of writing, the motif AP defines over 60 External Functions; one Function for each of the Motif widget set, plus several other utilities.

Before looking at how the APL→Motif interface might work, it is useful to discuss Motif concepts and examine how Motif is driven from C, the language for which it was designed. Motif consists of four components; a widget set, a window manager, a style guide (somewhat like IBM's CUA) and a user interface description language (UIL). For the purposes of this article we are concerned only with the Motif widget set.

Programming with Widgets

So what is a "widget"? Well, the text books say that a widget is a complex data structure that combines an X window with a set of procedures that perform actions on that window. Widgets are implemented using object oriented principles such as "a class hierarchy", "encapsulation" and "inheritance". For example, the PushButton widget is a sub-class of the Label widget and inherits many of its characteristics and behaviour as well as adding those of its own. The bad news is that you really have to understand all this clever stuff to be able to use widgets effectively, if only because all the documentation is written in object-oriented terms. The good news is that Motif works in roughly the same way as Windows 3.0, so that once you've learnt one system it is easy to understand the other. The basic process, which I have simplified considerably for the sake of brevity, is as follows:

1. Initialise the X Toolkit
2. Create your widgets and set their resources.
3. Register callbacks.
4. Realise the widget tree.
5. Enter the event loop.

Initialisation

Before you start, you must initialise the X Toolkit on which Motif is based. Among other things you must register the name and class of your application, as these are required by the X resource manager. Initialisation is performed by the C function `XtInitialize` which creates and returns a handle for a top-level "shell" widget.

Creating the Widget Tree

Widgets are constructed in a hierarchical tree, with the shell widget (created by `XtInitialize`) at the top. Every widget, except the shell widget, has a "parent" which "manages" it. To create a widget of a particular type or class, you can call the appropriate `xmCreate...` C routine specifying its name, its parent, and a set of "resources".

Setting Widget Resources

The term "resource" means a piece of data associated with or used by a widget. Resources are specified as {name, value} pairs. For example, to specify that you want your `PushButton` widget to display the text 'OK', you specify that the resource called "labelString" has the value "OK". In C you set up a resource list as an array of {name, value} pairs, where the value is either a number or a pointer to another data structure, such as a character string. This list is supplied as a parameter to the appropriate `xmCreate...`. You can also modify and query the value of resources after the widget has been created.

Callbacks

Most (although not all) widgets support one or more types of "callback". A callback is a particular action or event that can occur in the widget. The idea is that you can associate a list of functions with a callback. Then, whenever the particular action or event occurs, the functions are executed for you. For example, the `PushButton` widget supports three callback lists, namely `armCallback`, `activateCallback`, and `disarmCallback`. When the user presses a mouse button inside a `PushButton` widget, the functions on the `armCallback` list are executed. If the user releases a mouse button while the sprite is contained within the `PushButton` widget the functions on the `activateCallback` list are executed, followed by the functions on the `disarmCallback` list. If the user moves the sprite out of the `PushButton` widget before releasing the button, only the functions on the `disarmCallback` list are invoked. Callbacks provide the principle mechanism for your application to respond to user input. In C a callback is registered using the `XtAddCallback` routine, specifying the widget id, the callback type, the function to be added to the callback list, and (optionally) some data to pass to it when it is executed.

Realising the Widget Tree

Having created all your widgets, you have to "realise" the widget tree. This step actually creates all of the X windows that are required.

The Event Loop

This is the really interesting bit. Having initialised, created and realised your widgets, you now enter an infinite loop which performs just two steps over and over again.

```
GET EVENT
DISPATCH EVENT
```

This is normally performed by the C routine `XtMainLoop` which, once called, never exits. The only way your application gains control is when one of your callback functions is invoked as the result of some user action, such as a button press in a `PushButton` widget. It is perfectly OK for a callback function to add to or modify the widget tree in some way, so that an application is not entirely "static" in nature. But how does my program ever terminate?, I hear you cry. The answer is that a callback function has to an explicit `exit(0)` itself (like a naked $\rightarrow$ in APL). This is the only way out!!!

Incidentally, the equivalent loop in Windows 3.0 is ...

```
GET MESSAGE
DISPATCH MESSAGE
```

A PushButton Example

Figure 5 shows the listing of a C program. The program displays a `PushButton` widget which is 250 pixels wide by 150 high. It contains the legend "Press Here". If the user positions the sprite within the `PushButton` widget and presses a mouse button, the program prints the message "Button Pressed". If the user now releases the button, it prints "Button Released". Simple eh!

```
/****** C Program to Display a PushButton Widget *****/
#include <Xm/Xm.h>
#include <Xm/PushBG.h>

void main();
void armCB();           /* Callback for the Pushbutton */
void activateCB();      /* Callback for the Pushbutton */
char *btn_text;         /* button label */

void main (argc, argv)
int argc;
char **argv;
{
    Widget toplevel;     /* Shell widget          */

```

```

Widget button;      /* Pushbutton widget */
Arg args[10];       /* arg list */
int n;              /* arg count */

/* Initialize toolkit */
toplevel = XtInitialize (argv[0], "Example", NULL, NULL,
                        &argc, argv);

/* Create compoundstring for the button text */
btn_text = XmStringCreateLtoR("Press Here",
                              XmSTRING_DEFAULT_CHARSET);

/* Set up arglist for Pushbutton widget */
n=0;
XtSetArg(args[n], XmNlabelType, XmSTRING); n++;
XtSetArg(args[n], XmNlabelString, btn_text); n++;
XtSetArg(args[n], XmNwidth, 250); n++;
XtSetArg(args[n], XmNheight, 150); n++;

/* Create Pushbutton widget */
button = XmCreatePushButton(toplevel, "button", args, n);

/* Manage Pushbutton Widget */
XtManageChild(button);

/* Addcallbacks */
XtAddCallback(button, XmNarmCallback, armCB,
              "Button Pressed");
XtAddCallback(button, XmNactivateCallback, activateCB,
              "Button Released");

/* Realise Widget tree and give to user*/
XtRealizeWidget(toplevel);
XtMainLoop();
}

/***** Callbacks for the Pushbutton *****/

void armCB (w, client_data, call_data)
Widget w;          /* widget id */
caddr_t client_data; /* data from application */
caddr_t call_data;  /* data from widget class */
{
    XmAnyCallbackStruct *cb = (XmAnyCallbackStruct *)call_data;
    printf ("Callback reason = %d\n", cb->reason);
    printf ("Application data = %s\n", client_data);
}

void activateCB (w, client_data, call_data)
Widget w;          /* widget id */
caddr_t client_data; /* data from application */
caddr_t call_data;  /* data from widget class */
{
    XmAnyCallbackStruct *cb = (XmAnyCallbackStruct *)call_data;

```

```

printf ("Callback reason = %d\n", cb->reason);
printf ("Application data= %s\n", client_data);

XtFree(btn_text); /* Free button label */
exit(0);          /* and TERMINATE */
}

```

Figure 5: PushButton example in C

Figure 6 shows an equivalent Dyalog APL function which does exactly the same thing. It is probably more profitable to discuss the APL code ...

Line[1] starts the *motif* Auxiliary Processor which defines all the External Functions we need to drive Motif. Line [2] initialises the X Toolkit.

```

v BUTTON;TOP;BTN;ARGS;MSG;CB      a PushButton Example
[1] 'motif' SH                      a Start motif AP
[2] TOP←XtInitialize 'Example'      a Initialise X Toolkit
[3]
[4] ARGS←'labelType' 2              a Define resources
[5] ARGS←args,'labelString' 'Press Here'
[6] ARGS←args,'width' 250 'height' 150
[7]
[8] CB←'armCallback' 'Button Pressed'      a Define callbacks
[9] CB←CB,'activateCallback' 'Button Released'
[10]
[11] BTN←ARGS XmCreatePushButton TOP'btn' 1 CB a Create widget
[12]
[13] XtRealizeWidget TOP              a Realise widget tree
[14]
[15] Again:MSG←XtMainLoop              a Enter event loop
[16] DISPLAY MSG                      a Print message
[17] +(MSG[3]≠c'Button Released')/Again a Continue ...
[18] RESET                          a Close motif AP, exit
v

```

Figure 6: PushButton Example in APL

Lines[4 5 6] set up resources as a nested vector containing {name, value} pairs. Note that the "labelType" resource can take the value 1 (the label is a pixmap) or 2 (the label is a character string). A C programmer is expected to use global names which are defined in X header files (XmSTRING ↔ 2). Similarly, a C programmer does not specify the name of a resource as a string constant (e.g. "labelString") but uses a global name (XmNlabelString) instead. This extra level of indirection could also be employed in APL.

Lines [8 9] define callbacks. The "obvious" way to implement callbacks might have been to have the AP execute APL functions when a particular action occurs. We have (at this

stage) decided not to do this, but to have a single general-purpose callback routine in the AP which sends back a message consisting of a nested vector made up as follows:

```
{Widget ID} {callback reason} {client_data} {call_data_1}...{call_data_n}
```

where {client_data} is an APL array associated with the specific widget and callback type when the widget was created, and {call_data} is data supplied dynamically by the widget itself where appropriate. This type of interface is easier to implement and more consistent with Windows 3.0.

In this example we are asking the AP to send back 'Button Pressed' when an `armCallback` occurs, and 'Button Released' when an `activateCallback` occurs.

Line 11 creates the widget using the External Function `XmCreatePushButton`. The result `BTN` is the widget id.

All of the functions to create widgets have the same syntax. The left argument is optional and defines the resources to be set as a nested vector of {name, value} pairs. The right argument is a nested vector containing 2-4 elements:

Parent widget id	: integer
Name	: character vector
Managed flag	: 1 = manage, 0 = don't manage (optional, default 0)
Callbacks	: vector of {name, array} pairs (optional)

The "Managed Flag" determines whether or not the widget is to be managed immediately or not. If not, you must subsequently manage it explicitly using the `XtManageChild` External Function. An example where you would want to do this is a dialog box which you want to pop up (manage) and down (un-manage) dynamically.

If no callbacks are specified, a useful default which is appropriate for the widget in question is assumed. For example, the default callback for the `PushButton` widget is 'activateCallback'. By default, you will therefore be notified when the `PushButton` is clicked.

Line 13 realises the widget tree. It takes only the id of the top-level widget as its argument.

Line 15 implements the Motif event loop. This bit is rather tricky to understand, let alone explain, but is best thought of in terms of the Windows 3.0 message loop instead. Effectively, whenever one of the callbacks is fired, the External Function `XtMainLoop` returns a message in the format described above. Note that there must be *at least one* callback defined, or we would never get back from `XtMainLoop`.

Line 16 displays the message, and line 17 branches back into `XtMainLoop` unless we have received 'Button Released'. Don't even think about how the AP sorts this out (the one thing you cannot do is start the event loop again), just accept that it all works! The `RESET` function on line 18 closes the AP by `EXITING` all its External Functions.

File Selection Box Example

The PushButton widget is a very simple type of widget. An example of a far more complex widget is the FileSelectionBox, which is a so-called "composite" widget. It is however just as easy to drive from Dyalog APL. Figure 7 contains a screen photograph showing the `TEST_FILE` function being traced.

The FileSelectionBox widget allows the user to enter a file name. The user can type the name in or, using a selection mask, choose from a list of files using the mouse. In this simple example, we have defined two callbacks, `okCallback` which is actioned when the user clicks the "OK" button, and `cancelCallback` which is actioned by the "CANCEL" button. As you can see from the photograph, when the user clicks "OK" the name of the file is returned in the 4th element of the result of `XtMainLoop`. This is an example where the widget itself supplies data to the callback.

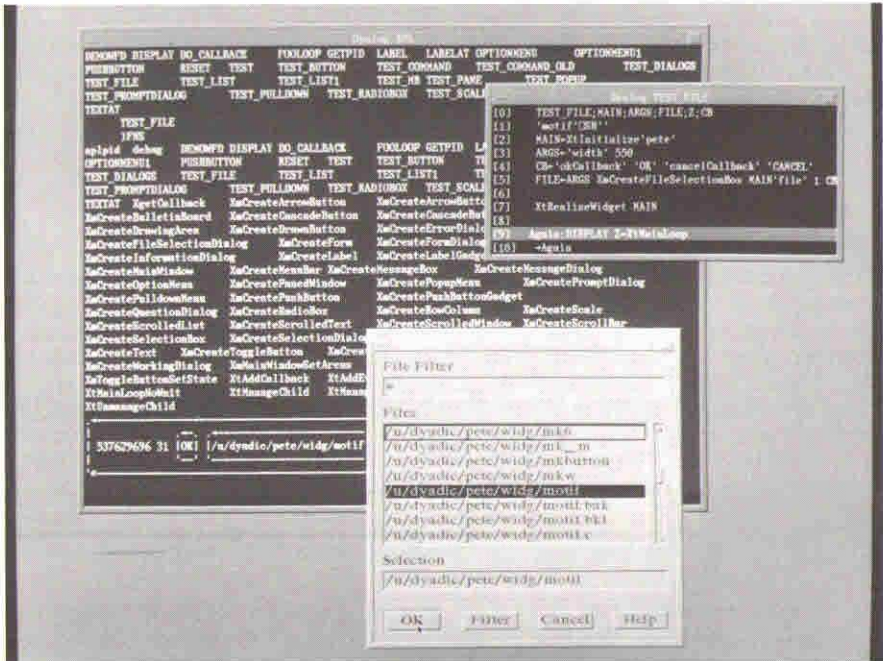


Figure 7: Photograph of the `TEST_FILE` function

Multiple Widget Example

The List widget is a general-purpose widget that lets a user choose one or more items from a list. The photograph in Figure 8 shows the function `TEST_LIST1` being traced. This function puts up a `ScrolledList` widget, and alongside it a `RadioBox` widget, for the user to choose the "selection policy". The List widgets support 4

different selection policies. You can set or change the selection policy by setting its "selectionPolicy" resource to one of 'browse', 'single', 'multiple', or 'extended'. Note that a RadioBox widget allows only one of its buttons to be enabled at any one time. The example also has 2 PushButtons labelled "Quit" and "Help".

As you can see from the photo, by the time we get to realising the widget tree we are down to line[72] of the function. OK, some of the code could have been put into sub-functions, but even so it is beginning to get a trifle long-winded.

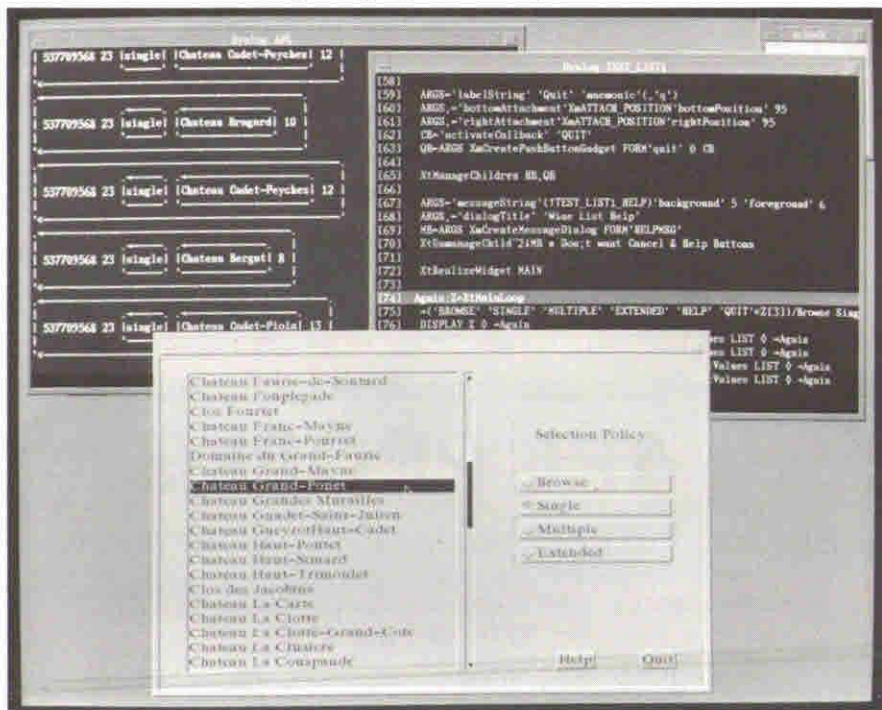


Figure 8: Photograph of the TEST_LIST1 function

So where do we go from here?

Dyalog APL's "window-based" development environment has been successfully implemented under DOS/386 in character mode, but really comes into its own under X. The Windows 3.0 implementation will take it a stage further, as it will be possible to use menus, scroll-bars and so forth that are not implemented at the X level. Indeed, Dyadic intends to demonstrate this new version at APL91.

$\square SM/\square SR$ currently provide a natural and powerful means to drive the user interface. With the simple extensions provided in Dyalog APL/X, it is possible today to generate multi-window applications with a similar "look and feel" to Motif.

The motif Auxiliary Processor proves to be a powerful and effective means for driving true Motif widgets from Dyalog APL. However, it has some disadvantages. Firstly, the Motif widget set is actually quite restrictive. It does not, for example, have anything to handle numbers. Secondly, once you start using a reasonable number of widgets, it begins to look a trifle cumbersome (although it still beats writing in C into a cocked hat).

We believe that the answer lies in extending $\square SM/\square SR$ to encompass the Motif widget set, and of course, Windows 3.0 menus, dialog boxes, etc. This will provide the best of both worlds - a consistent interface for the end-user, and a natural and concise interface for the APL programmer. This major extension of $\square SM/\square SR$ is in fact likely to appear first in the Windows 3.0 version of Dyalog APL that is being announced at APL91.

If you can get there, be sure to call in at Dyadic's booth for a demonstration.



APL PEOPLE

Serving the International APL Community

Consultancy advice and assistance with all aspects of APL systems development

Recruitment unique placement service for companies and individuals

Software Competitive prices on all APL software tools and interpreters

Contact: Jill Moss, APL People Ltd.,
The Old Malthouse, Clarence Street,
Bath BA1 5NS

Tel: 0225 462602 during office hours
0225 333618 evenings and weekends

An Interface Between APL2 and the X Window System

by John R. Jensen and Kirk A. Beaty

Abstract

This document describes APL2/X, an interface between the X Window System and APL2, designed and built at the IBM Cambridge Scientific Center. APL2/X enables all of the X Window System Xlib calls and data-structures to be used from the APL2 environment. In so doing, it enables APL2 to use a true windowing environment. Several APL2 sample programs using the interface have been coded to illustrate and validate the interface.

The X Window System is a large application written in C. APL2/X has therefore been built upon, as well as heavily influenced, a general APL2-to-C interface that runs on APL2/6000 under AIX, on APL2/370 under VM/CMS and MVS/TSO, and on APL2/PC under PC/DOS. When running under AIX or DOS, a new auxiliary processor, AP144, is used. When running under VM or MVS, APL2 will communicate with C using APL2's associated processor 11.

The APL2 AP144 X Window System interface has been announced as an integral part of the APL2/6000 product (5765-012). The APL2 X Window System interface for VM/CMS has been released as part of the TCP/IP Version 2 for VM product (5735-FAL). The other implementations remain experimental.

Overview of the X Window System

Since the X Window System is probably not that well-known in the APL2 community, a brief introduction may be in order. It is similar to other windowing systems such as Microsoft Windows and Presentation Manager in that it gives you a multitude of calls to control a windows-oriented system, handling the manipulation of the windows on the screen, drawing of text and graphics, and managing the event queue. It is the de-facto standard in the UNIX world, but it can also be used from other operating system environments as well.

The output capabilities of the X Window System can be summarized as follows:

- Managing a multitude of overlapping windows on one or more display screens.

- Drawing of graphics primitives (lines, arcs, rectangles and polygons with or without fills).
- Writing high-quality text.
- Supporting colour.
- Image support.

Inputs are received in the form of events. Events can be generated by:

- The user using the workstation keyboard.
- The user manipulating the mouse (moving it, using the mouse buttons).
- Other events, such as when a window is cleared and needs to be redrawn.

Other important facts about the X Window System:

- Network transparent - An application running on one machine can display on the same machine, or another.
- Vendor independence - The underlying display hardware is hidden by the protocol from the application, enhancing portability.
- Concurrency - Several applications active on the workstation at any one time.
- Information sharing across applications.

The X Window System is separated into three major components:

Component	Use
Client:	The client services represent the interface to the application programs. It consists of all the calls that the application programs can use to invoke the various services of the X Window System. It communicates with the X Window System server through the defined X Window System network protocol.
Server:	The server receives the network protocol requests, and acts upon them, drawing windows, graphics, text and images on the display. It also generates event messages whenever the user performs an action such as typing a letter, moving the mouse, etc., and returns these events to the user via the client services.
Window Manager:	The window manager controls the appearance of the various windows on the display. Different window managers exist. A window manager now in common use is the OSF/Motif window manager that gives the X Window System the same "look and feel" as that of the OS/2 Presentation Manager.

These three components of the X Window System need not run on the same physical machine. As shown in Figure 1, the application may be running on, say, a VM host, communicating through the X Window System client services with an X Window System server running on a RT PC or a RISC System/6000. The information flow between these machines is handled by TCP/IP. The net effect of this is that the application's results appear on the workstation's display as though the application had been run locally.

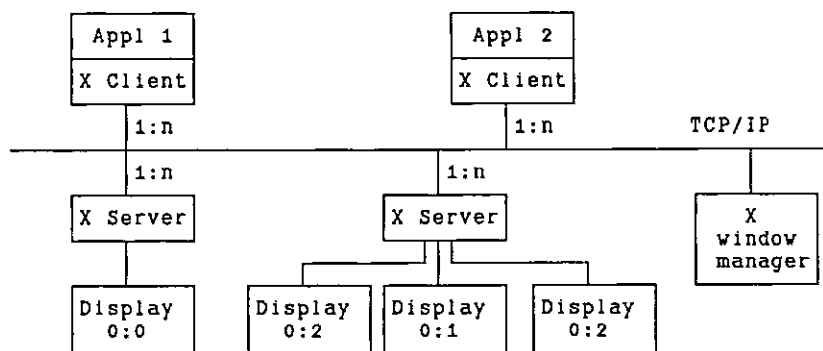


Figure 1. The X Window System client-server relationship

It is also possible for a single client to display on many servers at once. Likewise, a server can also service many clients at the same time, displaying the output of many application programs at once. Figure 1 illustrates the relationship.

The X Window System defines a layered architecture. Figure 2 shows the layers of the hierarchy. An application can draw upon the calls of all of these layers.

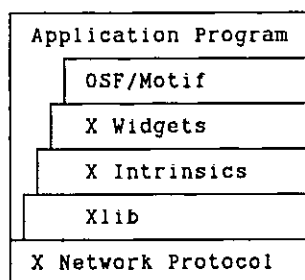


Figure 2. The X Window System hierarchy

The X Window System network protocol is at the base of this hierarchy, ultimately defining the traffic flowing between the client and server components.

The next level up is the Xlib level. Most application programmers will never interface with the X Window System at a lower level than the Xlib level. It consists of about 400 separate calls written in C, and more than a 100 data structures.

The next two layers, the X Intrinsics and the X Widgets, form sets of routines that create higher-level abstractions. These are then used as a basis for a number of X-based toolkits. A number of standard toolkits are provided with the X Window System, supplied by various vendors. *OSF/Motif* can be viewed as one of the more elaborate toolkits in existence today. It provides calls to handle such commonly occurring functions as menus, scrollbars, pop-up windows and the like.

Using the X Window System from APL2

APL2/6000 using the X Window System in native mode:

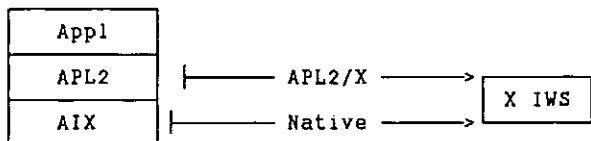


Figure 3. Using the X Window System from APL2/6000

APL2/6000 has several ways that it uses X Window System. The session manager (AP120) uses it, as does the fullscreen manager (AP124).

APL2/VM using the X Window System in compatibility mode:

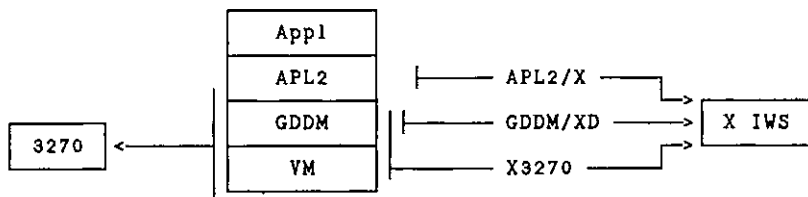


Figure 4. Using the X Window System from APL2/VM

The simplest way to use the X Window System today from APL2 is to use it in compatibility mode. Thus, the existing interfaces remain unchanged, but the display will now take place on the X Window System workstation. Two possibilities exist to do this:

X3270 is a terminal emulator that enables you to run a 3270-type session on an X Window System workstation, given the proper network attachments. It supports different size fonts including APL2, GDDM-style graphics, 3277GA emulation and some mouse-support. X3270 is an announced IBM product.

GDDM/XD is an interface which permits the display of graphics output from GDDM on workstations supporting the X Window System. It is available as part of the TCP/IP product for both VM and MVS. It displays both character and graphics output in a separate window on the X Window System workstation.

Exploiting the X Window System from APL2

APL2/X takes a different approach to the X Window System. In order to exploit fully the X Window System from the APL2 environment it is essential that APL2 have direct access to the basic X Window System calls. How this is achieved will be the topic for the rest of this paper.

All the X Window System calls are defined in a table, along with parameter type codes that are used to validate argument inputs and generate call outputs. All calls to the X Window System from APL2 are done using the single APL2 function *XWIN*, rather than define APL2 functions for each separate X Window System call.

The *XWIN* function uses the associated processor 11 in the VM environment to communicate with C, and the new auxiliary processor AP144 in the AIX environment, but this usage is completely transparent to the user. To him or her there is a single, common interface to the X Window System from APL2.

`(rc [results]) ← [c] XWIN command [parm] [parm]...`

command . . . The name of the X Window System call to be invoked, specified as an APL2 character vector.

[*parm*] All but a few of the X Window System calls require additional input parameters to be specified. These are given after the name of the call itself, in the same order as listed in the X Window System documentation.

[*c*] An optional vector of up to two elements controlling the behaviour of the *XWIN* function. The two elements are orthogonal to each another:

The first element determines what is returned by the function:

- c* = 0 *x* The function never returns anything
- c* = 1 *x* Return an integer return-code *rc*
- c* = 2 *x* Return any result data produced by the call, if any.
 If no data is produced, nothing is returned in *z*.
- c* = 3 *x* Return a one- or two-element vector. The first element
 always contains the return-code. The second element, if
 present, contains the result data produced by the call.

The second element determines what happens if the call results in an error:

- c* = *x* 0 Ignore any errors
- c* = *x* 1 Generate a message to the console listing the cause of the
 error
- c* = *x* 2 Halt execution on occurrence of an error
- c* = *x* 3 Generate a message to the console listing the cause of the
 error, followed by halting the execution of the program.

If only the first element is given, the second is assumed equal to 0. *c* can be omitted entirely. The behaviour will then be equivalent to a setting of "2 3".

[*rc*] The operation return-code.

[*results*] . The output from the call (if any) is returned in the form of an explicit result. This result includes the X Window System explicit result (if any), as well as any implicit results passed back via output parameters given on the call. More on this topic below.

The different control options caters to many different possibilities. If called with a setting of "3 0", then *XWIN* will pass back an error code as part of the function return. It is then up to the calling program to check this code and take appropriate action on a non-zero return-code.

If, on the other hand, *XWIN* is called with a control setting of "2 3", then *XWIN* will in effect provide a default error handler to check the return-codes as they are returned from each call to APL2/X. If an error is encountered, then *XWIN* will suspend operation in the function to give the programmer a chance to correct the problem.

The call `XOpenDisplay` can serve to illustrate the close correspondence between the X Window System calls issued from C and from APL2. In C, a call to this might look like:

```
dp = XOpenDisplay("");
```

The same call can be issued from APL2 via APL2/X:

```
dp ← XWIN 'XOpenDisplay' ''
```

The X Window System call name becomes the first parameter specified on the call through APL2/X. Apart from this, everything else remains the same.

HelloWorld

It seems as though it is a rite of passage for a windows-based package to have a HelloWorld sample program; APL2/X follows this trend. We will use the APL2 function *HelloWorld* to introduce many of the concepts of APL2/X.

The goal of *HelloWorld* is to create a window on the workstation, display some text, and then wait for the user to initiate any further processing. The program is notified of the user's actions. It responds to three different types of actions on the part of the user:

- **Pressing the left mouse button:** When this button is pressed, the text "Hi!" will be written at the tip of the pointer.
- **Pressing any alphanumeric key:** The key symbol is written in the window at the tip of the pointer, except in the case of a "q" being pressed. A "q" terminates the application, closing the window.
- **A window expose:** This event is generated whenever the window is uncovered. It is a signal to the application to redraw the initial content of the window. Any text written to the window as a result of either of the two actions mentioned above are lost.

HelloWorld exhibits two fundamental concepts of windows-based systems: window manipulation and responding to user-generated events. As such, it serves as an excellent vehicle to introduce these concepts. The discussion here will focus on how to run the HelloWorld program from the APL2 environment, with each X Window System call being generated from APL2. The design of this program is described in:

Oliver Jones: Introduction to the X Window system
Prentice Hall, 1989; ISBN 0-13-499997-4

See this text for a full description of each of the X Window System calls.

Initialization

```

      ▽
[0]   HelloWorld;␣IO;dp;w;gc;s;e;k;rw;bp;wp;m;hello;
      hi;done;None;hp;hints;rc;nl;x;ep
[1]   ␣ Sample X program, based on helloworld.c from
[2]   ␣ Oliver Jones:
[3]   ␣ Introduction to the X Window system
[4]   ␣ Prentice-Hall, 1989; ISBN 0-13-499997-4
[5]   ␣IO+0
[6]
[7]   ␣ Define some constant text-strings
[8]   hello+Hello, World.'
[9]   ␣ The exclamation point makes hi ugly:
[10]  hi+'Hi',('A'=␣AF 65)=␣AF 90 33
[11]

```

The initial section of the functions merits few comments. All that happens in this section is the setting up of the text strings that will be shown in the window later on, and definition of some local identifiers. The only note-worthy item is the way the exclamation point has to be defined. This ensures portability of the code.

Connect to X Window System

```

[12]  ␣ Initialization
[13]  +(0=dp+XWIN 'XOpenDisplay' '')+lopen
[14]  ␣+'XOpenDisplay failed ... HelloWorld aborted'
[15]  →lexit
[16]  lopen:
[17]

```

The first call to X Window System is to open a connection to the display. The call results in a 'handle' or magic number that will be used to identify the connection in all subsequent X call requests. The handle is saved in the *dp* variable.

Notes:

- All X Window System calls pass through the *XWIN* function.
- The first parameter to this function is always the name of the X call.
- If the X call requires parameters to be specified, then these are given following the name of the call. The *XOpenDisplay* call has one parameter: the name of the display to be used. This is given as an APL2 character string. (The default display will be used, as we have specified an empty string.)
- Character strings are passed (and returned) by value. APL2/X will convert these parameters to the C char * pointers that X ultimately requires. APL2/X also handles NULL-termination of the character strings.

- All output from invoking X Window System calls are returned as explicit results of *XWIN*.

The HelloWorld function as described performs minimal error checking. This should obviously be changed for programs and applications targeted for a production environment.

Get more information

```
[18]  a Default pixel values
[19]  s+XWIN 'XDefaultScreen' dp
[20]  bp+XWIN 'XBlackPixel' dp s
[21]  vp+XWIN 'XWhitePixel' dp s
[22]
```

Once the connection to the display is established, X can provide the calling program with a lot of information. Here we retrieve the number designating the default screen and the pixel values for black and white.

Notes:

- All input and output parameters on these three calls are integers. They are passed to (and returned from) X Window System by value.
- APL2/X handles any required conversion between boolean and integer data types. Floating point numbers cannot be used.

Using X constants

```
[23]  a Define an X constant[24]  None+0
[25]
[26]  a Prepare to set window position and size
[27]  (rc nl)+'H' axGetFF 'XSizeHints'
[28]  m++/PPosition PSize
[29]
```

X Window System defines a large number of named constants in its C header files. These constants should also be available to APL2. However, there are too many constants defined to keep them all in the workspace. (Although technically feasible, it would define so many symbols in the workspace that it would obscure the "real" application variables.)

The simplest solution to this problem is to specify these constants by value rather than by name. However, by doing so we lose a lot of the information content associated with the constant, and we increase any future maintenance burden.

Another fairly simple solution is to define the needed constants as APL2 variables, and then to use these variables whenever the constant is called. The variables can either be defined as global variables, or as local variables within the applicable functions, as is done with *None*. This is not a recommended solution, though, as it relies on the constants staying forever unchanged. (*None* will be used further down in the function, in the call to *XSetStandardProperties*.)

A better way is to use the support for constants built into APL2/X. APL2/X groups the constants by categories. Each group can be brought into the workspace independently of other groups, ensuring that the latest values are used. The *axGetFF* function loads a named group of constants and establishes them as APL2 variables in the workspace. *PPosition* and *PSize* are examples of two such constants.

Notes:

- *axGetFF* returns a list of names of constants established in the workspace. We will use the list later on to clear away the variables when they are no longer needed.
- The name of the group of constants is specified as the right argument to *axGetFF*. This name is in fact the name of an X data structure. We will return to this topic below.
- *PPosition* and *PSize* each contain 32 individual bits, although given to APL2 as four-byte integers. The simple adding works here because their bits are orthogonal. In the general case this cannot be assumed to be the case, so a combination of decode, logical operation and encode is the advisable course to take, e.g.:

```
m←2⊖⌈(32ρ2)⌈PPosition PSize
```

Creating a new structure instance

```
[30]  ⍝ Build an XSizeHints structure instance
[31]  hp←XWIN 'New' 'XSizeHints'
```

APL2/X provides extensive support for X Window System data structures. A number of these commands exist to help you manipulate the X Window System data structures from APL2. The command is specified as the first parameter, and the structure name as the second parameter. Additional parameters will often be required; these follow the structure name on the call.

In this case, `New` allocates space for a new instance of the `XSizeHints` structure and returns the address of this instance to APL2. The structure instances are stored in C free storage space (i.e. via calls to the `C malloc()` routine).

Importing a structure into APL2

```
[32]  hints←XWIN 'Get' 'XSizeHints' hp
```

To be used from within the APL2 workspace the structure must be imported into the workspace. This is done by using the `Get` command. This command copies the values of the structure into an APL2 vector. Each field in the structure is mapped to an element in the vector in the same order as in the X Window System structure definition. APL2/X handles the appropriate data conversions when copying the data.

Changing the values in a structure (within APL)

```
[33]  hints[H~flags H~x H~y]←m 200 300
[34]  hints[H~width H~height]←350 250
```

Once copied into the workspace the structure instances can be used like any regular APL2 array. Here five of the fields are updated.

The indices of the elements to be changed are given by constant variables prefixed by `H~`. These constants were generated by the `axGetFF` function when the `XSizeHints` structure definition was loaded in line 27. To recall:

```
[27]  (rc nl)←'H~' axGetFF 'XSizeHints'
```

The `axGetFF` function establishes a number of global variables in the workspace. These variables can be divided into two separate groups:

Field indices: . These variables are used to indicate the location of a given field in a structure. They provide a rough equivalence between the C structure member operation `hints.mask` and the APL2 version `hints[H~mask]`.

Since structure field names all tend to be very common identifiers, eg. "x", "name", or "size", it is possible (and advisable) to specify a prefix that will be applied to each field name before it is established in APL2. This avoids name conflicts between the various structures' field-name index variables. "H~" is the prefix used in the example given here.

Constants: . . . We met these earlier on. They are combined with the structure definitions because they often are used in conjunction with these. It is common that the valid values for a given field in a structure are given by constants, but they can be put to whatever use is wanted. Names of constants are **never** changed; i.e. the prefix does not apply to constants.

Updating a structure instance (within C)

```
[35]  XWIN 'Put' 'XSizeHints' hp hints
[36]
```

Changing the values of a structure variable only has effect within APL2. The Put command is necessary to make the updated structure values available to the X Window System calls.

The Put command expects exactly three parameters:

1. the name of the structure (e.g. *'XSizeHints'*),
2. a pointer to an already defined storage location (e.g. *hp*), and
3. the data values to be stored in the structure instance (e.g. *hints*).

Creating a simple X Window System window

```
[37]  a Window creation
[38]  rw+XWIN 'XDefaultRootWindow' dp
[39]  x+hints[1 2 3 4],5 bp wp
[40]  w+XWIN 'XCreateSimpleWindow' dp rw,x
```

We are now in a position to create our application window. This is accomplished with these two calls. From an APL2/X point of view these are both very ordinary calls.

Note: The *hints* values can be used like any other APL2 variable. Here four of them are used to specify the window start and extent.

Using the structure pointer

```
[41]  x+hello hello None('A' 'test')2 hp
[42]  XWIN 'XSetStandardProperties' dp w,x
```

The *XSizeHints* structure created is now put to use. As the name of the call implies it defines defaults for a number of the window attributes.

Notes:

- **XSetStandardProperties** uses an **argv** parameter. This type of parameter is specified from APL2 as a vector of character vectors. (*argv* and *argc* are well-known C variable types. X uses them in several of its calls).
- The **argv** parameter is followed by an **argc** parameter. The information content of this parameter is in a sense redundant in APL2, since the information can be determined by examining the *argv* variable. One of our general design rules in building the APL2/X interface has been to maintain a fidelity to the standard X Window System call syntax, even though at times it means specifying more information in the call than is strictly necessary.

Freeing the structure after use

```
[43]  XWIN 'SFree' 'XSizeHints' hp
[44]
```

Note: It is important that the structure is freed after it is no longer needed. If this is not done, it will continue to occupy space in storage, and may lead to storage fragmentation in the long run.

- The name **SFree** is used in place of **Free** so as not to conflict with the **free** associated with the C **malloc** function.

Creating a Graphics Context

```
[45]  a Graphics Context creation and initialization
[46]  gc+XWIN 'XCreateGC' dp w 0 0
[47]  XWIN 'XSetBackground' dp gc bp
[48]  XWIN 'XSetForeground' dp gc wp
[49]
```

A graphics context is the X Window System version of a magic paintbrush. It is used on any drawing calls to define colours, patterns, and like items. We define one here, so we can use it later on to write some text to the window.

Show the window outline

```
[50]  a Window mapping
[51]  XWIN 'XMapRaised' dp w
[52]
```

We are now ready to show the window to the user. This call will do just that. But note that we have yet to write any text into it, nor are we quite ready to accept input from the user just yet.

Which events are we interested in

```
[53]  * Input event selection
[54]  m+ 'ButtonPressMask' 'KeyPressMask'
[55]  m+m, c 'ExposureMask'
[56]  (rc m)+m axGetFF1 'XEvent'
[57]  XWIN 'XSelectInput' dp w(+/m)
[58]  ep+XWIN 'XGetEventBuffer'
[59]
```

XSelectInput tells the X Window System which events we are interested in being notified about. It is a straight-forward call, except that the last parameter is a bit-mask. The various constants defining the individual bits are defined in the **XEvent** structure. We could now use the **axGetFF** function introduced earlier to load in all fields and constants, and then use the variables thus established. However, we chose to take slightly different route:

The **axGetFF1** function is used in place of **axGetFF**. Instead of establishing variables based on the names of the fields and constants it returns the values of a set of named items. Since we are only interested in three specific constants, we use **axGetFF1** to retrieve their values.

- The values retrieved represent individual bits. It is therefore possible to just add them instead of performing a bit-wise "or".
- Bit-flags are returned in "packed" form, i.e. as integers.

The **XGetEventBuffer** establishes a pointer to the buffer used to hold events. The same buffer is used for all events, hence we only need to make this call once per session. More on this below, in the discussion of the key events.

Get the event codes

```
[60]  * Get some more constants
[61]  m+ 'KeyPress' 'ButtonPress'
[62]  m+m, 'Expose' 'MappingNotify'
[63]  (rc m)+m axGetFF1 'XEvent'
[64]  * ... and some event structure layouts
[65]  nl+nl, 1> 'K_' axGetFF 'XKeyEvent'
[66]  nl+nl, 1> 'B_' axGetFF 'XButtonEvent'
[67]  nl+nl, 1> 'E_' axGetFF 'XExposeEvent'
[68]
```

X Window System notifies us of the user's actions by returning events to us upon our requesting them. The type of events will be limited to ones enabled by the **XSelectInput** call just executed. One of the fields in the returned event will specify the type of event that is being returned. These event types are also

defined in the XEvent structure. To be able to distinguish the returned events by type we set up a vector with the expected types with the *axGetFF1* function.

We also load in the field definition variables describing the event structures we might receive. They will help us access the right fields in the structures.

The event loop

```
[69]  * Main event-reading loop
[70]    done+0
[71]    levent:=(done=0)+lend
[72]
```

We now sit in an event loop, responding to the incoming events as they come in from the display, until done.

XNextEvent

```
[73]  * Read and process the next event
[74]    x+!KeyPress !ButtonPress !Expose !MappingNotify
[75]    →(m=K_type⇒e←XWIN 'XNextEvent' dp)/x
[76]
```

XNextEvent returns the next event in the queue to us. Once the event has been returned the event type is compared with the mask defined earlier to set up the proper branch. Effectively this serves as a "Case-on-Event" statement. The event is returned as a vector of structure fields, much the same way that the structure Get call would return a structure instance.

This differs from the way that the XNextEvent call is handled in X Window System. In native X Window System, the XNextEvent call also specifies a second parameter, a pointer to an XAnyEvent structure. XNextEvent fills in the structure fields, thereby making the event values available the calling program. This is an example of how APL2/X differs from native X Window System by automatically handling the call's output parameters.

Handling the Expose events

```
[77]  !Expose: * Repaint window on expose events
[78]    →e[E_count]+levent  * Count > 0 ?
[79]    x+e[E_display E_window],gc,50 50
[80]    XWIN(c'XDrawImageString',x,hello(ρhello)
[81]    →levent
[82]
```

Expose events are generated whenever a window is made visible on the screen. Multiple expose events may be generated, so it makes sense only to respond to the last one. X Window System caters for this approach by providing a count of future, already pending expose events as one of the fields in the expose event itself. We use this field to ignore all but the last expose event.

When we receive the final expose event we want to write the content of the *hello* variable to the screen. This we do using the *XDrawImageString* call.

- All events provide the value of the originating display connection in the *e[E_display]* item in the event structure. This identifies the origins of an event, in case of multiple, concurrent display connections have been established. Here its value equals that of *dp*.
- Most (but not all) events also specify the window that generated the event (some events do not come from windows). This equals the value of *w* here.
- The last parameter (*(phi)*) may seem superfluous. This is certainly the case in the APL2 environment, where the information could be had from the preceding parameters. Again, as stated earlier, APL2/X sticks as closely to the defined syntax of the X Window System calls as possible. In this case, the inconvenience of having to specify this size field did not outweigh the desire to maintain fidelity to the X Window System call syntax.

Handling the ButtonPress events

```
[83] 1ButtonPress: a Process mouse-button presses
[84]  x←e[B_display B_window],gc,e[B_x B_y]
[85]  XWIN(←'XDrawImageString'),x,hi(phi)
[86]  →levent
[87]
```

The comments here mimic those made for Expose events. The only noteworthy change is the use of *events[B_x B_y]* to indicate the position where the writing is to start. This coordinate pair is returned as part of the ButtonPress event structure, and indicates the position of the tip of the mouse pointer when its button was pressed. The net effect of this code fragment is to write the text "Hi!" at the mouse pointer when the mouse button is pressed.

Handling the KeyPress events

```
[88] 1KeyPress: a Process keyboard input
[89]  →(done+(+k+←1>XWIN 'XLookupString' ep)←'qQ')+levent
[90]  x←e[K_display K_window],gc,e[K_x K_y]
[91]  XWIN(←'XDrawImageString'),x,k(phi)
[92]  →levent
[93]
```


KeyPress events are generated when the user presses any of the keys on the keyboard. However, information about the key pressed is not part of the returned event structure, so we need to use the `XLookupString` call to acquire this information. This call has one input parameter, the event whose key we want to ascertain. This parameter can be specified either as the set of values returned on the `XNextEvent` call, or using the pointer to the event buffer that we set up earlier on. The latter method is faster, as it does not involve passing a lot of information through the interface, so we use it here.

The key pressed is returned as the second element of the `XLookupString` return. We test on this field to see if it matches a "q". If so, we terminate the program. If not, we print the key character at the current mouse pointer location.

All event structures are exactly as defined by X Window System, except for the `XKeyEvent` structure returned by the KeyPress (and KeyRelease) events. The `XKeyEvent` structure contains one additional field not found in the X Window System definition: the character representation of the key. This last field is named "key" and is appended to the end of the structure.

Mapping Notify

```
[94]  lMappingNotify: a Reset keyboard
[95]  XWIN 'XRefreshKeyboardMapping' e
[96]  +!event
[97]
```

MappingNotify events are generated when the keyboard mapping needs to be re-established (i.e. when it has been changed elsewhere). See the referenced text for more information on this call.

Termination

```
[98]  lend: a Termination
[99]  XWIN 'XFreeGC' dp gc
[100] XWIN 'XDestroyWindow' dp w
[101] XWIN 'XCloseDisplay' dp
[102] nl+EX"nl
[103] lexit:      v  1991-4-16 18.43.0 {GMT-4}
```

Before terminating the *HelloWorld* sample program we free up the resources we have used. Some of these resources are controlled by X Window System, so the X Window System must free them.

We also clear the workspace of all the APL2/X-defined variables. These could conceivably be saved with the workspace, obviating the necessity of loading

them every time the application is run. A production level application probably should take this latter approach, unless the underlying structures change very frequently.

Current Status

At the time of this writing, all the X Window System Xlib calls have been implemented. The full interface is available as AP144 in the APL2/6000 product (5765-012), and as part of the TCP/IP Version 2 for VM product (5735-FAL).

Where do we go from here? A number of possibilities are possible as experimental work in the Cambridge Scientific Center. The following is a list of some of these:

- Port APL2/X to MVS.
- Implement selected higher-level calls from the Toolkit and OSF/Motif.
- Build an APL2 workspace of functions to make better use of the X Window System calls. The present set of functionality is very rich, but very tedious to work with. It is like using GDDM directly (via AP126) to generate 3270 panels, without the use of any APL2 cover functions, except that the problem is further magnified.
- Enable other packages written in C to be used from APL2. Since APL2/X is built upon a general APL2-to-C interface, these possibilities are made available. One such set of functions would be the TCP/IP communications protocol.

Trademarks

X Window System is a trademark of Massachusetts Institute of Technology.

Microsoft is a registered trademark of Microsoft Corporation.

Windows is a trademark of Microsoft Corporation.

OS/2 is a registered trademark of International Business Machines Corporation.

Presentation Manager is a trademark of International Business Machines Corp.

UNIX is a trademark of UNIX System Laboratories, Inc.

OSF/Motif is a trademark of Open Software Foundation, Inc.

RT PC and **RISC System/6000** are trademarks of International Business Machines.

Index to Advertisers

APL People (half)	124
Cocking and Drury	84
Dyadic Systems Ltd	65,72
MicroAPL	2
Optima	6,10,96

All queries regarding advertising in VECTOR should be made to Alison Chatterton, at the address on the inside back cover.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Jonathan Barman,
Hill Top House,
East Garston,
NEWBURY, Berks RG16 7HD
Tel: 048839-575

(not after 10.00pm please!)

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members news) should be sent to Vector Production, c/o Adrian Smith, Brook House, Gilling East, YORK Tel: 04393-385 (6.00pm - midnight).

Product Guide updates should continue to go to Alison Chatterton, as should requests for advertising space.

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box): 91/92

UK private membership	£12	☐
Overseas private membership	£20	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£100	☐
Corporate membership overseas	£155	☐
Sustaining membership	£430	☐
Non-voting student membership (UK only)	£6	☐

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____ | ____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

Signature: _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Access or Visa number. Send the completed form (no stamp required from the UK) to:

British APL Association, FREEPOST (SG923), 9 Oak Grove, HERTFORD, SG13 8BR

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1991/92 Committee

Chairman:	David Eastwood 071-922 8866	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Secretary:	Anthony Camacho 0727-860130	2 Blenheim Rd, ST ALBANS, Herts AL1 4NR.
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Jonathan Barman, 048839-575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Activities:	Dr Peter Branson 081-848 8989	Electronic Data Systems, Stockley Park, UXBRIDGE, Middx UB11 1BQ
Education:	Dr Alan Mayer 0792-295296	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Technical:	Peter Donnelly 0256-811125	Dyadic Systems Ltd., Riverside View, Basing Road, Old Basing, BASINGSTOKE, Hants RG24 0AL
Projects:	John Searle 081-858 6811	4 Hawks Mews, Greenwich, LONDON SE10 8RA
Publicity:	Misha Jovanovic 0753-853141	99 Oxford Road, WINDSOR, Berks SL4 5DX
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse, Clarence St., BATH, Avon BA1 5NS
Administration:	Alison Chatterton 0992-552489	9 Oak Grove, HERTFORD, SG13 8AT

Journal Working Group

Editor:	Jonathan Barman	048839-575
Secretary:	Anthony Camacho	0727-860130
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	John Searle (081-858 6811), Ray Cannon (0252-874697), Sylvia Camacho, Bridget Barman, Gill Smith	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Malthouse
Clarence St, BATH, BA1 5NS
Tel: 0225-482602

Intelligent Programs Ltd
Unit 7, Hermitage Court
6-10 Sampson St
LONDON, E1 9NA
Tel: 071-481 4813

Compass R&D Ltd
15 Frederick Sanger Rd
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel: 0483 302249
Fax: 0483 302279

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 0256 811125
Fax: 0256 811130

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON, EC4Y 0HA
Tel: 071-353 4212
Fax: 071-353 3325

Cocking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel: 071-436 9481

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 071-922 8868

REUTER:FILE
1-4 Singer St
LONDON EC2A 4BQ
Tel: 071-867 1166
Fax: 071-867 9792

Peter Cyriax Systems
12 Gloucester Place
LONDON W1H 3AW
Tel: 071-935-2983



The British Computer Society, 13 Mansfield Street, London W1M 0BD.